

Applications of Neural Networks for FMCW Automotive Radars

Techniques for Improving Detection and DoA Estimation

MÁRCIO LUIZ LIMA DE OLIVEIRA

Applications of Neural Networks for FMCW Automotive Radars

TECHNIQUES FOR IMPROVING DETECTION AND
DOA ESTIMATION

MÁRCIO LUIZ LIMA DE OLIVEIRA

Applications of Neural Networks for FMCW Automotive Radars

TECHNIQUES FOR IMPROVING DETECTION AND DOA
ESTIMATION

DISSERTATION

to obtain
the degree of doctor at the University of Twente,
on the authority of the rector magnificus,
prof. dr. ir. A. Veldkamp,
on account of the decision of the Doctorate Board
to be publicly defended
on Tuesday, 13th of May of 2025 at 16:45 hours

by

Márcio Luiz Lima de Oliveira

born on the 21st of June, 1990
in Salvador-BA, Brazil

This dissertation has been approved by:

Promotor: prof. dr. ir. M. J. G. Bekooij

Copromotor: dr. ir. N. Alachiotis

Cover design: M. Orrico and M. L. Lima de Oliveira

Printed by: Ipskamp Printing

Lay-out: M. L. Lima de Oliveira and V. el Hakim

ISBN (print): 978-90-365-6600-1

ISBN (digital): 978-90-365-6601-8

DOI: 10.3990/1.9789036566018

© 2025 M. L. Lima de Oliveira, The Netherlands. All rights reserved. No parts of this thesis may be reproduced, stored in a retrieval system or transmitted in any form or by any means without permission of the author. Alle rechten voorbehouden. Niets uit deze uitgave mag worden vermenigvuldigd, in enige vorm of op enige wijze, zonder voorafgaande schriftelijke toestemming van de auteur.

Graduation committee:

Chair/Secretary:	Prof. dr. ir. B. R. H. M. Haverkort	University of Twente
Promotor:	Prof. dr. ir. M. J. G. Bekooij	University of Twente
Co-promotor :	Dr. ir. N. Alachiotis	University of Twente
Committee Members:	Prof. dr. ir. A. B. J. Kokkeler	University of Twente
	Prof. dr. ir. F. E. van Vliet	University of Twente
	Prof. dr. H. Corporaal	Eindhoven University of Technology
	Dr. F. Fioranelli	Delft University of Technology

This work has been conducted within the ZERO Energy project P3: Autonomous Roadside Monitoring, sponsored by NWO in the Netherlands.

*It was carried out at the
Computer Architecture for Embedded Systems (CAES) group, Faculty of Electrical Engineering, Mathematics and Computer Science (EEMCS), University of Twente, The Netherlands.*

UNIVERSITY OF TWENTE.

*"Imagination will often carry us to worlds that never were.
But without it we go nowhere."*
— Carl Sagan

Acknowledgements

The journey of building this thesis and pursuing a Ph.D. has been a formidable and transformative experience, one that would not have been possible without the relentless support and contributions of many people. This monumental task has tested my limits and pushed the boundaries of my knowledge, challenging me in unexpected ways. To face these challenges, I have been fortunate to have so many people by my side, supporting, guiding, and pushing me further than ever before. Their collective support is also expressed on all the written pages of this thesis.

First and foremost, I would like to start by expressing my gratitude to my promotor Prof.dr.ir. Marco Bekooij for not only providing me with the opportunity to embark on this academic journey in the Netherlands, but also for his guidance and mentorship. Your insights helped me to become sharper throughout these years, and your dedication made me grow as a researcher. I often tell people, when prompted about doing a Ph.D., that having a supervisor who aligns with your needs and expectations is crucial for a successful academic life, and I am incredibly grateful to have had such a supervisor. Now, as the end approaches, I cannot help but recall your words from my acceptance email: “I hope that pursuing a Ph.D. will be a challenging but memorable journey for you,” and I must say, you could not have been more accurate in your foresight. ‘Bedankt voor alles’.

I also would like to extend my thanks to my committee members, Dr.ir. Nikolaos Alachiotis (co-promotor), Prof.dr.ir. Andre Kokkeler, Prof.dr.ir. Frank van Vliet, Prof.dr. Henk Corporaal, and Dr. Francesco Fioranelli for accepting to be on my committee, reviewing, evaluating, and for their critical thinking about this work. Their expertise and commitment to excellence have enriched this thesis.

A special thanks to my master’s supervisors in Brazil Prof.dr. J  s Cerqueira and Prof.dr. Eduardo Simas. J  s, without you I would have never left Brazil in pursuit of a doctoral degree. Your help and contacts were immeasurable for me and will never be forgotten.

My sincere gratitude to my colleagues and fellow Ph.D. candidates, as having good company is an essential ingredient for success. I would like to thank all the

Acknowledgements

people in the CAES and DACS groups, especially Oğuz, Bayer, and Konstantinos. Our first-year lunches at Waaier will be forever in my memory.

I also would like to thank everybody from the P-NUT association for their hard work and companionship for the doctoral candidate community. Roberto, Pieter, Rob, Anton, Kevin. Thank you all for strengthening the community and for all the good times.

Nevertheless, life within the university is just half the equation for an enjoyable pursuit of a doctorate degree. Life would have no meaning without being surrounded by so many remarkable individuals from diverse backgrounds and countries. Thank you to all my dearest friends that I left in Brazil but who never abandoned me. Flávia, Ribeiro, Sara, Carlos F., Danilo P., Bruno, Mateus, and Bruna.

Building new connections in foreign lands can be daunting, but it has also been exceptionally rewarding. I would like to express my gratitude to all my international and Brazilian friends whom I have had the privilege of meeting over the past few years. Your presence and support have kept me sane, particularly during the challenging times of the pandemic. Isadora, Livia, Felipe, Pramod, Giulia, Fernanda, Mauricy, Lorena, Diogo, Inês, Gabriel, Danilo C., Carolina, Victor, and Adriano. you have been a source of comfort in this journey, and for that, I am immensely thankful. I also would like to extend my appreciation to everybody else in our little Brazilian group in Enschede.

I would like to sincerely thank both Fernando and Raian for their invaluable friendship, support over the past years and for kindly accepting to be my paranymphs during my defense. You made this journey even more memorable.

As I acknowledge the vital role my friends have played in shaping my doctoral journey, it is of greatest importance to recognize the unconditional support of my family throughout these years. After all, behind every successful story lies strong family support. To my parents, thank you for always standing by my side and for never giving up on me. To my father, João, you are the eternal source of my inspiration and ambition. As a professor doctor yourself, I know this means as much to you as it means to me. Thank you for never giving up on me and my dreams, even when things got quite complicated. Thank you for always wanting my best and for pushing me towards my goals. To my mother, Gracil, you are my eternal source of kindness and support. As a lioness yourself, you have taught me invaluable lessons in strength, resilience, and compassion. Thank you for being the best mother in the world. I would not be the person I am today without you. 'Eu amo vocês. Obrigado por tudo'.

To my dear siblings, you are my lifelong companions on this remarkable journey of life. My dear sister Marina, thank you for your role as the oldest sibling, for correcting my homework, and for being the first of my siblings to

bring me to bars and concerts. To my dear older brother João, thank you for the companionship and shared moments in some of the best years of my life. I still cherish my memories of those years, as well as our inside jokes such as 'Bob Marley'. To my dear brother Ricardo, the youngest of four, thank you for a lifetime of friendship. I absolutely still love our long talks about random topics, sometimes quite scientific like the theory of relativity and its implications, and sometimes quite silly like favorite types of beers. You are a great source of inspiration for me.

Furthermore, I would like to extend my gratitude to my grandparents Célia and Adroaldo for their affection and love; to all my dearest cousins for all the support and long conversations; for my aunt and uncle for their long love; to my extended family for their welcome and support; to my niece and nephews Luana, Luquinha, and J.H.; and a special thanks to Evelyn for her support and for helping me settle in the Netherlands when I was completely lost in a new country.

Last but certainly not least, I would like to express my eternal gratitude to my beloved wife. Kris, putting in mere words all that you represent to me is an impossible challenge. You are my constant motivation, my enduring source of inspiration, my creative muse, and my safe haven. To say I am finishing this journey thanks to you is an understatement. You followed me to the Netherlands with nothing but courage and embarked on your own career path here. You are strong, determined, and ambitious. You bring out the best in me. I cannot even start to imagine what my life would have been like if I hadn't picked you up at Salvador's airport all those years ago. 'Я тебя люблю сегодня и навсегда'. Thank you for your unconditional love and support throughout my Ph.D.

Abstract

In recent decades, the automotive industry has witnessed a remarkable transformation as vehicles have evolved from purely mechanical machines into sophisticated electronic-mechanical hybrids equipped with hundreds of sensors. These sensors, including 3D cameras, LiDARs, and automotive Frequency Modulated Continuous Wave (FMCW) radars, have revolutionized the way vehicles perceive and interact with their surroundings.

Automotive radars, in particular, play several roles in modern vehicles, extending beyond their conventional use for driver assistance features such as adaptive cruise control and automatic emergency braking. These radar systems can provide a precise estimation of speed, distance, and angle between the vehicle and various objects in their environment. By using radio wave reflections, these radars can effectively detect and track different targets, including pedestrians, bicycles, and other vehicles.

Since they were first proposed, radars have undergone a significant transformation, transitioning from their historical roots into affordable and reliable components in modern vehicles. However, this set of new applications has introduced other unique challenges. For example, the need for low latency, limited computational power, and a restricted number of antennas complicate the target detection and angle estimation. Addressing these challenges is essential for supporting future advancements in the automotive industry.

Traditionally, these challenges have been addressed through handcrafted signal-processing techniques, designed to meet the specific needs of automotive radar systems. However, in parallel with these analytical methods, the usage of machine learning has offered a new way of addressing these challenges. In recent years, data-driven techniques have shown unexpected achievements, rivaling human-level abilities in some tasks such as image classification. This success prompted the interest of researchers across various fields, including radar signal processing.

In contrast to some other fields where deep learning has led to significant breakthroughs, the underlying mathematics and physics of current radar signal processing pipelines and models are well understood. To effectively apply data-driven techniques to the radar data, it is necessary to work within the framework

of these established models. On the other hand, many of these traditional models rely on approximations and assumptions that may not always hold in real-world situations. Therefore, integrating deep learning with analytical signal processing might be a beneficial way to overcome some of the current limitations of the automotive radar pipeline while taking advantage of the strengths of both approaches.

In this thesis, we discuss automotive radar signal processing, focusing on the integration of deep learning techniques to complement the traditional analytical signal processing pipeline. Our underlying objective is to reduce the gap between deep learning and analytical techniques for automotive radars. Here, we propose novel approaches that incorporate deep learning techniques into specific parts of the traditional radar processing pipeline to combine the strengths of both domains.

Given the diverse range of challenges in the field of automotive radars, we can narrow the scope of this thesis into five key problems. These include detecting moving objects in various scenarios and noise levels using range-Doppler maps, generating representative synthetic range-Doppler maps for training and evaluating other neural networks, estimating the direction of arrival (angle) of targets using a non-linear antenna array and a single snapshot, estimating the number of targets for multiple antennas, and incorporating imperfections into our antenna models.

More concretely, this thesis tackles each of the proposed key problems as follows.

Firstly, we address the limitation of some analytical detector algorithms used in range-Doppler maps. These algorithms assume Gaussian-distributed background noise and struggle with closely spaced targets or weak reflections. We propose using a deep denoising convolutional autoencoder to enhance object detection.

Secondly, we address the generation of synthetic range-Doppler maps, as data-driven techniques need large quantities of data for training and evaluation. For this, we developed a Generative Adversarial Network (GAN) model to create new, unseen range-Doppler maps.

Thirdly, we address the direction of arrival (DoA) estimation problem using non-linear radar antenna arrays and a single snapshot. The analytical technique, Maximum Likelihood Estimation (MLE), is computationally demanding. For this, we explored two approaches: replacing MLE with a ResNet model and using ResNet as a pre-filtering step to reduce computational complexity.

Fourthly, we address the problem of estimating the number of targets using a single snapshot with non-linear antenna arrays. Here, we introduce ResNet-based deep learning models to efficiently estimate the number of targets.

Lastly, we address the sole usage of synthetic data for DoA estimation evaluation. We discuss the impact of using measured data on DoA estimations by incorporating imperfections into our antenna model. In addition, we evaluate our hybrid ResNet-MLE model using measured data, showing the practical applicability and validity of our proposed models.

In conclusion, this thesis introduces some data-driven solutions to address some of the challenges in the automotive radar domain by reducing the gap between deep learning and analytical approaches.

Samenvatting

De afgelopen decennia heeft de auto-industrie een opmerkelijke transformatie ondergaan, aangezien voertuigen zijn geëvolueerd van puur mechanische machines tot geavanceerde elektronisch-mechanische hybriden die zijn uitgerust met honderden sensoren. Deze sensoren, waaronder 3D-camera's, LiDARs en automotive Frequency Modulated Continuous Wave (FMCW) radars, hebben de manier waarop voertuigen hun omgeving waarnemen en ermee interageren revolutionair veranderd.

Automotive radars spelen met name verschillende rollen in moderne voertuigen, verder dan hun conventionele gebruik voor die de bestuurder assisteren functies zoals adaptieve cruisecontrol en automatisch noodremmen. Deze radar-systemen kunnen een nauwkeurige schatting geven van snelheid, afstand en hoek tussen het voertuig en verschillende objecten in hun omgeving. Door gebruik te maken van radiogolfreflecties kunnen deze radars effectief verschillende doelen detecteren en volgen, waaronder voetgangers, fietsen en andere voertuigen.

Sinds ze voor het eerst werden voorgesteld, hebben radars een aanzienlijke transformatie ondergaan, waarbij ze zijn overgestapt van hun historische wortels naar betaalbare en betrouwbare componenten in moderne voertuigen. Deze set nieuwe toepassingen heeft echter andere unieke uitdagingen geïntroduceerd. Bijvoorbeeld, de behoefte aan lage doorlooptijd, beperkte rekenkracht en een beperkt aantal antennes bemoeilijken de detectie van doelen en hoekschatting. Het aanpakken van deze uitdagingen is essentieel om toekomstige vooruitgang in de auto-industrie te ondersteunen.

Traditioneel zijn deze uitdagingen aangepakt via op maat gemaakte signaalverwerkingstechnieken, ontworpen om te voldoen aan de specifieke behoeften van automotive radarsystemen. Echter, parallel aan deze analytische methoden, heeft het gebruik van machine learning een nieuwe weg geboden om deze uitdagingen aan te gaan. In de afgelopen jaren hebben op data gebaseerde technieken onverwachte prestaties laten zien, waarbij ze in sommige taken, zoals beeldclassificatie, menselijke bekwaamheid evenaren. Dit succes heeft de interesse gewekt van onderzoekers in verschillende vakgebieden, waaronder radar signaalverwerking.

In tegenstelling tot sommige andere vakgebieden waar diep leren heeft geleid

tot aanzienlijke doorbraken, zijn de onderliggende wiskunde en fysica van de huidige radar signaalverwerking pipelines en modellen goed begrepen. Om data-gedreven technieken effectief toe te passen op de radar data, is het noodzakelijk om te werken binnen het kader van deze gevestigde modellen. Aan de andere kant vertrouwen veel van deze traditionele modellen op benaderingen en aannames die niet altijd geldig zijn in echte situaties. Daarom kan het integreren van diep leren met analytische signaalverwerking een goede manier zijn om sommige van de huidige beperkingen van de automotive radar pipeline te overwinnen, terwijl gebruik wordt gemaakt van de sterke punten van beide benaderingen.

In deze scriptie bespreken we de signaalverwerking van automotive radar, met de nadruk op de integratie van diep leren-technieken om de traditionele analytische signaalverwerkingspipeline aan te vullen. Ons onderliggende doel is om de kloof tussen diep leren en analytische technieken voor automotive radars te verkleinen. Hier stellen we nieuwe benaderingen voor die diep leren-technieken integreren in specifieke delen van de traditionele radarverwerkingspipeline, om de sterke punten van beide domeinen te combineren.

Gezien de diverse reeks uitdagingen op het gebied van automotive radars, beperken we het bereik van deze scriptie tot vijf belangrijke problemen. Deze omvatten het detecteren van bewegende objecten in verschillende scenario's met behulp van range-Doppler-kaarten, het genereren van representatieve synthetische range-Doppler-kaarten voor training en evaluatie van andere neurale netwerken, het schatten van de aankomstrichting (hoek) van doelen met behulp van een niet-lineaire antenne-array en een enkele momentopname, het schatten van het aantal doelen voor meerdere antennes en het opnemen van onvolkomenheden in onze antenne-modellen.

Meer concreet pakt deze scriptie elk van de voorgestelde belangrijke problemen als volgt aan.

Ten eerste gaan we in op de beperking van sommige analytische detectiealgoritmen die worden gebruikt in range-Doppler-kaarten. Deze algoritmen gaan uit van een Gaussisch verdeeld achtergrondruis en hebben moeite met nauw naast elkaar geplaatste doelen of zwakke reflecties. We stellen voor om een diepe ruisreducerende convolutionele auto-encoder te gebruiken om objectdetectie te verbeteren.

Ten tweede gaan we in op de generatie van synthetische range-Doppler-kaarten, aangezien op data gebaseerde technieken grote hoeveelheden data nodig hebben voor training en evaluatie. Hiervoor hebben we een Generative Adversarial Network (GAN) model ontwikkeld om nieuwe, niet eerder geziene range-Doppler-kaarten te creëren.

Ten derde behandelen we het probleem van het schatten van de aankomstrichting (DoA) met behulp van niet-lineaire radarantenne-arrays en een enkele mo-

mentopname. De analytische techniek, Maximum Likelihood Estimation (MLE), vergt veel rekenkracht. Hiervoor hebben we twee benaderingen onderzocht: het vervangen van MLE door een ResNet-model en het gebruik van ResNet als een vooraf filterende stap om de rekencomplexiteit te verminderen.

Ten vierde pakken we het probleem aan van het schatten van het aantal doelen met behulp van een enkele momentopname met niet-lineaire antenne-arrays. Hier introduceren we op ResNet-gebaseerde diep leren-modellen om efficiënt het aantal doelen te schatten.

Tot slot behandelen we het exclusieve gebruik van synthetische data voor de evaluatie van DoA-schatting. We bespreken de impact van het gebruik van gemeten data op DoA-schattingen door onvolkomenheden in ons antenne-model op te nemen. Bovendien evalueren we ons hybride ResNet-MLE-model met behulp van gemeten data, waarbij we de praktische toepasbaarheid en geldigheid van onze voorgestelde modellen aantonen.

Concluderend introduceert deze scriptie enkele data-gedreven oplossingen om enkele van de uitdagingen in het domein van de automotieve radar aan te pakken door de kloof tussen diep leren en analytische benaderingen te verkleinen.

Resumo

Nas últimas décadas, a indústria automotiva testemunhou uma transformação notável à medida que os veículos evoluíram de máquinas puramente mecânicas para sofisticados híbridos eletrônico-mecânico equipados com centenas de sensores. Esses sensores, incluindo câmeras 3D, LiDARs e radares automotivos de Modulação Contínua de Frequência Modulada (FMCW na sigla em inglês), revolucionaram a forma como os veículos percebem e interagem com seu ambiente.

Os radares automotivos, desempenham vários papéis nos veículos modernos, podendo ir além de seu uso tradicional de assistência ao motorista, como controle adaptativo de cruzeiro e frenagem automática de emergência. Esses sistemas de radar podem fornecer uma estimativa precisa de velocidade, distância e ângulo entre o veículo e os vários objetos em seu ambiente. Usando as reflexões das ondas de rádio, esses radares podem, de forma eficaz, detectar e rastrear diferentes alvos, incluindo pedestres, bicicletas e outros veículos.

Desde que foram primeiramente propostos, os radares passaram por uma transformação significativa, transitando de suas raízes históricas para se tornarem componentes baratos e confiáveis nos veículos modernos. No entanto, esse conjunto de novas aplicações introduziu novos desafios. Por exemplo, demanda por baixa latência, poder computacional limitado e um número restrito de antenas complicam a detecção de alvos e a estimativa dos ângulos. Discutir sobre esses desafios é essencial para futuros avanços na indústria automotiva.

Tradicionalmente, esses problemas foram resolvidos por meio de técnicas específicas de processamento de sinais, projetadas para atender as necessidades únicas dos sistemas de radar automotivo. No entanto, em paralelo a esses métodos analíticos, o uso de aprendizado de máquina ofereceu uma nova abordagem para enfrentar esses desafios. Nos últimos anos, técnicas baseadas em dados têm mostrado resultados inesperados, rivalizando com as habilidades humanas em algumas tarefas, como classificação de imagens. Esse sucesso despertou o interesse de pesquisadores em várias áreas de conhecimento, incluindo processamento de sinais de radares.

Ao contrário de algumas outras áreas em que o aprendizado profundo levou a avanços significativos, a matemática e a física nos atuais modelos e pipelines do processamento de sinais de radar são bem compreendidas. Para aplicar eficiente-

mente técnicas baseadas em aprendizado por dados, é necessário trabalhar dentro do framework desses modelos já estabelecidos. Por outro lado, muitos desses modelos tradicionais dependem de suposições e aproximações que nem sempre se aplicam em situações do mundo real. Portanto, integrar o aprendizado profundo com o processamento analítico de sinais pode ser uma boa maneira de superar algumas das limitações dos modelos atuais dos radar automotivo, aproveitando as vantagens de ambas as abordagens.

Nesta tese, discutimos o processamento de sinais de radares automotivos, com foco na integração de técnicas de aprendizado profundo para complementar a pipeline tradicional (e analítica) do processamento analítico de sinais. Nosso objetivo é reduzir a distância entre o aprendizado profundo e as técnicas analíticas para radares automotivos. Aqui, propomos abordagens inovadoras que incorporam técnicas de aprendizado profundo em partes específicas da pipeline tradicional dos radares, explorando as vantagens de ambas as áreas.

Dada a ampla gama de desafios no campo de radares automotivos, podemos focar o escopo desta tese em cinco problemas-chave. Isso inclui a detecção de objetos em movimento em diferentes cenários e níveis de ruído, usando 'range-Doppler maps'; a geração de 'range-Doppler maps' sintéticos mais representativos para treinar e avaliar outras redes neurais; estimar a direção de chegada (ângulo) de alvos usando um conjunto de antenas não lineares; a estimativa do número de alvos para várias antenas; e a incorporação de imperfeições em nossos modelos de antena.

Mais concretamente, esta tese aborda cada um dos problemas-chave propostos da seguinte forma:

Primeiramente, abordamos a limitação de alguns algoritmos analíticos de detecção usados 'range-Doppler maps'. Esses algoritmos assumem ruído de fundo distribuído de forma gaussiana e enfrentam dificuldades com alvos próximos ou reflexões fracas. Propomos o uso de um 'deep convolutional autoencoder' (autoencoder convolucional profundo) para melhorar a detecção de objetos.

Em segundo lugar, abordamos a geração de 'range-Doppler maps' sintéticos, já que as técnicas baseadas em redes neurais precisam de grandes quantidades de dados para treinamento e avaliação. Para isso, desenvolvemos um modelo de Rede Generativa Adversarial (GAN na sigla em inglês) para criar novos e úteis 'range-Doppler maps'.

Em terceiro lugar, abordamos o problema da estimativa da direção de chegada (DoA na sigla em inglês) usando um conjunto de antenas de radares não lineares. A técnica analítica, Estimativa de Máxima Verossimilhança (MLE na sigla em inglês), é computacionalmente exigente. Para isso, exploramos duas abordagens: substituir MLE por uma rede neural ResNet e usar ResNet como etapa de pré-processamento para reduzir a complexidade computacional.

Em quarto lugar, abordamos o problema da estimação do número de alvos usando um único 'snapshot' com um arranjo de antenas não lineares. Aqui, introduzimos modelos de aprendizado profundo baseados na rede ResNet para estimar, eficientemente, o número de alvos.

Por fim, abordamos o problema do uso exclusivo de dados sintéticos para avaliar a estimativa de DoAs. Discutimos o impacto do uso de dados reais ao estimar DoAs, incorporando imperfeições em nosso modelo de antena. Além disso, avaliamos nosso modelo híbrido ResNet-MLE usando dados reais, demonstrando a aplicabilidade prática e a validade dos nossos modelos.

Em conclusão, esta tese introduz algumas possíveis soluções utilizando redes neurais para enfrentar alguns dos desafios no domínio dos radares automotivos, reduzindo a distância entre o aprendizado profundo e as abordagens analíticas.

Contents

1	Introduction	1
1.1	Deep Learning and Automotive Radars	4
1.2	Current Limitations	12
1.3	Problem Statement	18
1.4	Key contributions	20
1.5	Outline	24
2	Background	27
2.1	Automotive FMCW Radars	28
2.2	MIMO Radars	35
2.3	Detectors and Estimators	42
2.4	Deep Learning	56
2.5	Deep Learning Architectures	64
2.6	Advanced Deep Learning Topics	71
2.7	Metrics	74
3	Convolutional Autoencoders for Reducing Noise in RD Maps	79
3.1	Introduction	80
3.2	Architectures and Configurations	81
3.3	Dataset	85
3.4	Experiments and Results	87
3.5	Conclusion	93
4	Generating Synthetic RD Maps Using Deep Learning	95
4.1	Introduction	96
4.2	Dataset	97
4.3	Architecture and Configurations	98
4.4	Evaluation	102
4.5	Conclusion	105
5	ResNet for Single-Snapshot DOA Estimation	107
5.1	Introduction	108

CONTENTS

5.2	Signal Model	109
5.3	Deep Learning Architecture and Data	112
5.4	Evaluation and Results	116
5.5	Conclusion	123
6	Fusion of ResNet and MLE for DoA Estimation	125
6.1	Introduction	126
6.2	Signal Model	127
6.3	Deep Learning Architecture	129
6.4	Deep-MLE	130
6.5	Evaluation and Results	132
6.6	Conclusion	135
7	Fusion Model for Antenna Imperfection Mitigation	137
7.1	Introduction	138
7.2	Signal Model	138
7.3	Deep Learning Architectures	140
7.4	Data	142
7.5	Evaluation, and Results	145
7.6	Conclusion	149
8	Conclusion	151
8.1	Summary of the Thesis	152
8.2	Contributions and Results	154
8.3	Directions for Future Works	157
	Abbreviations	159
	List of Symbols	161
	Notation	165
	Bibliography	167
	Publications	181

Introduction

ABSTRACT

This thesis presents deep-learning techniques applied to FMCW automotive radars for improving detection in range-Doppler maps and direction of arrival estimation. The introduction chapter serves as an opening point to the rest of this thesis. Here, we provide general concepts of deep learning and automotive radars, followed by a detailed description of the main problems addressed in this thesis. In the sequence, we define the scope of the thesis and discuss the current limitations of the field. We then present our problem statement. Furthermore, our contributions are highlighted, which focus on reducing some of the gaps between analytical signal processing approaches for automotive radars and machine learning techniques. Finally, the outline section describes the organization of the rest of the thesis and the main topics discussed in each chapter, providing a road map for the reader to follow.

The usage of data-driven techniques, such as Artificial Intelligence (AI), for processing large amounts of information has skyrocketed over the past few years [AH23]. These days, it is hard to imagine the functioning of our modern world without the help of AI tools. From social media [Mon+19] to medical applications [Enh+21], passing through analyzing complex imageries [PML23], processing texts [Bro+20], or any usage that processes large amounts of data [Ram+21], it is undeniable that this technology will continue to be a part of our future. One of the fronts driving the rapid growth of these techniques in various fields, including the automotive industry [Ben+18], is the development of low-cost yet precise sensors such as 3D high-resolution cameras, LiDARs, and automotive radars. These sensors produce a vast quantity of data that is often processed using AI assistance [KN21]. In addition, the employment of AI techniques, such

as Neural Networks (NNs), to these sensors, brought a completely new set of methodologies that quickly became state-of-the-art within several fields for its ability to provide versatile and accurate outputs [KN21]. Segmenting objects in point clouds [Qi+17] obtained with LiDARs, face recognition with cameras [Red+16a], and object tracking with radars [FB21] are among a few examples of neural network usage within the signal processing field.

As our technology to process data evolves, so does the definition of data [Ols21]. These days, we can define data as any signal that carries information through a series of values or symbols. In this sense, signal processing can be understood as the manipulation or analysis of data in order to extract or modify information in some useful manner [Ham99]. In this expanded scope, image processing, natural language processing, point cloud processing, and radar data processing can be understood as special cases of signal processing, where the signals (data) in question are images, text, 3D points, and radar data, respectively.

The development of new technologies pushed the known boundaries of academia and industry as they started to incorporate various novel techniques (e.g., AI) into their new low-cost sensors. One of the industries that greatly benefited from the rise of this new era of combining inexpensive sensors with large data processing was the automotive industry [KN21]. Over the last decades, cars have departed from being purely mechanical machines to become a mechanical-electronic hybrid, with hundreds of sensors running in real-time, making life-or-death decisions in a split-second. The key benefit of using low-cost sensors, such as radars, in technologically advanced cars is to ensure the safety of the passengers while bringing more comfort to the driver. The trend of autonomous driving systems [Ben+18] is one of the reasons behind the new exploration of Deep Learning¹ (DL) applied to inexpensive high-precision automotive sensors such as radars [Wol09]. In Figure 1.1, we show an example of a car using an automotive radar to detect and estimate the relative speed and position of the car ahead, as well as other elements from its environment, such as the road and curbstones.

Although radars have been widely used for several decades [SER87], recent advancements in road surveillance and autonomous systems have brought a new perspective on their applications [TZQ22]. Radars can calculate the distance and speed of an object more precisely than other sensors, they can estimate the angle of an object with good precision, they work in any light conditions (unlike cameras [Zho+22]), and they are more robust and cheaper than laser sensors (such as LiDARs [Wan+20]) [TZQ22; Dav+21].

¹ In this thesis, Artificial Intelligence (AI), Deep Learning (DL), Machine Learning (ML) and Neural Network (NN) might be used interchangeably, unless stated otherwise.

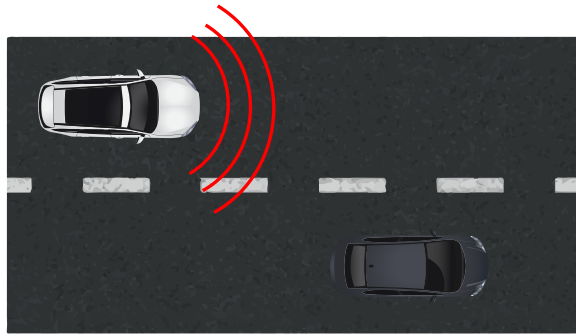


Figure 1.1: An example of a car using an automotive radar to detect elements of the environment such as other vehicles, roads, curbstones, and so on.

However, the usage of automotive radars [Bro05] for road surveillance and autonomous systems brings up a new set of challenges to be faced [Pat+17]. Finite computational processing power, a limited number of transmitting and receiving antennas due to cost, low-power requirements, and the limited availability of datasets are among a few examples of the challenges to be addressed.

One of the ways to address some of these and other challenges is to apply several handcrafted signal-processing analytical techniques [LM04] to diminish some limitations of these radars. However, in order to do so, it is necessary to empirically choose several parameters and add a considerable amount of situational-dependent processing steps within the pipeline. On the other hand, the application of data-driven techniques, such as Neural Networks (NN), proved to be a possible solution to some of these challenges [Dav+21]. Impulsed by the trend brought by image processing [Alo+18], some of these novel neural networks showed to be a promising solution if applied correctly, respecting the nature of radar signals and their physical properties.

In this thesis, we propose novel applications of deep learning techniques applied to the Frequency Modulated Continuous Wave (FMCW) radars. From object detection on the so-called Range-Doppler (RD) maps to estimating the angle of multiple targets, we introduce several deep learning models and evaluate them against traditional analytical signal processing techniques.

The rest of this chapter is organized as follows. In Section 1.1, we provide an overview and limitations of deep learning and automotive radars, as well as the idea of reducing the gap between analytical and data-driven techniques. Section 1.2 discusses the current limitations faced by the industry, which will set the context for our problem statement. In Section 1.3, we present the problem statement of this thesis and the subset of problems that we have addressed.

In [Section 1.4](#), we present and summarize the key contributions of this thesis. Finally, in [Section 1.5](#), we conclude this chapter with an outline of the remaining chapters of this thesis.

1.1 Deep Learning and Automotive Radars

Automotive radars are active sensors that measure the physical properties of nearby objects, such as their relative speed, distance, and angle [\[Pat+17\]](#). Their key advantage over other sensors is that they are not affected by weather conditions (e.g., fog, rain, snow) or light conditions (e.g., day, night) [\[TZQ22\]](#). In [Figure 1.2](#), we show a table with the main differences between radars, cameras, and LiDARs in the automotive field [\[All+19\]](#).

Automotive radars serve several purposes in vehicles, providing driver assistance features, such as adaptive cruise control and automatic emergency braking [\[de 17\]](#). However, radars are capable of much more, as they can quite precisely estimate the speed, distance, and angle between the vehicle and other objects (e.g., pedestrians, bicycles, and cars) since they use radio waves to detect (and even track [\[FB19\]](#)) targets in the environment [\[Pat+17\]](#). Due to the complexity of their data representation, which is challenging to interpret for non-experts, and the lack of widely accessible datasets, automotive radars have received less attention than other sensors. Nevertheless, with appropriate data processing

	Radars	Cameras	LiDARs
Cost	\$ \$	\$	\$ \$ \$
Speed Estimation	✓ ✓ ✓	✓	✓ ✓
Distance Estimation	✓ ✓ ✓	✓	✓ ✓
Angle Estimation	✓ ✓	✓ ✓	✓ ✓ ✓
Robust to Weather	✓ ✓ ✓	✓	✓ ✓
Robust to Light	✓ ✓ ✓	✓	✓ ✓ ✓
Intuitive Data	✓	✓ ✓ ✓	✓ ✓

Figure 1.2: A table with the main differences between radars, cameras, and LiDARs. Although cameras are the cheapest and the most intuitive to use, they do not render the best estimations for speed, distance, or angle.

techniques, it is possible to obtain, for example, radar point clouds that can be used for segmentation and classification tasks, similar to LiDARs.

Deep learning can potentially improve several parts of the traditional radar pipeline. From reducing interference [All+19] to better detection and estimation [LB20a; LB22b], passing through object classification [FB19], there are plenty of possible applications.

In the following subsections, we discuss deep learning, its limitations, automotive radars, and the existing gap between deep learning and the traditional analytical pipeline often used for automotive radars.

1.1.1 Deep Learning

To address some of the limitations of handcrafted signal-processing analytical techniques, we propose the use of deep learning within the automotive radar pipeline.

Deep Learning (DL) is a type of Machine Learning (ML) that uses artificial Neural Networks (NN) to learn complex, non-linear relationships in data [WZP20]. In this context, a deep neural network typically consists of multiple layers of interconnected neurons that process and transform the input data through a series of weighted sums and non-linear activation functions [Mac16]. The training goal of a DL model is to find the parameters (e.g., weights) that minimize the so-called loss function so that its predictions may be as close as possible to the expected outputs [Li+21].

Deep learning algorithms can improve themselves through data examples, either directly or indirectly. These algorithms use large amounts of data to learn how to recognize patterns and to infer predictions on new data [Sch15]. Due to the increase in the processing power of computers, especially the ability of fast matrix computing by graphic cards, deep learning algorithms have exponentially increased their pool of applications over the past few years. These novel algorithms were particularly effective in areas where humans were traditionally superior to machines, like understanding images and speech.

In 2016, for the first time, human-level classification performance on images was obtained in the ImageNet competition [He+16]. This is partially due to the fact that DL models have the ability to learn complex, non-linear relationships in data.

Although the usage of machine learning on other automotive sensors is not exactly new [Mar+89], the recent trend of human-level AI performance brought fresh attention to possible applications of these algorithms in other domains.

The quick success of this new class of algorithms caught the attention of researchers of another growing field: LiDARs and point clouds. Initially, their DL models were adaptations of the successful models used by the image processing

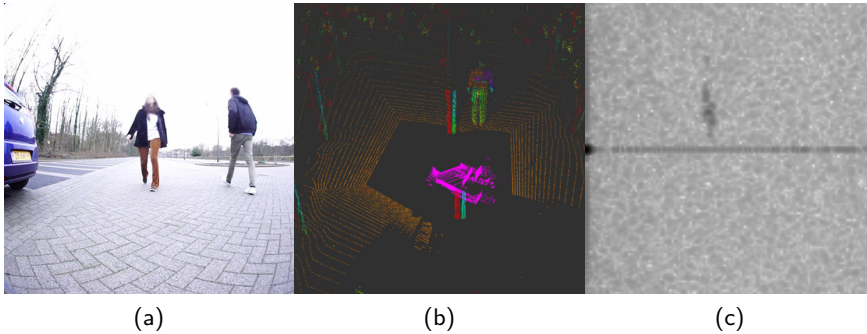


Figure 1.3: Examples of input data: (a) Image from an automotive camera, (b) LiDAR point cloud, (c) Black-and-white range-Doppler map (radar data).

field [Qi+16]. However, the new DL models for LiDARs and point clouds may differ considerably already from their image counterparts due to the distinct nature of their data. For example, KPConv [Tho+19] is a unique way to perform convolution operations in point cloud data.

Similarly, radar researchers took an interest in the potential of DL techniques and started adapting them to fit radar signals [Dav+21]. However, the sophisticated nature of radar data has made it more challenging to build and label datasets suitable for DL training when compared to other types of data such as images, text, or point clouds, which can be more easily labeled by humans. In Figure 1.3, we show the difference between the data between cameras (a) [FB21], LiDARs² (b) and radars (3) [LB20a]. The radar range-Doppler map (c) looks less interpretable to an untrained eye than a photo (a) or a 3D point cloud (b).

The reason radar data is less intelligible than other types of data is that it differs from the way our senses perceive the world. We, humans, are used to light reflecting off surfaces and being captured by our eyes, but not how radar waves reflect, which can be influenced by factors such as the Doppler effect. In our brains, like in the computer vision domain, objects are usually distinguished by their size, shape, and colors. In the radar domain, however, objects are distinctive if they have different speeds, distances, and angles. This fundamental difference in the way various reflected waves are perceived makes radar data harder to understand, giving an advantage to cameras and LiDARs.

The lack of widely available data is another challenge for the development of more data-driven radar models, given that the first open-source radar datasets became available around the year 2019 [Cae+20]. As a comparison, the Ford

² <https://canopies.inf.uniroma3.it/> Extracted from the CANOPIES project simulator.

Campus LiDAR dataset [PME11] dates from around 2011. As a result, the adoption of deep learning in the radar field has been slower than in other domains.

Despite the sophisticated nature of its data, the usage of deep learning for radar data analysis has the potential to improve the performance of radar-based systems and applications, as many people in recent years have been exploring this line of research, being [Fuc+21; PLK22; BGT19; Elb20; CPK22; GLR21; Cao+20] a few examples. Davoli et. al [Dav+21] gives an overview of several published DL techniques for radars.

As deep learning enters the radar processing research field, it is important to note that it has some limitations when compared to the already established analytical signal processing models.

1.1.2 Deep Learning Limitations

As previously mentioned, radar data can be sophisticated and difficult to interpret, which makes it harder to label it in a way that is suitable for training, therefore increasing the challenges of applying DL to radars for some scenarios (for example, denoising or tracking).

Traditional analytical signal processing techniques are normally designed after specific mathematical models and the behavior of the resulting algorithms can be perfectly understood [LM04]. In contrast, deep learning models are typically complex, non-linear systems, in which the behaviors are harder to interpret and understand. This lack of interpretability, caused by all the non-linear functions, may make it difficult to diagnose and fix problems [Mus+22]. This is one of the reasons for some distrust in applying deep learning models in safety-critical applications, where the ability to understand, predict, and explain the model is crucial [Gla17]. To deal with this challenge, some researchers are considering the mathematical models of their problems inside their DL architectures. For example, Cui et. al. [Cui+22], trained a mathematical unfolding neural network to increase its interpretability.

Another potential limitation of deep learning models is that it is generally not possible to guarantee that a global minimum has been reached during training. This happens because there may be multiple local minima in the so-called loss function, and it is difficult to know for certain which minimum is global [Ska18; Bis06]. In Figure 1.4, we show an example of a local minimum in comparison to the global minimum of a non-convex function. In this example, by gradually updating the ball's position, we get stuck into a local minimum that does not represent the global minimum of this function.

Furthermore, even if the global minimum is reached, it may still not perform well on new, unseen data. In other words, the model may achieve a low value for its loss function during training but still makes inaccurate predictions on test

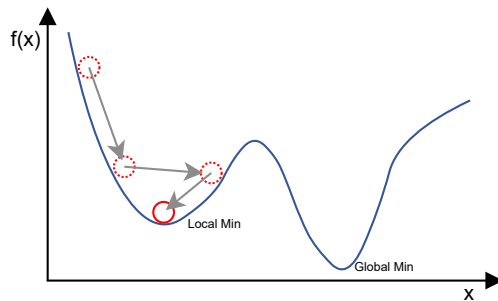


Figure 1.4: An example of a local minimum versus global minimum.

data. Perhaps its loss function does not reflect the correct nature of the problem. It is important to keep in mind that a loss function is a simplified mathematical representation of a phenomenon, and it may not capture all of its complexities and nuances. For example, a loss function may assume that the modeled data is noise-free, following a particular statistical distribution, when this may not be the case. This can be challenging to address, as we can not mathematically guarantee that the global minimum was achieved for a given application. However, extensive testing against analytical signal processing models, in various and diverse scenarios, can mitigate this limitation.

Additionally, another reason for inaccurate predictions on test data is overfitting. Overfitting is a common problem in DL, and it can happen when the model is too complex for the given problem or when the training data is not representative enough [Agg18]. In general, overfitting occurs when a model learns the training data too well and is not able to generalize to new, unseen data. In other words, rather than learning the underlying patterns and relationships, it is merely memorizing the training data and repeating it. Underfitting is the opposite problem. It usually happens when the model is too shallow to capture all the complexities and nuances of the data [Agg18]. To deal with both overfitting and underfitting, researchers can use synthetic and augmented data [Won+16]. In addition, several architectures with various sizes and shapes can be tested to ensure a suitable network size that avoids overfitting and underfitting.

In automotive radar applications, deep learning limitations present unique challenges. Lack of interpretability is a major concern for safety-critical systems like ADAS, where model decisions must be explainable and verifiable. Overfitting risks are high due to limited labeled radar data, and ensuring robustness for various driving conditions is essential. To address these issues, researchers are, for example, integrating signal processing models [You+22] and hybrid approaches to increase generalization.

1.1.3 Automotive Radars

Automotive radars are one of the technologies that facilitate, for example, cruise control, road surveillance, and autonomous vehicles. These systems use radio waves to detect and localize objects in the environment - such as vehicles, pedestrians, and bicycles [Pat+17]. Automotive radars have the ability to operate in various frequency bands. They can provide a high-resolution and wide-angle view of the surrounding environment. Due to these capabilities, radars can detect objects that are often challenging to perceive with the naked eye, including those in difficult weather conditions or under poor lighting [TZQ22].

A common type of automotive radar uses Frequency Modulated Continuous Wave (FMCW) signals to detect and localize objects in the environment. An FMCW radar continuously transmits a wave modulated at a certain frequency range [Pat+17]. The time delay between the transmitted and reflected signals allows the calculation of the distance and velocity of the surrounding objects [HY19]. For angle estimation, the system needs to process information from multiple antennas [Dav+21].

After processing the received raw data by estimating the range and velocity of each target based on their reflected signals, through the usage of 2D Fast Fourier Transforms (FFTs), we can obtain Range-Doppler (RD) maps [HY19]. RD maps are a graphical representation of the range and velocity of radar targets. This map contains the range along one axis and the velocity along the other axis. In this thesis, we often use the horizontal axis to represent distances and the vertical axis to represent velocities, although the opposite is also possible.

Range-Doppler maps are often used in radar systems as a way to detect and visualize targets, and their applications may vary. They can be used, for example, to track multiple targets [FB19], identify the target's relative positions and movements [Pat+17], and classify types of objects [FB21]. Additionally, they can be used to identify points of interest for calculating the direction of arrival [Rao18].

Another type of information that can be extracted from the raw data of a Multiple-Input and Multiple-Output (MIMO) antenna radar system is snapshots [Dav+21]. A radar snapshot refers to a single frame of data, or in other words, radar measurements taken over a short known time interval. It is often used to estimate angles. Direction of Arrival (DoA) refers to the angle (direction) at which a wavefront (such as a sound or electromagnetic wave) arrives at the receivers [KV96]. In the radar context, DoA refers to the angle at which a reflected signal arrives at the antennas. It can be measured in the horizontal (azimuth) and/or vertical planes. DoA estimation is an important problem in radar systems, as it allows the radar to determine the position of the detected targets and generate the so-called radar point clouds. This thesis focuses on

the range-Doppler maps and snapshots part of the radar pipeline, leaving the exploration of radar point clouds for other researchers.

The exploration of automotive radar data presents challenges and opportunities for the application of deep learning. However, although there is a growing trend toward combining signal processing and machine learning, there is a risk of overlooking the unique characteristics of radar signals when applying generic deep learning techniques, creating a certain gap between these fields. Reducing this gap requires a cross-disciplinary understanding of both fields.

One side of the gap between analytical signal processing approaches and machine learning techniques is represented by a strong mathematical and physical foundation and the usage of well-defined models and algorithms. On the other side of the gap lies a focus on data-driven approaches, which prioritize the extraction of patterns and features from data without the need for explicit mathematical modeling. The main objective of this thesis is to find novel ways to reduce some of the gaps between these two approaches and discuss the potential usage of deep learning in the radar domain.

1.1.4 Reducing the gap between analytical and DL approaches

The analytical signal processing approach relies on a set of methods and techniques that are used to analyze, process, and transform signals (e.g., audio, video, or radar data) [Ham99]. These methods are based on mathematical models and algorithms that have been developed over many decades and are well-established in the field [Sch91]. They can provide robust and reliable performance and can be used in a wide range of applications. However, they often make assumptions about the data. For example, noise is often modeled as Gaussian. Nonetheless, these assumptions may not always hold in practice, and the analytical approach may not perform well in reality [Leg+23]. In such cases, some techniques, such as machine learning, may be used to improve the performance of the signal processing system as they can model various distributions if its training dataset is representative enough.

Analytical signal processing techniques can be computationally efficient if necessary approximations are made. However, there is a constraint in the number of variables that can be accurately estimated before it becomes computationally intense [CS84]. This can be especially challenging for complex multi-variable real-time applications that require low latency or that process large amounts of data.

Normally there is a trade-off between performance and computational complexity in signal processing [Zha+21b]. In general, more complex algorithms and approaches tend to provide better performances at the cost of increased computational complexity (although this is not a rule). This means that the optimal

	Deep Learning	Analytical
Benefit	• Handles complex non-linear signals • Good at pattern recognition and feature extraction • Can take randomness in data • Potential for better generalization to unseen data • May have a good performance/cost ratio	• Well-established and proven performance • Well-understood mathematical models • High interpretability of the algorithms • Low implementation cost with the necessary approximations
Challenges	• Needs lots of unbiased data • Unknown adaptability to unforeseen situations • May need to break the problem into smaller parts • Risk of overfitting • Hard to recover implemented function • High computational cost for training	• Approximations might not reflect reality • Challenging adaptation to unmodeled data distributions • Hard to model complex non-linear phenomena • Limited ability to handle complex and non-linear data • May grow in computational complexity

Figure 1.5: A table with some of the benefits and challenges of using deep learning and analytical models.

solution for a given application will depend on the specific requirements and constraints of the situation.

While both analytical and deep learning techniques have their advantages and disadvantages, the different nature of these approaches may increase the risk of using a generic machine learning algorithm, originally designed for a different field of study, which might overlook important analytical features of the input signal, thereby increasing the gap between these areas of study.

We show a few benefits and challenges of using deep learning and a traditional analytical technique for solving some of the problems related to the automotive radar field in [Figure 1.5](#). This figure summarizes some of the trade-offs between DL and analytical techniques.

In the context of the scope of this thesis, reducing the gap means combining the strengths of two different approaches, classical analytical signal processing and machine learning, in order to create novel solutions. Our goal is to enhance the performance of the current systems by adding, removing, or replacing parts of the traditional analytical radar signal processing pipeline with the data-driven capabilities of machine learning.

By modifying parts of the classical analytical pipeline with tailored deep learning techniques, we can reduce some of the gaps between these two fields that can vastly benefit from each other as analytical approaches are well-understood, and DL models can learn non-obvious features from the data.

One way we can help reduce some of the gaps is by adapting neural network

models to better suit the properties of the radar signal. For example, [Chapter 3](#) presents a neural network solution for detecting objects in range-Doppler maps. The approach is inspired by some models used for computer vision but takes into account the specifics of a radar application. Furthermore, instead of replacing a whole portion of the analytical pipeline, we can use NNs as a filtering step before applying traditional approaches. For instance, [Chapter 6](#) introduces a fusion model that combines both neural network and analytical techniques to estimate DoAs.

In short, reducing some of the gaps between analytical and deep learning techniques requires a combination of both approaches, utilizing the unique characteristics of each field.

In the following sections, we narrow our discussion to more specific topics, starting with the current limitations and then moving to the problem statement and key contributions.

1.2 Current Limitations

This thesis focuses on detecting moving objects in Range-Doppler (RD) maps and distinguishing closely spaced objects using the Direction of Arrival (DoA) estimation techniques. To accomplish this, it is important to understand the current limitations of RD map detectors and DoA estimators.

In the context of automotive radars, a detector receives the input and provides a sort of binary output, indicating whether there is an object present or not. On the other hand, an estimator produces a numerical output, using the input data to make a prediction about the true value [[Van01](#)].

1.2.1 Range-Doppler Map Detectors

A Range-Doppler (RD) maps map is a two-dimensional graphical representation where the range of objects is displayed along one axis and their velocity along the other axis. It is created by processing radar signals that are reflected off moving and still targets [[Kok14](#)]. In the context of automotive radars, range-Doppler maps are often used to detect and track moving objects in the environment as they can provide information about the range, velocity, and direction of movement of vehicles, pedestrians, and other objects.

Several challenges come with interpreting range-Doppler maps. First of all, they are not as intuitive as pictures as our senses are not naturally attuned to “see” range vs. Doppler. However, this is not the only challenge. Range/velocity resolution, clutter, target reflectivity, occlusion, and multipathing are known problems when processing RD maps. In this thesis, we focused on target reflectivity and clutter as the main challenges of this topic.

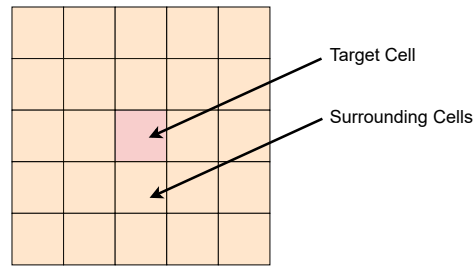


Figure 1.6: A representation of a possible CFAR algorithm.

In the context of RD maps, clutter refers to the radar reflection from stationary objects in the environment, such as buildings, trees, and terrain [JYB16]. Clutter can interfere with detecting and tracking moving objects, especially if the radar itself is moving.

The reflectivity of an object is an important factor for its detectability by radars [RSC66]. Metallic objects, such as vehicles, tend to reflect a larger portion of the incident radar energy, which makes them easier to detect. On the other hand, objects with low reflectivities, such as pedestrians or animals, tend to be harder to detect. In some cases, the reflected signal might be too weak to be distinguishable from the scene noise or clutter.

Detection techniques can be used to identify and locate moving objects in range-Doppler maps [Win07]. These techniques are designed to filter targets from noise and clutter. A detector typically uses signal processing algorithms and statistical models to separate targets from noise. Therefore, they need to be carefully tuned to the surrounding environment, also considering the radar's parameters.

Among the possible techniques to detect objects in RD maps, the Constant False Alarm Rate (CFAR) [Sch91] detector algorithm and its variations are well-known [Win07]. These algorithms estimate the background noise level by looking at the cells around the target cell. The estimated noise level is then compared to the target cell to determine if an object is present or not. An example of this is shown in Figure 1.6. More specifically, CFAR filters out noise and clutter by comparing radar measurements to a local threshold, which is calculated based on the surrounding values at each range and velocity bin in the map. In addition, the local threshold must be carefully calibrated because if it is set too low, there will be a high probability of false detection, and if it is set too high, there will be a high probability of missed detection [Sch91]. More about CFAR in Subsection 2.3.2.

However, these algorithms have some limitations. One of the main limitations is that the algorithm assumes that the background noise is always Gaussian distributed, which is rarely the case for real-world applications. Clutter, for example,

typically does not follow a bell-shaped distribution curve [Bla88]. Additionally, these algorithms may struggle to detect two closely spaced targets [VBC14] as the presence of the second object can interfere with the background noise level estimates. A similar effect might happen to large or highly reflective objects as these algorithms are not typically designed to combine search cells. Furthermore, weakly reflected targets may be mistaken for clutter or noise and might not be detected.

1.2.2 Direction of Arrival Estimators

Besides range-Doppler maps, where relative distance and speed are the main variables, we can consider other types of data that can be extracted from the reflected signal of MIMO radar antennas. Radar snapshots [TF09; Van02] refer to radar measurements taken in a fixed, short, known interval. One measurement typically lasts 30 milliseconds for an automotive FMCW radar, in which the transmitted energy is received after being reflected off objects. Direction of Arrival (DoA) estimation involves determining the angles at which targets are located relative to the radar antenna array using one or more radar snapshots. This information can help determine the location of objects, especially if two or more targets are at similar distances and speeds from the radar.

Although the angle information can be valuable for several applications, many problems can arise when estimating the direction of arrival of radar reflections. Dealing with non-Gaussian noise, resolving the angles of multiple targets, and the difficulty of precisely estimating the number of targets present in the scene are just a few examples of challenges faced by these estimators [AN14].

When estimating the angles of different targets, the antenna configuration (and its mathematical equations) matters, as each antenna configuration brings a new set of advantages and challenges. Using a radar with a non-uniform antenna array, such as Minimum Redundancy antenna Array (MRA) [Mof68] instead of Uniform Linear antenna Array (ULA), decreases the used number of antennas without reducing the available angular resolution (the minimum angle at which two targets can be distinguished) [SN13]. This is especially important for automotive systems as fewer antennas means less power dissipation and heat.

In general, using more snapshots improves the precision of the DoA estimation. However, in the context of automotive applications, there are constraints on the maximum latency (i.e., processing time) that is allowed, which may limit the number of snapshots that can be used [SY07]. As a result, it is often necessary to estimate the DoAs using just a single snapshot.

It is possible to separate DoA estimator algorithms into four categories, three of them classical [ATN00] and one data-driven. The three classical categories are spectrum-based estimators, subspace-based estimators, and parametric estima-

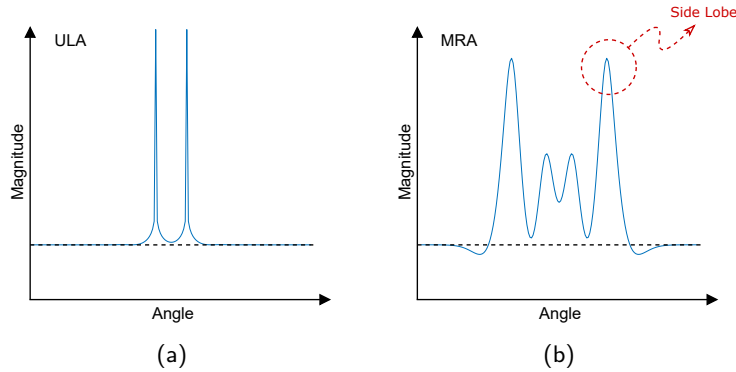


Figure 1.7: Spectral representation output, using a MUSIC algorithm, for two targets located at -10° and 10° : (a) Using a Uniform Linear antenna Array (ULA), and (b) Using Minimum Redundancy antenna Array (MRA).

tors [ATN00]. Although the list is by no means exhaustive, it serves as a way to distinguish some popular DoA estimators. Algorithms such as MUSIC (subspace-based) and ESPRIT (subspace-based) are popular choices as they are easy to implement and do not require much computational power, unlike MLE (parametric) [KV96]. In some cases, the spectrum and subspace-based techniques may have near-MLE performance [VBC14].

However, these algorithms have also major drawbacks. When the number of available snapshots is limited or insufficient, algorithms such as MUSIC and ESPRIT tend to have poor performance [VBC14]. The loss in performance can be mitigated by using some other techniques such as spatial smoothing at the cost of a loss in resolution, which is problematic in the case of closely spaced signals (super-resolution problem). In general, using a single snapshot for estimating DoAs decreases the performance of these types of algorithms significantly.

Another main drawback is related to the antenna array geometry. Both spectrum and subspace-based techniques may have a decrease in their estimating performances when using an MRA instead of ULA. For example, as we show in Figure 1.7, MUSIC (subspace-based) may generate large side lobes in its spectral representation that could be larger than the targets themselves.

Hacker and Yang [HY10] have tested several algorithms, including MUSIC and ESPRIT, to determine which one would be more appropriate for the case of a single snapshot with MRA antennas. They concluded that the Maximum Likelihood Estimation (MLE) algorithm had the best performance in this scenario with a known number of targets. MLE is a parametric algorithm that can be used for estimating the direction of arrival by finding the values that maximize

its likelihood function. However, a major drawback of the MLE algorithm is that estimating multiple DoAs (multiple targets) can be a computationally intense task.

1.2.3 Number of Targets Estimators

Besides estimating DoAs, snapshots can be used to estimate the number of targets within the same distance and speed but with different DoAs. Estimating the number of targets is a challenging task. This requires the ability to disambiguate the signal from multiple objects and to identify the unique features of each object. Spectrum and subspace-based algorithms, such as MUSIC, may struggle when two or more targets have similar angles when using a single snapshot [TF09; KV96].

Moreover, MLE can be used to estimate the number of targets in the scene using a single snapshot [LB22b]. This estimation happens iteratively by making successive assumptions about the number of targets in the scene and evaluating the likelihood of those observations. The algorithm starts by assuming there is only one object in the scene. If the likelihood of this assumption is low, it presumes that there are two targets, and so on, until an acceptable level of likelihood is reached. However, because the number of target estimations happens in iterations, the computational load of MLE increases even further.

1.2.4 Imperfections

When studying the Direction of Arrival (DoA) and the number of targets estimation, many researchers use mathematically generated snapshots, such as [BGT19; Elb20]. However, this kind of snapshot often contains approximations that may not fully reflect the complexity and variability of real-world scenarios. One common approximation is the usage of ideal antennas, which do not take into account the various imperfections that can affect the estimator's performance.

Imperfections in the antenna array can bring challenges, such as systematic errors, mutual coupling, and imprecise antenna location [TF09]. These imperfections can affect the accuracy and reliability of the estimator and can have a negative impact on the performance of the system if they are not properly addressed. Therefore, it is important to consider the effects of antenna imperfections when designing and evaluating estimator algorithms, especially for applications where accuracy and reliability are critical.

In Figure 1.8, we show an example of an imperfection in the gain (amplitude) of the signals that are received. Often researchers try to compensate it for the average imperfection across the array [CS96]. However, this approach may not always be effective as the imperfections can also depend on the array orientation.

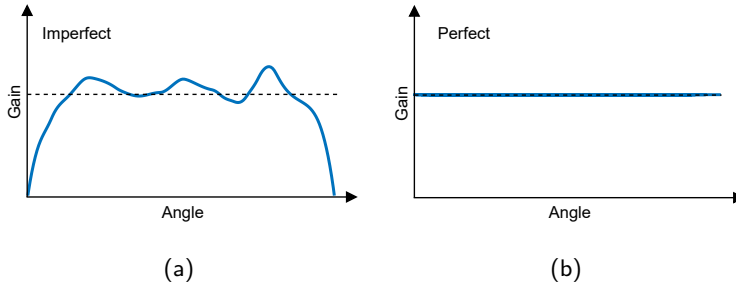


Figure 1.8: The gain of two different antennas: (a) an antenna with gain imperfection, and (b) an ideal antenna.

Making the compensation angle dependent is challenging as, besides introducing additional variables that need to be estimated, the compensation is dependent on the existing parameters, increasing the overall complexity of the system.

1.2.5 Synthetic Data

It is hard to research analytical and data-driven techniques without discussing the data itself. As previously stated, radar data is less intuitive and not widely available. Obtaining a large and diverse enough dataset can be difficult as it requires collecting and labeling a large number of data from various scenarios and environments. For this reason, many researchers restrict their work to using synthetic data.

Synthetic data refers to data that is artificially generated rather than gathered from the physical world [Jor+22]. It can be used for evaluating new techniques, training machine learning techniques, or just augmenting an existing dataset. In addition, synthetic data can be automatically labeled, saving time and labor.

Synthetic data can be obtained through a variety of methods. In this thesis, we categorized it into three groups: mathematical, simulation, and generative. The mathematical approach [Jor+22] uses known physical models to generate data for specific scenarios. This method is light but limited in terms of complexity. For example, sometimes non-linearities are not modeled. The simulation approach [JHS22] involves simulating several real-world properties, such as ray tracing, reflected wave signals, and multipathing, in a virtual environment [QY16]. This method allows for greater flexibility and control over the properties of objects and surroundings. However, this method can be computationally intense, particularly for large or sophisticated scenarios, which can make it challenging to generate a given dataset in a reasonable amount of time. The third approach uses

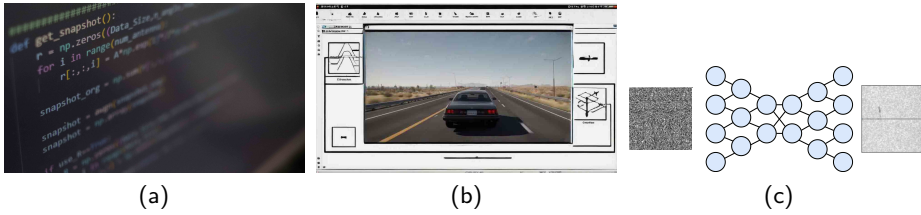


Figure 1.9: A representation of three kinds of synthetic data: (a) a mathematical data generation through code, (b) a simulation through a virtual environment used to generate data, and (c) an AI generative model generating new data from noise.

a deep generative model [Goo+14] to generate new, unseen data. This method is less computationally intense than simulations and can be more complex than the mathematical approach.

On the other hand, the usage of synthetic data also creates its own set of challenges. Synthetic data is generated through an approximation process that may not accurately reflect the complexity and variability of real-world scenarios, such as the usage of ideal antennas. As a result, models trained with synthetic data might perform well in theory, but underperform in real-world scenarios. This can make it difficult to accurately evaluate the performance of techniques based solely on synthetic data and highlights the importance of evaluating models with measured data whenever possible.

In Figure 1.9, we show three representations for the three categories that we used. An image with lines of code to represent the mathematical synthetic data generation, an image³ of a virtual environment inside of software that generates synthetic data through a simulation, and an image of an artificial neural network generating synthetic data through noise, representing generative models.

In the following sections, we present the problem statement of this thesis and its key contributions.

1.3 Problem Statement

To summarize, as previously discussed, the automotive radar field has several challenges and limitations including a gap between the analytical techniques and the growing trend of data-driven techniques. To address some of these challenges and push the development of better radar systems, we propose the introduction of deep learning techniques into the existing automotive radar pipeline. By adding

³ <https://stability.ai/stable-image> Image generated using Stable Diffusion.

or replacing existing parts of the current pipeline with a data-driven technique, we may mitigate some of the current limitations, previously discussed in [Section 1.2](#).

In this thesis, we address the problem of improving parts of the radar signal processing pipeline by adding or replacing parts of it with a NN-based equivalent. Within this main problem, we focus on five sub-problems: (1) The detection of different moving objects in various situations and noise levels using range-Doppler maps. (2) The generation of representative synthetic range-Doppler maps for training other models. (3) The estimation of DoAs for the single snapshot MRA scenario. (4) The estimation of the number of targets in a single-snapshot MRA scenario. (5) The usage of imperfect antennas and measured data to evaluate models.

More concretely, we can list the key problem and its sub-problems we address throughout this thesis.

The key problem that this thesis addresses is how to introduce DL-based techniques into the automotive radar signal processing to potentially improve its pipeline. The five sub-problems we address are as follows:

- The first sub-problem we address is the reliance of some analytical detector algorithms on fixed statistical models for detecting moving objects in range-Doppler maps. The limitations of these algorithms include: assuming a Gaussian distributed background noise, difficulties detecting closely spaced targets, and mistaking weakly reflected targets for clutter or noise. We address this problem by replacing the analytical detector with a deep denoising convolutional autoencoder that can more accurately detect objects to later extract their distances and velocities.
- The second sub-problem we address is the generation of synthetic range-Doppler maps. We address this problem by developing a Generative Adversarial Network (GAN) model trained for the task of creating new, unseen RD maps. This can potentially be useful for training other DL models, evaluating analytical techniques, or augmenting existing datasets.
- The third sub-problem we address is the direction of arrival estimation for a single snapshot using a non-uniform radar antenna array. Spectrum and subspace-based techniques have difficulties in this scenario, and while the MLE algorithm is the better suited for such scenarios [HY10], it may be computationally intense and may not be suitable for applications with low-latency requirements. We address this problem by using two different approaches. One approach is to replace the MLE algorithm with a ResNet-based (neural network) model. The other approach is by using a ResNet-based model as a pre-filtering step before the MLE algorithm to decrease the overall computational complexity.
- The fourth sub-problem we address is the estimation of the number of

targets using a single snapshot and a non-linear antenna array. Subspace-based algorithms such as MUSIC may struggle with the super-resolution problem. The MLE algorithm, although more suitable, increases its computational load with each iteration when estimating the number of targets. We address this problem by developing two deep learning models that utilize ResNet architectures to better estimate the number of targets in the scene. In the first model, the training for the estimation of the number of targets occurs after the DoA estimation, while in the second model, the estimation of the number of targets and the DoA estimation are performed simultaneously.

- The fifth and final sub-problem we address is the sole usage of synthetic data to accurately evaluate the performance of DoA estimators. Many researchers utilize only mathematically generated data to validate their models [CPK22]. This means that although some models might seem effective in theory, they may not be deployable in reality. We address this problem by analyzing the impacts of imperfections on DoA estimation and evaluating our fusion model (named deep-MLE) using measured data.

In the following section, we link the main problem and sub-problems that we address with our key contributions.

1.4 Key contributions

In this thesis, we address the problem of improving parts of the analytical radar pipeline by integrating data-driven techniques. Our primary goal is to reduce some of the gaps between these two fields by adding or replacing parts of the pipeline with deep learning techniques, thereby improving target detection in range-Doppler maps and the estimation of their DoAs.

By integrating deep learning models into the existing pipeline, we can: improve the detection and estimation; reduce the computational load of certain analytical algorithms; and generate synthetic data.

More concretely, our six key contributions, which are related to the five sub-problems previously presented, are listed below and then further explained in the following subsections.

1. An approach for detecting moving objects using deep Convolutional Auto-Encoders (CAE) [LB20a] in various architectures and scenarios (Chapter 3).
2. An approach that uses a Generative Adversarial Network (GAN) and a Convolutional Auto-Encoder (CAE) to generate synthetic Range-Doppler (RD) maps of moving objects [LB20b] (Chapter 4).
3. An approach that uses a Residual Neural Network (ResNet) architecture to estimate the direction of arrival using a single snapshot for FMCW radars

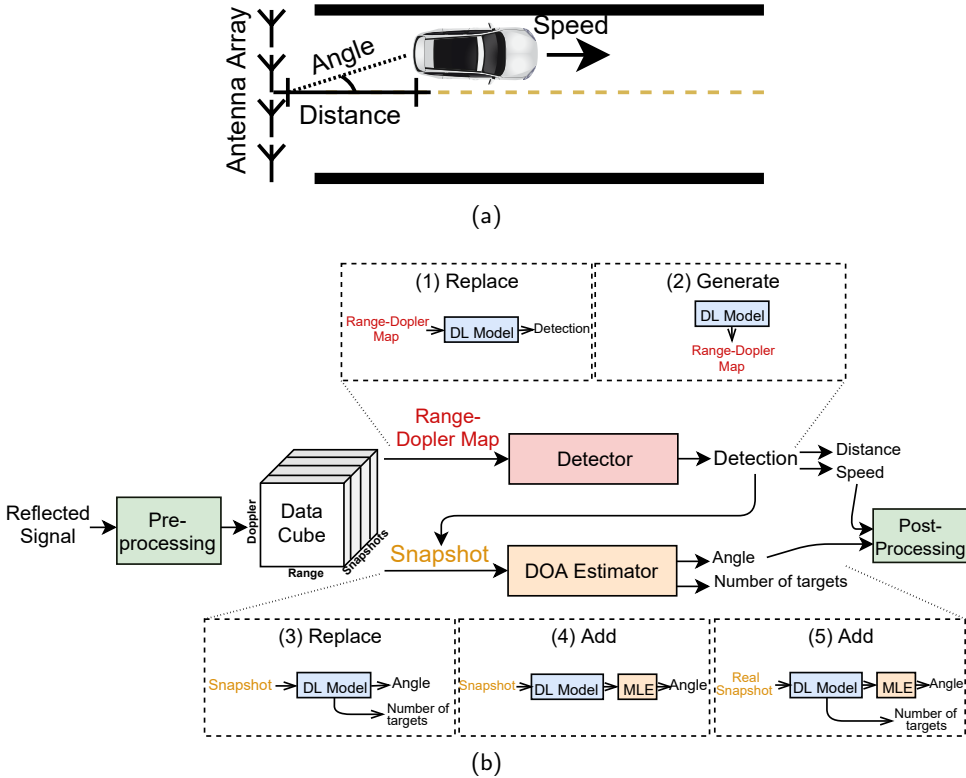


Figure 1.10: Figure (a) represents the antenna array detecting a car and estimating its relative distance, speed, and angle. Figure (b) shows a diagram with the basic ideas of a traditional analytical signal processing pipeline for radar signals to obtain the distance, speed, angles, and number of targets by processing the data cube (containing range, Doppler, and snapshots) of a certain pre-processed reflected signal. Here we show an overview of this thesis and where it fits into the traditional radar pipeline. In (1), [Chapter 3](#), we replace the CFAR by using a deep learning model; in (2), [Chapter 4](#), we show a deep learning model generates new synthetic Range-Doppler maps instead of gathering new measured data; in (3), [Chapter 5](#), the MLE algorithm is replaced by a deep learning model; and in both (4) and (5), [Chapter 6](#) and [Chapter 7](#), a deep learning model is added before the MLE algorithm in the pipeline.

[LB22b] (Chapter 5).

4. An approach to estimate DoAs by fusing the classical MLE algorithm with a 1D ResNet-based neural network [LB22a; LB23] (Chapter 6 and Chapter 7).
5. An approach to estimate DoAs considering imperfect antennas and that works with measured data [LB23] (Chapter 7).
6. An approach that uses residual neural networks to estimate the number of targets using a single-snapshot MRA radar [LB22b; LB23] (Chapter 5 and Chapter 7).

In Figure 1.10, we show an overview of this thesis condensed into a single pipeline. In this figure, we show one object that is moving away from a radar antenna with a certain speed, distance, and angle. From the reflected signals, we can extract the data cube that contains all the information needed to estimate the velocity, distance, angles, and number of targets. This figure can help to clarify the contributions of this thesis to the automotive radar pipeline.

In the following subsections, each key contribution is further detailed to provide a better understanding of the work presented and analyzed in this thesis

1.4.1 Denoising CAE Applied to Detection on RD Maps

Our first contribution is in object detection in range-Doppler maps. RD maps are two-dimensional matrices that can be thought of as image-like data. This allows the usage of computer vision techniques, such as deep convolutional autoencoders. The advantage of using a denoising deep convolutional autoencoder to detect moving objects in RD maps over an analytical detector algorithm is that the data-driven approach can learn patterns, making it better at detecting targets in environments with high noise or clutter. However, it is important to have a versatile training dataset to avoid being constrained by the training data. We show this contribution in (1) of Figure 1.10.

1.4.2 Generating Synthetic RD maps with GAN and CAE

Our second contribution is the development of a deep learning model to generate synthetic range-Doppler (RD) maps. We have created a simple model for generating synthetic range-Doppler maps that closely mimic measured data by processing and augmenting existing RD maps. This was achieved by using a Generative Adversarial Network (GAN) as a generator and a Convolutional Auto-Encoder (CAE) as a predictor. This method provides a solution for small or incomplete datasets without the need for manual annotation and without losing many generalizations of the real world. This is a departure from traditional RD

map generation methods, which often rely on simulated environments or simple algorithmic generation. We show this contribution in (2) of [Figure 1.10](#).

1.4.3 ResNet for a Single-Snapshot DoA Estimation

Our third contribution is the development of a deep learning model to estimate DoAs using a Residual Neural Network (ResNet) for a single snapshot. The method was tested across various fields of view, resolutions, and antenna configurations (MRA and ULA). The proposed model was compared against MLE, MUSIC, and ESPRIT. The key aspect of this approach is the output of the neural network that relies on a classification model rather than regression. Additionally, we also discuss about an algorithm that selects the final DoA. We show this contribution in (3) of [Figure 1.10](#).

1.4.4 Fusing ResNet and MLE for DoA Estimation

Our fourth contribution is the development of a fusion model, called deep-MLE, using a 1D ResNet-based neural network and the Maximum Likelihood Estimation (MLE) algorithm to improve the DoA estimation in radar systems. The deep learning model serves as a pre-processing step, reducing the number of angles for the MLE algorithm to process, which ultimately reduces the computational complexity of MLE while improving its accuracy. In other words, by not replacing MLE, we can take advantage of its robustness in estimating DoAs, while reducing its computational complexity. We show this contribution in (4) of [Figure 1.10](#).

1.4.5 Imperfect Antennas and Real-World Data

Our fifth contribution is the usage of our fusion method (1D ResNet and MLE algorithm) with a non-ideal antenna array. We started by testing our model with synthetic data containing imperfect antenna approximations. Further, we have experimented using measured data, where the neural network was fine-tuned with it. Since most DoA estimation techniques are only tested with generated data, testing our model with measured data offers a more realistic evaluation of its performance. In addition, most techniques overlook that antenna imperfections vary with the target's DoA, while our model accounts for this dependency. We show this contribution in (5) of [Figure 1.10](#).

1.4.6 Estimating the Number of Targets

Our sixth and final contribution is the development of novel ways to estimate the number of targets using a neural network classification approach. Estimating the number of targets can be a computationally intense task, especially for large

or complex scenarios. By utilizing parts of the main architecture of our ResNet-based DoA estimator, we can use a classification neural network and estimate the number of targets with minimal computational load. We show this contribution in both (3) and (5) of [Figure 1.10](#).

1.5 Outline

This section serves as a roadmap for a better understanding of the structure and organization of the thesis as it provides an overview of its topics and chapters. Furthermore, we present here the main discussed topics and where to find them, making it easier to navigate through the thesis.

Following the introduction, the general background mathematics and techniques are presented in [Chapter 2](#). Here we discuss the main concepts of FMCW radars, extending it to MIMO radars. We then explain the differences between estimators and detectors. Furthermore, we present analytical methods such as CFAR, MUSIC, ESPRIT, and MLE. As our goal is to bring data-driven techniques into the automotive radar traditional pipeline, in the sequence we transition to deep learning techniques as we discuss the basics behind neural networks, main architectures, and some advanced techniques. We finish the background chapter by establishing some of the used metrics for evaluating our models.

Once discussing the main ideas of automotive radars, deep learning, and analytical signal processing, we proceed, in the following chapters, to address the main key problems within the scope of this thesis. Since the radar pipeline starts with RD detectors (as shown in [Figure 1.10](#)), we first start exploring range-Doppler maps, as we analyze them for two chapters, where [Chapter 3](#) focuses on object detection and [Chapter 4](#) on RD map generation.

After finishing discussing the detection part of the pipeline, we move on to the estimation part, where we address DoA estimation (angles) related problems. We spend three chapters ([Chapter 5](#), [Chapter 6](#), and [Chapter 7](#)) discussing different deep learning models we use to estimate DoAs in various configurations for a single snapshot scenario. Within [Chapter 6](#), and [Chapter 7](#), we discuss the usage of a fusion model to add a neural network as a pre-processing step before using the MLE algorithm. Within [Chapter 5](#) and [Chapter 7](#), besides estimating their angles, we estimate the number of targets using deep learning.

Finally, the usage of measured data to evaluate our models in a real-world scenario is discussed in [Chapter 7](#). Although only here measured snapshots are used, we use measured RD maps in [Chapter 3](#) and [4](#).

More concise and giving a brief explanation of each chapter, the organization of the rest of the thesis is defined as follows:

- In **Chapter 2**, we provide a comprehensive background of key subjects that are essential to comprehend our research.
- In **Chapter 3**, we present a method to denoise range-Doppler maps for detections, following our publication [LB20a].
- In **Chapter 4**, we present a method to generate synthetic Range-Doppler maps, following our publication [LB20b].
- In **Chapter 5**, we present a method that replaces MLE for estimating the number of targets and their DoAs using neural networks, following our publication [LB22b].
- In **Chapter 6**, we present a method that uses a DL model as a processing step to MLE to estimate DoAs, following our publication [LB22a].
- In **Chapter 7**, we present a method that estimates DoAs and the number of targets on real-world scenarios, following our publication [LB23].
- In **Chapter 8**, we conclude this thesis with a summary of the subjects that were examined during the development of this research.

2

Background

ABSTRACT

The application of deep learning methods in automotive FMCW radars has opened new opportunities for reducing the gap between analytical and data-driven techniques. To effectively utilize deep learning, it is necessary to understand both the inner workings of a radar system and the fundamental principles of machine learning models. This background chapter provides an intuitive explanation of radar description, different algorithms for detectors and estimators, as well as an overview of various machine learning models.

This chapter provides readers with the necessary knowledge of automotive FMCW radars and deep learning models to understand the research presented in subsequent chapters. First, we introduce the concept of an FMCW radar and its working principles. In the sequence, we discuss the importance of detectors and estimators in radar systems while presenting different algorithms for detection and estimation. Furthermore, this chapter introduces some deep learning concepts, which are used in this thesis to modify parts of the existing data processing pipeline in FMCW radars. Here, we discuss the different types of deep learning architectures and some more advanced techniques. Finally, we conclude this chapter by explaining some of the used metrics to evaluate our results.

To better understand the content of this chapter, readers can refer to the following structure: Firstly, in [Section 2.1](#), provides an overview of automotive FMCW radars, including their working principles and applications. The second one, [Section 2.2](#), presents MIMO radars, how they operate, and why using several antennas on a radar system. [Section 2.3](#) is dedicated to detectors and estimators, where the different algorithms used for detection and estimation, such as

CFAR, MUSIC, ESPRIT, and MLE, are discussed. After discussing the analytical methods, the following three sections introduce some deep learning techniques, exploring various aspects of it. [Section 2.4](#) covers the basics of neural networks, including different types of learning and concepts such as loss function and optimizer. [Section 2.5](#) focuses on the main neural networks used in the thesis, including fully connected neural networks, convolutional neural networks, convolutional autoencoders, residual neural networks, and generative adversarial networks. Furthermore, [Section 2.6](#) discusses advanced techniques for deep learning, including reducing model complexity and fine-tuning. After presenting both analytical and data-driven techniques, we need to present a way to compare the results obtained by these approaches. Therefore, we finish this chapter with [Section 2.7](#) which explains the most frequently used metrics of this thesis.

2.1 Automotive FMCW Radars

A radar (short for radio detection and ranging) is a system that uses electromagnetic waves to locate, detect, or track nearby objects. Radars work by emitting a signal wave through one or more Transmitter (Tx) antennas, and gathering back reflected signals from objects through one or more Receiver (Rx) antennas. By processing the received signal, a radar system can determine the distance, velocity, and direction of one or more objects. Due to the operational frequencies of radar systems, they are not affected by visible light and, therefore, are not affected by lighting and weather conditions.

Radar systems are versatile tools that have been developed for a wide range of applications, including aviation, military, weather forecasting, maritime navigation, traffic control, road monitoring, Advanced Driver Assistance Systems (ADAS), among others [[Ric14](#); [Jos+19](#); [Gam20](#)]. In this thesis, we discuss the usage of automotive FMCW mmWave radars in various situations. We discuss the use of such systems with both analytical and data-driven approaches.

One way that we can categorize the emitted electrical-magnetic wave of an automotive radar is based on the frequency of their emitted waves and the design of their waveform. Millimeter Wave (mmWave) radars usually operate between 30 and 300 GHz (in other words, between 1 and 10 millimeters). By waveform design, automotive radars usually emit a pulse or continuous wave.

Although intuitively a pulse radar would render simpler mathematical equations, it usually requires larger components than a continuous wave radar as the returned wave needs to be relatively more powerful to be detected by the Rx antennas [[Sto92](#)]. However, calculating range becomes less obvious when using a continuous wave radar as it is hard to distinguish an emitted signal from a received one. Therefore, in order to estimate the range, the transmitted waves

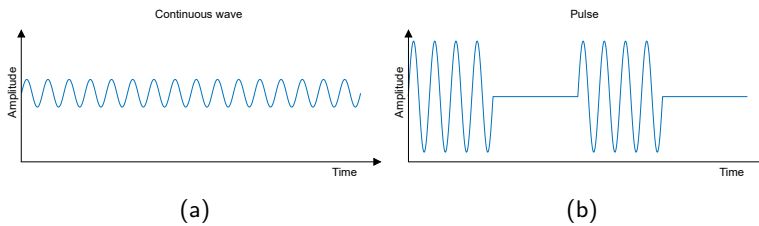


Figure 2.1: Two different waveforms used by automotive radars: (a) a continuous wave, and (b) a pulse wave.

are modulated in frequency. Figure 2.1 shows an example of a continuous wave and a pulse wave.

A Frequency Modulated Continuous Wave (FMCW)¹ radar transmits a type of continuous wave radar that modulates its frequency linearly over a known period of time before returning to its initial frequency. To achieve this modulation, the radar sends out a single unit signal called a “chirp,” which has an increasing linear modulation. The end of the chirp is indicated by the signal returning to its original frequency. This frequency variation allows the radar to measure the range and velocity of a target by comparing the frequency difference between the transmitted and received signals. In the next subsection, we discuss the FMCW radar and its equations.

When a radar detects a moving object, the frequency of the received signal is shifted due to the Doppler effect². This Doppler shift causes an increase in the apparent frequency for objects moving toward the radar and a decrease for objects moving away from the radar.

Another possible way to categorize automotive radars is based on their antenna configuration. Linear arrays, which include Uniform Linear Array (ULA), are some of the most basic arrangements available. These antenna arrays are composed of a single row of radiating elements with transmitters and receivers. They offer simplicity and cost-effectiveness, making them a popular choice for many automotive applications. For example, some advantages of using linear arrays over other antenna configurations are simpler equations and fewer elements for the same azimuth resolution [Van02].

However, non-uniform linear arrays, such as Minimum Redundancy Array (MRA) [Mof68], can increase the angular resolution of a radar without the need

¹ https://mllimadeoliveira.github.io/Thesis/FMCW_move.html Animation of a FMCW's amplitude and frequency.

² <https://mllimadeoliveira.github.io/Thesis/Doppler.html> Animations of the Doppler effect for still and moving targets.

for additional antennas. A disadvantage is the introduction of higher side lobes (unwanted peaks in the power spectrum of received signals) [SN13].

Although there are several kinds of radar for various applications, the scope of this thesis is mainly focused on FMCW automotive radars with a Minimum Redundancy Array (MRA). For this, in the following sections and subsections, we discuss the main characteristics and equations of such radars.

2.1.1 Frequency Modulated Continuous Wave (FMCW) Radar

The adoption of Frequency Modulated Continuous Wave (FMCW) radars [Gam20] became a popular choice in the automotive industry sector [Pat+17] due to their ability to provide range and Doppler information using low-cost components. In this section, we present signal processing techniques of FMCW radars, which include estimating the distance and velocity of objects.

For the development of the necessary equations, we take an intuitive approach rather than a more rigorous one. For more in-depth mathematical information, see [Ric14; Gam20; SR10; Wol18].

As shown in Figure 2.2, consider a transmitting antenna Tx sending a certain signal at a certain time t , and a receiving antenna Rx getting the same signal after being reflected by an object initially at the time $t + \tau$. A simplified signal

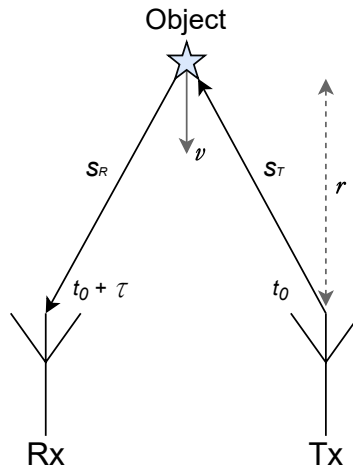


Figure 2.2: An object reflecting a signal coming from a Tx antenna to a Rx antenna in the time $t = t_0$.

equation for the transmitted signal $s_T(t)$, with a varying frequency $f(t)$, is:

$$s_T(t) = e^{j2\pi f(t)t} = e^{j\omega(t)t} . \quad (2.1)$$

Consider that the received signal $s_R(t)$ is the same as the transmitted signal $s_T(t)$, but delayed by a certain τ amount as it takes some time for a wave to travel to an object and back. The time shift will also cause a phase shift Φ_τ :

$$s_R(t) = s_T(t - \tau) = e^{j\omega(t-\tau)(t-\tau)} = e^{j\omega(t-\tau)t} e^{-j\Phi_\tau} , \quad (2.2)$$

where the phase shift Φ_τ caused by the time delay τ and f_s is the carrier frequency or source frequency.

$$\Phi_\tau = \omega(t - \tau)\tau = 2\pi f_s \tau . \quad (2.3)$$

In order to retrieve range and velocity information from the received signal, we can mix the received and transmitted signals. This information can be obtained in the mixed signal as its frequency is proportional to the difference between the transmitted and received frequencies, which, in turn, depends on the distance and velocity between the radar and the target. The resulting signal from the mix (or “beating”) is called a beat signal b and its frequency is called beat frequency $f_b = 2\pi\omega_b$:

$$b(t) = s_T(t)s_R^*(t) = e^{(j\omega(t)t) - (j\omega(t-\tau)t)} e^{j\Phi_\tau} = e^{j\omega_b(t)t} e^{j\Phi_\tau} , \quad (2.4)$$

where the beat frequency is the difference between the frequencies of Tx and Rx $\omega_b(t) = \omega(t) - \omega(t - \tau)$.

On the other hand, it is easy to understand that the time delay itself depends on the distance between the radar and the object r and on the speed of light c :

$$\tau = 2r/c . \quad (2.5)$$

As observed in [Figure 2.3](#), the slope of a chirp is given by the bandwidth B over the period of one chirp T : slope = B/T . In a similar way, the same slope can be obtained using the beat frequency f_b and the time delay τ : slope = f_b/τ . Since both triangles are similar, we can conclude that $B/T = f_b/\tau$. Combining this knowledge with the time delay given by [Equation 2.5](#), we can calculate the instant distance (range) of an object as [\[Gam20\]](#):

$$r = \frac{cf_b T}{2B} . \quad (2.6)$$

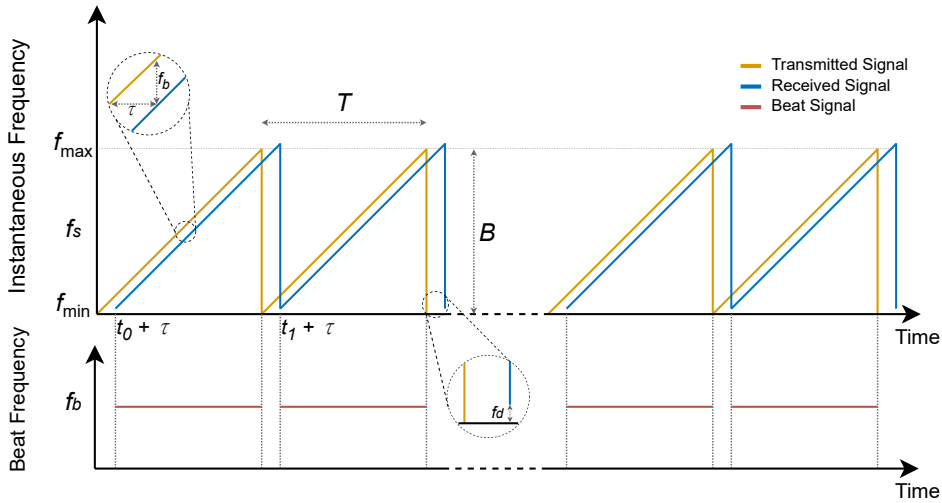


Figure 2.3: Instantaneous frequency of transmitted and received waves in case of a single moving object (Top). The resulting beat frequency (Bottom). In this figure, we explicit the following variables: the time delay τ , the beat frequency f_b , the period of one chirp T , the bandwidth of a single chirp B , the source frequency f_s (also called carrier frequency), and the Doppler frequency f_d .

Besides from the distance between the radar and the object, we can also consider that the object has some velocity and it is moving towards the radar, as previously shown in [Figure 2.2](#). To calculate its velocity, we analyze the phase shift caused by the time delay. If we substitute [Equation 2.5](#) in the [Equation 2.3](#) we obtain that $\Phi_\tau = 2\pi f_s \frac{2r}{c}$. By calculating the derivative of Φ_τ , we obtain the velocity, as the derivative of r is v , and the derivative³ of Φ_τ is ω_d .

$$\frac{d(\Phi_\tau)}{dt} = 2\pi f_s \frac{2}{c} \frac{d(r)}{dt} \therefore \omega_d = 2\pi f_s \frac{2v}{c} . \quad (2.7)$$

If we substitute $\omega_d = 2\pi f_d$, where f_d is the amount that the carrier frequency was shifted due to the Doppler effect, in the [Equation 2.7](#) and rearrange it, we can obtain the velocity of the moving object [[Gam20](#)]:

$$v = \frac{cf_d}{2f_s} . \quad (2.8)$$

³ Because the Doppler effect causes a phase shift, our notation for the derivative of Φ_τ is ω_d and not ω_τ .

Another way that we can come to the same conclusion is by departing from the Doppler effect equation of classical physics if we consider that $c \gg v$ and that the received signal suffered the Doppler effect twice.

$$f_s + \frac{f_d}{2} = \frac{c}{c-v} f_s \therefore f_d = \frac{2v}{c} f_s \quad (2.9)$$

Coming back to [Figure 2.3](#), we can observe some key elements involved in determining the relative speed and distance of objects using FMCW radars. The figure shows the transmitted signal in orange and the received signal in blue. The Tx antenna sends a modulated signal that varies between minimum and maximum frequencies. The duration and bandwidth of each chirp are used to calculate the slope of the chirp. By mixing the transmitted and received signals, we can obtain a beat signal (in red) whose frequency corresponds to the difference between the signals. The received signal has a delay that is proportional to the distance of the target. The velocity of the target causes a small Doppler shift when compared to the original signal.

In the next subsection, we discuss how to obtain range-Doppler maps.

2.1.2 FMCW Radar Range-Doppler Maps

A Range-Doppler (RD) map [[Ric14](#)] is a two-dimensional representation of the echoes received by a radar over time. It shows the distance and velocity information of objects in the radar's field of view. To obtain an RD map, it is necessary to perform a 2D FFT on the beat signal, as the transform provides information on the frequency content of the signal as a function of time.

For that, first, we need to choose the size of the range-Doppler map [[Ric14](#)]. This is related to the number of Analog-to-Digital Converter (ADC) samples (also called fast-time) and the number of chirps used for a single frame (also called slow-time). In the sequence, the beat signals of each chirp are sampled and grouped together as a single matrix. By taking an FFT along the fast-time, we can obtain range samples (for each chirp) that are placed into bins. Furthermore, by taking another FFT along the slow-time, we can obtain the velocity (Doppler) samples that are also placed into bins. [Figure 2.4](#) shows an example of how to obtain an RD map.

Each element of Range-Doppler maps is a complex number [[Sen+20](#)]. They can also be converted to power. By taking the logarithm of the magnitude of each complex number in the range-Doppler map, we obtain a real-valued map representing the power at each range and Doppler bin in decibels:

$$\text{Power} = 10 \times \log_{10} |z|. \quad (2.10)$$

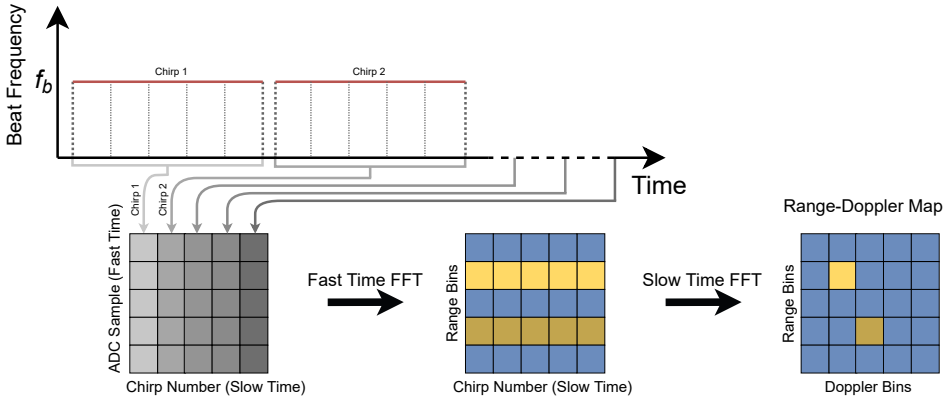


Figure 2.4: An example of how to obtain Range-Doppler (RD) maps. In short, the beat signals are sampled and put together as a matrix. Each matrix has a certain number of chirps, called a chirp train. A FFT is applied to the fast-time. A second FFT is applied to the slow-time.

Each power bin can also be converted to a black-and-white image for other types of representation. For this, we can normalize the range-Doppler map expressed in decibels. The normalized powers are then converted to an 8-bit grayscale image [Bas19]. Figure 2.5 (b) shows an image of an RD map⁴ of a person walking towards the radar that was converted into a black-and-white picture.

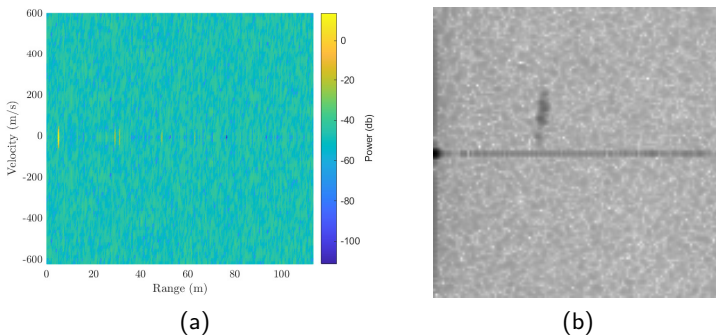


Figure 2.5: Two examples of RD maps: (a) an RD map reflecting some targets (small yellow objects) of a simulated environment, (b) an RD map of a person walking (long vertical dark object) converted to a black-and-white picture.

⁴ https://mllimadeoliveira.github.io/Thesis/RD_noise.html Animation with several RD maps of a person walking.

The difference in speed between the person's torso and legs creates the vertically elongated shape. The horizontal line in the middle of the map represents all the still objects in front of the radar.

We further discuss the usage of range-Doppler maps in [Chapter 3](#) and [Chapter 4](#). In the next section, we discuss MIMO radars and direction of arrival.

2.2 MIMO Radars

So far, we have shown that FMCW radars can measure the range and velocity of targets. In addition, we have discussed how to obtain range-Doppler maps, which show the distribution of targets across various ranges and Doppler bins.

However, RD maps do not provide any information about the angular position of the targets relative to the radar. For that, we can estimate the direction of arrival (DoA) of targets if multiple antennas are used. Multiple-Input and Multiple-Output (MIMO) radars utilize multiple antennas to transmit and receive signals simultaneously, allowing for DoA estimation [[Gam20](#)]. In the following subsections, we discuss the concept of MIMO radars and how they can be used for DoA estimation.

2.2.1 Snapshots

Radar snapshots [[KV96](#); [TF09](#)] are essential elements in the Direction of Arrival (DoA) estimation. A radar snapshot refers to one or a sequence of radar measurements taken within a short, fixed, and known interval. By analyzing the properties of the received snapshot(s), we may determine the direction of the incoming wavefronts from targets, as it is common to assume that the target is

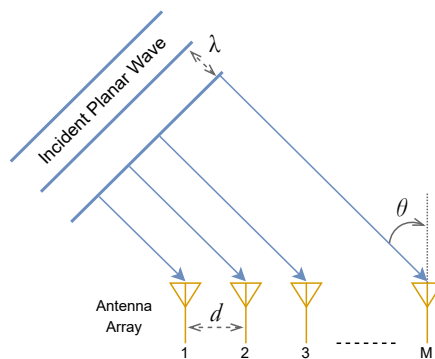


Figure 2.6: An example of a planar wave, containing the angular information of a reflected wave, arriving at a linear antenna array.

Chapter 2: Background

far away and that the incident wave is flat. We show in [Figure 2.6](#) an arriving wave that contains DoA (angular) information.

Consider a linear antenna array containing M antennas. Following [\[KV96\]](#), the output vector, containing the snapshot $\mathbf{x}(t)$ for a single DoA $\theta \in \mathbb{R}$, coming from the antenna array is:

$$\mathbf{x}(t) = \mathbf{a}(\theta)\mathbf{s}(t), \quad (2.11)$$

where $\mathbf{a}(\theta)$ is the steering vector, and $\mathbf{s}(t)$ is the signal emitted by the source⁵ (or reflected by the target).

The steering vector is a function of the source's angle. It also depends on the antenna geometry. Considering a linear array with a geometry $D(d)$ depending on the minimum distance d between two adjacent antennas, the equation for the steering vector $\mathbf{a}(\theta) \in \mathbb{C}^M$ is:

$$\mathbf{a}(\theta) = [e^{j2\pi\lambda^{-1}D_1(d)\sin(\theta)}, \dots, e^{j2\pi\lambda^{-1}D_M(d)\sin(\theta)}]^\top, \quad (2.12)$$

where λ is the wavelength of the transmitting signal $\lambda = f_s/c$.

We can extend the previous equations to their more general form. For this, the superposition principle is applicable, assuming a linear system. Consider that N sources are present and that they are in the far field. The received snapshot $\mathbf{x}(t)$ containing the DoAs $\theta = \theta_1, \dots, \theta_N$ is given as the sum of the individual snapshots for a single DoA.

$$\mathbf{x}(t) = \sum_{n=1}^N \mathbf{a}(\theta_n)\mathbf{s}_n(t), \quad (2.13)$$

where $\mathbf{s}_n(t)$, $n = 1, \dots, N$ is the baseband signal waveforms [\[KV96\]](#).

We can put the previous equation in a more compact form by defining the steering matrix $\mathbf{A}(\theta) \in \mathbb{C}^{M \times N}$ and the vector of signal waveforms $\mathbf{s}(t) \in \mathbb{C}^N$:

$$\mathbf{A}(\theta) = \begin{bmatrix} e^{j2\pi\lambda^{-1}D_1(d)\sin(\theta_1)} & \dots & e^{j2\pi\lambda^{-1}D_1(d)\sin(\theta_N)} \\ \dots & \dots & \dots \\ e^{j2\pi\lambda^{-1}D_M(d)\sin(\theta_1)} & \dots & e^{j2\pi\lambda^{-1}D_M(d)\sin(\theta_N)} \end{bmatrix}, \quad (2.14)$$

$$\mathbf{s}(t) = [\mathbf{s}_1(t), \dots, \mathbf{s}_N(t)]^\top. \quad (2.15)$$

⁵ When discussing the DoA estimation problem, it is equivalent to consider that the targets are reflecting waves or the sources are emitting waves. Unless stated otherwise, targets and sources may be used interchangeably.

We can now obtain the commonly used signal model for array processing by adding some Gaussian noise $\eta(t) \in \mathbb{C}^M$ to the snapshot $x(t) \in \mathbb{C}^M$ equation:

$$x(t) = \mathbf{A}(\theta)s(t) + \eta(t). \quad (2.16)$$

We discuss the usage of snapshots in [Chapter 5](#), [6](#), and [7](#). In the next subsections, we discuss the geometries of antenna arrays within our scope.

2.2.2 Linear Arrays

Antenna geometries are an essential part of radar systems, as they are responsible for directing and shaping the radar beam to achieve the desired performance characteristics. While there are several types of antenna geometries, such as uniform circular arrays and planar arrays, linear arrays, in particular, offer advantages such as reduced complexity, low cost, and easier integration with other system components, making them well-suited for use in automotive radar systems.

An Uniform Linear Array (ULA) [[KV96](#); [Gam20](#)] consists of a collection of antennas equally spaced along a straight line. It is a common type of antenna array used for many sorts of applications. This arrangement may produce a narrow beam and low side lobes, making it well-suited for spectrum and subspace-based DoA estimation techniques.

A Minimum Redundancy Array (MRA) [[Mof68](#); [HY10](#); [SN13](#)] consists of a non-uniform linear arrangement of antennas that has a minimum number of repeated spacing between the elements. It is designed to reduce the correlation between the signals received by different elements and improve the resolution of the DoA estimation. An MRA can achieve the same resolution as an ULA with fewer elements, which reduces the pre-processing (fewer signals), power consumption, and heat generation.

One of the main differences between ULA and MRA is the trade-off between side-lobes⁶ and the resolution⁷ [[TF09](#); [Mof68](#)]. Side lobes are unwanted peaks in the power spectrum of received signals that can cause interference with the main lobe, which corresponds to the direction of the desired signal source. Meanwhile, the resolution is the ability to distinguish between two or more closely spaced signal sources. The resolution is proportional to the number of equispaced antennas separated by d [[Rao18](#)]. An Uniform Linear Array (ULA) has lower side-lobes than an MRA, which means it has less interference from other directions. On

⁶ <https://mllimadeoliveira.github.io/Thesis/SideLobe.html> Animation with various polar plots of Gain vs. Angle (in degrees) for ULAs and MRAs [[Ma+23](#)].

⁷ Depending on the part of the radar signal processing, resolution might have a different definition. Here it is related to the radar's synthetic aperture that is linked to the minimum angle in which two targets can be distinguished.

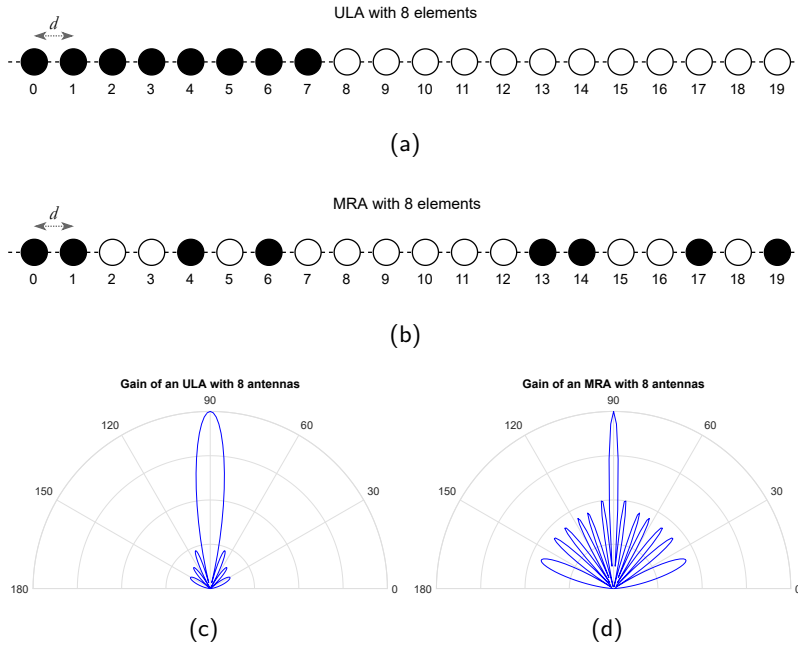


Figure 2.7: The difference between ULAs and MRAs. For (a) and (b), the black circles represent the presence of an antenna element, a white circle the absence, and d is half the radar's wavelength. (a) An Uniform Linear antenna Array (ULA) with 8 elements. (b) A Minimum Redundancy antenna Array (MRA) with 8 elements. (c) A polar plot of the gain vs. the angle, in degrees, where the side lobes are visible for a ULA with 8 antenna elements. (d) Gain vs. angle for a MRA with 8 antennas, where the side lobes are visible.

the other hand, MRA has a higher resolution than ULA with the same number of elements, meaning it can detect more signal sources with less ambiguity.

We can observe this trade-off more explicitly in [Figure 2.7](#). Here, it becomes clearer that the gaps in MRAs increase both the resolution and the magnitude of the side lobes. In general, for automotive FMCW radar applications, the choice between an MRA or an ULA depends on the specific application.

More specifically, as previously stated, $D(d)$ is a linear array with a geometry depending on the distance d between each antenna. For an ULA, this variable is:

$$D_{\text{ULA}}(d) = [0, d, \dots, Md] = (m-1)d, \quad (2.17)$$

where $m = 1, \dots, M$, and $d = \lambda/2$.

For MRAs, the approach to determining the array geometry differs from ULAs, since it depends on the number of repeated spacing. The work of Ma et. al. [Ma+23], brings a table with possible MRA spacing.

To define the geometry of MRAs, we first start by defining a vector $p(m) = [p_1, \dots, p_M]$ that contains the minimum spacing and defining a difference set DIF such as:

$$\text{DIF} = \{|p_i - p_j| : p_i, p_j \in p(m), i < j\}. \quad (2.18)$$

We can define the vector $p(m)$ as containing MRA spacing if the size of DIF is equal to the distance between the first and last element of $p(m)$:

$$\begin{aligned} \text{If MRA : } \text{size}(\text{DIF}) &= \max(p(m)) - \min(p(m)) \\ \text{If not MRA : } \text{size}(\text{DIF}) &\neq \max(p(m)) - \min(p(m)). \end{aligned} \quad (2.19)$$

If the vector $p(m)$ satisfy the conditions for being MRA, we can define $D_{\text{MRA}}(d)$ as:

$$D_{\text{MRA}}(d) = p(m)d, \quad (2.20)$$

where $m = 1, \dots, M$, and $d = \lambda/2$. For example, in Figure 2.7, $D_{\text{MRA}}(d) = [0, 1, 4, 6, 13, 14, 17, 19]d$.

2.2.3 Virtual Arrays

Virtual antennas [TF09; CSC16; Rao18] are not physical antennas but rather combinations of signals from multiple transmitting and receiving antennas that form a larger array (MIMO radars). By creating a set of virtual antennas from the Tx and Rx signals from the physical antennas, it is possible to achieve a higher resolution with fewer elements.

The key benefit of virtual antennas is that they enable the creation of a large effective aperture from a physically small antenna array. This leads to improved angular resolution and detection range as we obtain a larger effective aperture. By using digital signal processing techniques to combine the signals received by multiple antennas, virtual arrays can achieve a much higher number of antennas than would be possible with a physical array of the same size since the number of virtual antennas is equal to the product of the number of transmitting and receiving antennas [Gam20].

As shown in Figure 2.8, by using 3 Tx antennas and 4 Rx antennas, we can obtain a uniform linear virtual array with 12 elements. In addition, by using 2 Tx antennas and 4 Rx antennas, we can obtain a minimum redundancy virtual antenna array with 8 elements such as $D_{\text{MRA}}(d) = [0, 1, 4, 6, 13, 14, 17, 19]d$.

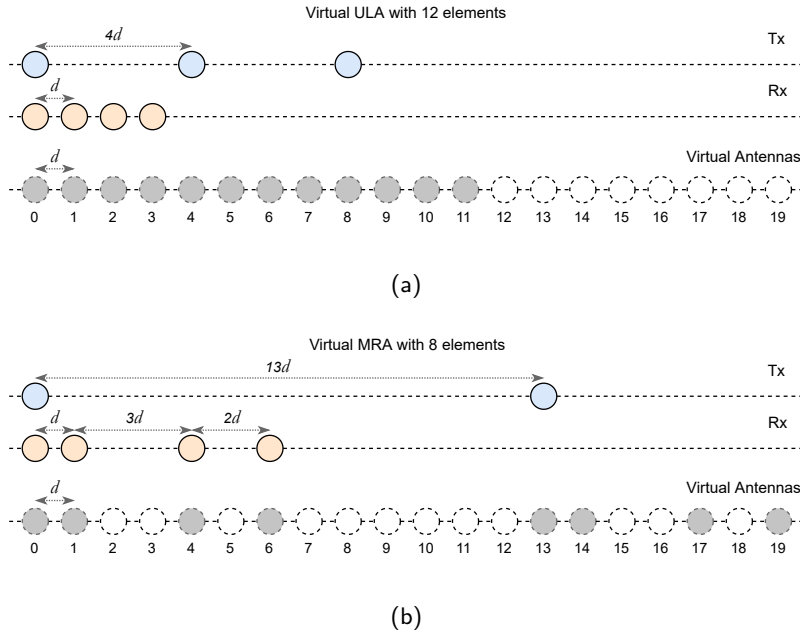


Figure 2.8: Two virtual arrays. (a) An Uniform Linear virtual antenna Array (ULA) with 12 elements - 3 Tx and 4 Rx. (b) A Minimum Redundancy virtual antenna Array (MRA) with 8 elements - 2 Tx and 4 Rx.

As we can observe, the virtual antennas formed by combining Tx and Rx elements act like a single antenna array receiving waves from unknown sources. For this, all the equations described in [Subsection 2.2.1](#) are valid in the same way as if they were physical antennas.

2.2.4 Data Cube

After having discussed the properties of a range-Doppler map, that is a matrix containing distance and velocity information embedded into its axis, and the properties of a snapshot, which are radar measurements taken within a known interval and used for estimating the direction of arrival, we can now discuss the concept of a radar cube.

The radar cube is a three-dimensional matrix that stores radar information in a structured way. There are several ways to make this cube, and, depending on the application, it will store different information, such as fast-time and slow-time [Ric14; Gam20]. Here, we consider that the data cube stores range and Doppler information as its first two dimensions and snapshots as its third. This means that each element of the cube is a complex number containing information about

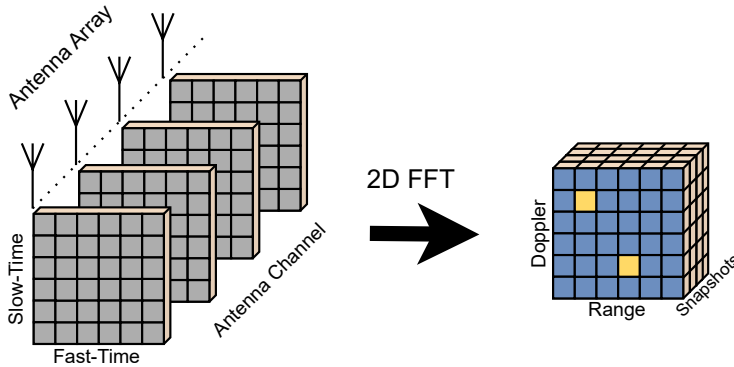


Figure 2.9: Obtaining a radar cube. Our radar cube is a stack of M range-Doppler maps, producing the radar snapshot in the third dimension.

the received signal, including amplitude and phase. In other words, each element of the cube represents the signal reflected from a specific point in the radar's field of view. The radar cube can be thought of as an information cube where each slice is a two-dimensional range-Doppler map, and the third dimension is the snapshot information that can provide DoA estimation.

To produce this radar cube, as shown in [Figure 2.9](#), first we process the chirps from all antennas to RD maps using a 2D FFT, as previously shown in [Subsection 2.1.2](#). In the sequence, we stack up these maps to form the radar cube. This means that a single radar cube is built for each “frame”.

The radar cube is useful when extracting information about the targets. First, we process one or several RD maps from a single cube using a detector to find all important objects. The distance and speed of all targets are extracted in this step. In the sequence, for each detection, the snapshot is extracted and processed using an estimator. The estimator then retrieves the angles and number of targets for each detection. In the end, we extract four pieces of information from each detection: distance, velocity, angle, and number of objects. An example was previously shown in [Figure 1.10](#). The range-Doppler map part of the radar cube is further discussed in [Chapter 3](#) and [4](#), while the snapshot part is further discussed in [Chapter 5](#), [6](#) and [7](#).

In the following section, we discuss some useful analytical algorithms for detecting and estimating targets in the radar domain.

2.3 Detectors and Estimators

In the context of FMCW radars, the fundamental concepts of radar systems have been discussed, including the range-Doppler map, the snapshot equations, and the radar cube. In order to extract useful information from the radar data, it is necessary to implement algorithms and techniques that can process the received signals and provide meaningful information about the targets.

Figure 2.10 shows that a detector is used to identify the presence of a target in the vicinity of the radar, while an estimator is used to determine the angle of the detection.

This section further discusses commonly used analytical algorithms for detection and estimation. We start with a class of algorithms for detection called Constant False Alarm Rate (CFAR). In the sequence, we discuss a few estimators, such as Multiple Signal Classification (MUSIC), Estimation of Signal Parameters via Rotational Invariance Technique (ESPRIT), and the Maximum Likelihood Estimation (MLE) algorithm. In the following section, we discuss machine learning techniques that may also be applied as detectors or estimators for FMCW radar systems.

2.3.1 Radar Target Detection

One of the initial steps in processing a received radar signal is target detection, which distinguishes genuine target reflections from noise and clutter. In the following subsections, we cover some basic concepts and techniques employed for detecting targets in radar systems.

Identifying whether something is a target might not be so trivial. The most rudimentary solution is to establish a basic threshold. However, this approach has some drawbacks, as a shift in the noise level may lead to changes that may cause the detector to underperform. In this case, just lowering the threshold may increase the detection rate, but at the same time may also result in a higher number of false detections [SER87].

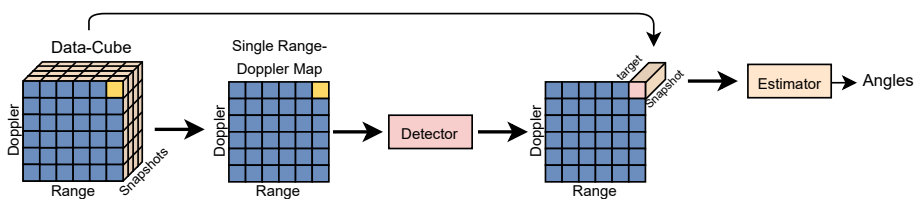


Figure 2.10: An example of how a detector and an estimator can be used to extract information in automotive radars.

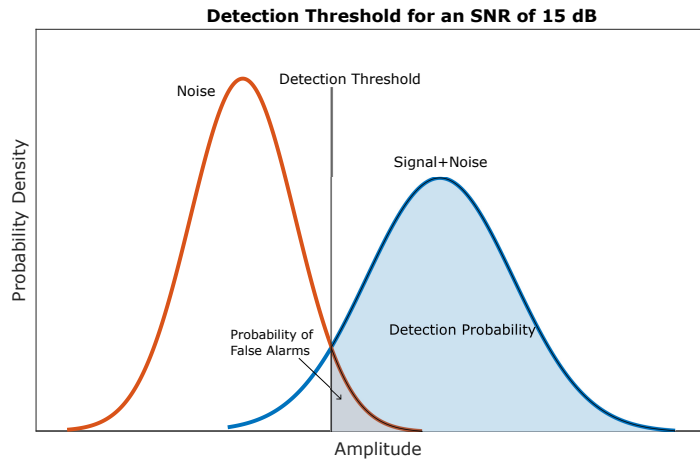


Figure 2.11: The curves of noise following a normal distribution and received signal points plus noise following a normal distribution. The received signal will intercept the noise curve to some degree, in most cases.

Radar antennas receive both noise and signals at all times, therefore, detecting targets involves differentiating between the two. The probability distributions of noise and signal depend on the noise power and radar design. For a more naive design (Neyman–Pearson detector [SER87]), a way to increase the probability of detection without increasing the false alarm rate is to increase the Signal-to-Noise Ratio (SNR), which can be done, for example, by combining several RDs maps together.

Figure 2.11 shows that the probability density of the noise following a normal distribution with zero mean and variance of 1, and the probability density of the signal points following a normal distribution with the mean varying with the Signal-to-Noise Ratio (SNR) and a variance of 1. The detection threshold⁸ depends only on the SNR for more simplistic detectors [Ric14]. A higher SNR shifts the whole detection distribution to the right, reducing the overlap between noise and signal areas.

2.3.2 CFAR

Constant False-Alarm Rate (CFAR) [Sch91] is a popular family of detection algorithms used in radar signal processing to detect targets against background noise, clutter, and interference. The idea of finding a detector that would keep the

⁸ <https://mllimadeoliveira.github.io/Thesis/PDF.html> Animations of probability densities for different SNRs.

false alarm rate constant dates back to the '60s [SER87]. The main idea behind CFAR is to set a threshold based on the statistical properties of the background clutter while keeping the probability of false alarms at a constant rate. In other words, CFARs differs from simple detectors as they utilize a threshold that varies dynamically depending on an estimate of the noise power.

This algorithm uses the surrounding cells to estimate interference (noise or clutter) and determine the presence of a target in the center cell. This process is repeated for all cells. The assumption is that the noise or clutter is spatially or temporally uniform [Gam20]. The algorithm aims to produce a constant false alarm rate that is independent of the noise or clutter level, provided that the noise has a Rayleigh distribution in all the cells. The cell that is currently being evaluated to dynamically change the threshold is called Cell Under Test (CUT).

The CFAR algorithms also has several challenges. One issue is that when multiple targets are in proximity (or if the target is large), the threshold may be raised, making it harder to detect some (or all) targets. Furthermore, non-Gaussian clutter (such as trees, cars, or buildings) is a common occurrence that can complicate the detection process [JYB16].

Although there are many variations of the CFAR algorithm, Cell-Averaging CFAR (CA-CFAR) is one of the most used versions of it, being most of the other variations a version of it, such as Ordered-Statistic CFAR (OS-CFAR) [Bla88].

CA-CFAR

In this CFAR algorithm, instead of taking a single surrounding value, the average of the close cells is taken to compute the threshold.

As shown in Figure 2.12, the Cell-Averaging CFAR (CA-CFAR) [JYB16; Gam20] takes the average of the training cells to calculate the threshold that will be compared to the CUT. The guarding cells are not used in this calculation as they are in the vicinity of the cell under test. To ensure that the threshold calculation is not affected by the CUT, we exclude the cells immediately surrounding it from the computation.

More specifically, considering that all interference (e.g., noise, and clutter) is Gaussian and considering that the noise power P_n can be defined as the average of K cells around the evaluated input cell z (CUT). The threshold $\mathcal{T}$ is then calculated as the P_n times a factor ψ and an offset of β to adjust for the fact that the noise power is being estimated [Ric14]:

$$\mathcal{T} = \psi P_n + \beta, \quad (2.21)$$

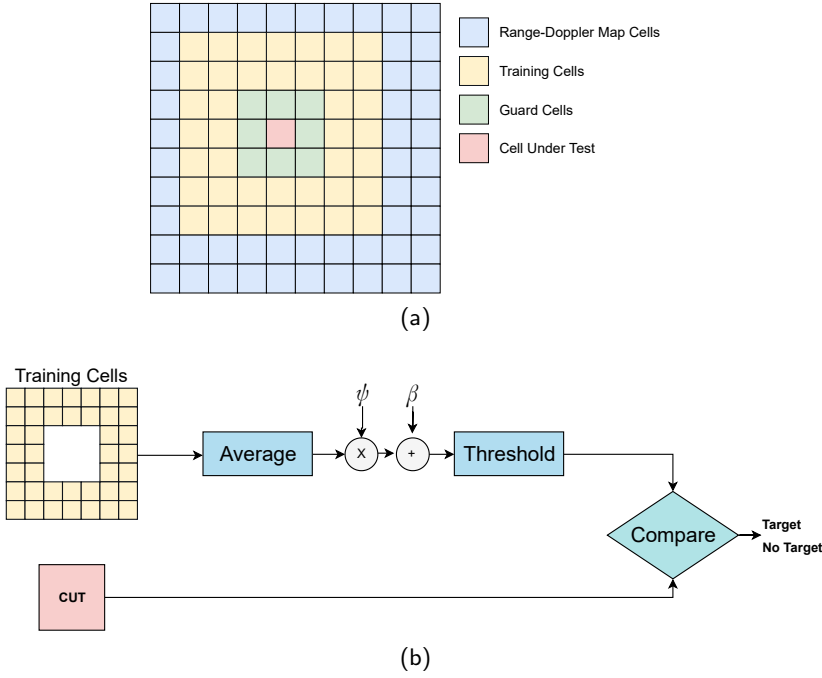


Figure 2.12: An example of: (a) a RD map cells using a 2D CA-CFAR, and (b) its algorithmic pipeline.

where:

$$P_n = \frac{1}{K} \sum_{k=1}^K z_k. \quad (2.22)$$

OS-CFAR

An alternative way to calculate the threshold is to order the training cells statistically instead of averaging them. The performance of a cell averaging constant false-alarm rate detector degrades rapidly in non-ideal conditions. The Ordered-Statistic Constant False Alarm Rate (OS-CFAR or CAOS-CFAR) [Bla88; JYB16; Gam20] is an alternative to the CA-CFAR that, instead of averaging the reference window, sorts the samples in an ascending numerical order to create a new sequence. The κ th element of the ascending list is called the κ th order statistic and is a tunable parameter. In other words, the κ th strongest signal is chosen to be the threshold $\mathcal{T}$ [Ric14]:

$$\mathcal{T} = \psi z_{\kappa}, \quad (2.23)$$

where z_κ is the κ th element of the ascending list, and ψ is a factor parameter to adjust for the approximation uncertainties.

The usage of OS-CFAR is further discussed in [Chapter 3](#).

2.3.3 Direction of Arrival Estimators

Range-Doppler maps are widely useful when used together with a detector in order to obtain the distance and speeds of detected objects. Once the objects are detected, however, we can use the third dimension of the radar cube (snapshots, as shown in [Subsection 2.2.4](#)) and utilize an estimator to retrieve the angular information of the targets.

Direction of Arrival (DoA) estimation is a crucial problem in various fields, including automotive radars [[TF09](#)]. The objective of a Direction of Arrival (DoA) estimator is to determine the directions from which signals of interest are arriving at an antenna array. In the following subsections, we discuss three popular Direction of Arrival (DoA) estimators: MULTiple Signal Classification (MUSIC), Estimation of Signal Parameters via Rotational Invariance Techniques (ESPRIT), and Maximum Likelihood Estimation (MLE).

In order to further explain some of the Direction of Arrival (DoA) estimators, it is important to understand the spatial covariance matrix [[KV96](#)], its notations, and equations. As previously shown in [Equation 2.16](#), the received snapshot is given as: $x(t) = \mathbf{A}(\theta)s(t) + \eta(t)$. Knowing that $E\{\cdot\}$ is the statistical expectation, we can obtain the covariance matrix $\mathbf{R}$ as:

$$\mathbf{R} = E\{x(t)x(t)^H\} = \mathbf{A}(\theta)E\{s(t)s(t)^H\} + E\{\eta(t)\eta(t)^H\}. \quad (2.24)$$

Considering that the source's covariance matrix is given as $\mathbf{P} = E\{s(t)s(t)^H\}$. If the noise is Gaussian, we can consider that the noise covariance matrix is given as $\sigma_\eta^2 \mathbf{I} = E\{\eta(t)\eta(t)^H\}$, where σ_η^2 is the variance of the noise, and $\mathbf{I}$ is the identity matrix, being the unity matrix $\mathbf{U}\mathbf{U}^H = \mathbf{I}$. We can rewrite the covariance equation as [[KV96](#)]:

$$\mathbf{R} = \mathbf{A}\mathbf{P}\mathbf{A}^H + \sigma_\eta^2 \mathbf{I} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H, \quad (2.25)$$

where $\mathbf{\Lambda}$ is a diagonal matrix containing real eigenvalues in descending order (meaning the first eigenvalue greater than or equal to the second eigenvalue and so on). If the signal subspace is orthogonal to the noise subspace, we can separate $\mathbf{\Lambda}$ into two subspaces. $\mathbf{\Lambda}_s$ contains the N strongest eigenvalues of $\mathbf{\Lambda}$ while $\mathbf{\Lambda}_\eta$ contains the $M - N$ weakest eigenvalues of $\mathbf{\Lambda}$. This means that $\mathbf{\Lambda}_\eta = \sigma_\eta^2 \mathbf{I}$ [[KV96](#)]:

$$\mathbf{R} = \mathbf{U}_s \mathbf{\Lambda}_s \mathbf{U}_s^H + \mathbf{U}_\eta \mathbf{\Lambda}_\eta \mathbf{U}_\eta^H. \quad (2.26)$$

In practice, we don't have access to the full covariance matrix. The estimate of the covariance matrix depends on the number of snapshots available. A larger number means a more precise covariance matrix. The estimate of the covariance matrix (also called sample covariance matrix) is used by many techniques and it is given as [KV96]:

$$\mathbf{R} \approx \frac{1}{L} \sum_{i=1}^L x(t)x^H(t). \quad (2.27)$$

This representation is commonly used in subspace-based estimation algorithms. When we have a signal $x(t)$ that is consistent over time, using the covariance matrix $\mathbf{R}$ and its components (eigen-elements) gives us the best possible estimate of what the true signal properties are, even if we don't know the signal's structure.

In the next subsections, we discuss some of the most used techniques to estimate the direction of arrival. We first start with techniques that assume we know the number of targets N . Later, we discuss what happens if the number of sources is also unknown.

2.3.4 MUSIC Algorithm

The MULTiple Signal Classification (MUSIC) algorithm [TF09; KV96; HY10; Gam20] is a widely used DoA estimator. It is based on the concept of signal subspace, which is the subspace spanned by the eigenvectors corresponding to the signals of interest. By assuming that the signal subspace is orthogonal to the noise subspace, the MUSIC algorithm can estimate the DoAs of the signals by projecting the received signals onto the signal subspace. This projection is used to construct a so-called "MUSIC spectrum," which exhibits peaks at the DoAs of the signals. In other words, MUSIC uses eigenvalue decomposition of the estimated covariance matrix to calculate the DoAs.

While the MUSIC algorithm uses a spectral representation to estimate the DoAs, it is considered to be a subspace-based method because it estimates the signal subspace using the eigenvectors of the received signal covariance matrix, rather than directly computing the power spectrum of the received signals. Subspace-based methods are generally more robust than spectrum-based methods, particularly when the signals of interest are weak and/or closely spaced in angle.

More specifically, using the Equation 2.26 and 2.27, we can start the algorithm by calculating the estimated covariance matrix. In the sequence, MUSIC searches for all possible candidate steering vectors $\tilde{\mathbf{a}}(\theta)$ that are orthogonal to the noise subspace. To do this search, the algorithm constructs a pseudo-spectrum

Chapter 2: Background

called MUSIC “spatial spectrum” (or just MUSIC spectrum) $p(\theta)$ that is defined as:

$$p(\theta) = \frac{1}{\tilde{\mathbf{a}}^H(\theta) \mathbf{U}_\eta \mathbf{U}_\eta^H \tilde{\mathbf{a}}(\theta)} . \quad (2.28)$$

When the steering vector is orthogonal to the noise subspace, the pseudo-spectrum exhibits infinite peaks. However, in real-world scenarios, noise is present, and the estimated covariance matrix differs from the true covariance matrix. As a result, the vectors are never exactly orthogonal to the noise. Therefore, the MUSIC spectrum has finite peaks corresponding to the desired directions of arrival. Since we can have more peaks than the number of sources, the MUSIC algorithm requires this as a parameter. The algorithm then selects the N largest peaks. In the case of a linear array, the search space consists of a one-dimensional grid of broadside angles.

To simplify, we can summarize the MUSIC algorithm as a sequence of steps [KV96]:

1. Compute the sample covariance matrix $\mathbf{R}$ based on the collected snapshots (Equation 2.27)
2. Choose the number of signal sources N that we want to estimate.
3. Calculate the eigenvectors and eigenvalues of the covariance matrix $\mathbf{R}$.
4. Use the $M - N$ eigenvectors corresponding to the $M - N$ smallest eigenvalues to form the noise subspace (Equation 2.26).
5. Compute the spatial spectrum $p(\theta)$ of the noise subspace using the MUSIC algorithm for all candidate steering vectors $\tilde{\mathbf{a}}(\theta)$ (Equation 2.28).
6. Identify the N peaks in the spatial spectrum to determine the DoAs of the signal sources.

Despite its promising performance and relatively low complexity, MUSIC has several significant drawbacks. For instance, when the number of snapshots available for estimation is insufficient, MUSIC and other subspace-based techniques, tend to suffer from poor performance [VBC14; HY10]. For example, Tuncer [TF09] suggests a rule of thumb for the number of snapshots of at least three times the number of antennas. Additionally, it is not possible to determine the number of targets using MUSIC [TF09]. Moreover, it assumes Gaussian-distributed noise, which may not always be the case in practice. Finally, in situations involving MRAs, MUSIC tends to detect side lobes as targets, leading to poor performance, as previously shown in Figure 1.7.

2.3.5 ESPRIT Algorithm

Estimation of Signal Parameters via Rotational Invariance Techniques (ESPRIT) [TF09; KV96; HY10; Gam20] is another subspace-based DoA estimator for mul-

multiple signals arriving on an antenna array. The algorithm works by exploiting the rotational invariance property of the signal subspace. Specifically, the steering matrix can be decomposed into two submatrices. The key insight behind ESPRIT is that these submatrices can be estimated independently of each other, allowing for an accurate DoA estimation.

To explore the invariance properties of the subspaces, first, we divide our steering matrix $\mathbf{A}(\theta)$ (from Equation 2.16) into two sets of matrices such as:

$$\mathbf{A}_1 = [\mathbf{I} \ 0] \mathbf{A} \quad ; \quad \mathbf{A}_2 = [0 \ \mathbf{I}] \mathbf{A}, \quad (2.29)$$

where $\mathbf{A}_1$ contains the first $M - 1$ rows of $\mathbf{A}$, and $\mathbf{A}_2$ contains the last $M - 1$ rows of $\mathbf{A}$. With this knowledge, we can relate $\mathbf{A}_1$ and $\mathbf{A}_2$ as:

$$\mathbf{A}_2 = \mathbf{A}_1 \mathbf{H}, \quad (2.30)$$

where $\mathbf{H}$ is a diagonal matrix such as $\text{diag}(\mathbf{H}) = [e^{j\omega_1}, \dots, e^{j\omega_M}]$. This means that the DoA problem can be reduced to finding $\mathbf{H}$.

From a set of subspace approximations (especially in the noise) [KV96], we can also obtain another set of relations:

$$\mathbf{U}_1 = \mathbf{A}_1 \mathbf{T} \quad ; \quad \mathbf{U}_2 = \mathbf{A}_2 \mathbf{T}. \quad (2.31)$$

Further explanations about either $\mathbf{T}$ or $\mathbf{H}$ are beyond our scope. However, from the previous equations, and by defining that $\mathbf{\Psi} = \mathbf{T}^{-1} \mathbf{H} \mathbf{T}$, we can find the relation between $\mathbf{U}_1$ and $\mathbf{U}_2$:

$$\mathbf{U}_2 = \mathbf{U}_1 \mathbf{\Psi}. \quad (2.32)$$

Since $\mathbf{\Psi}$ and $\mathbf{H}$ have similar transformations, they share the same eigenvalues. This means that by solving the Equation 2.32 to find $\mathbf{\Psi}$, we can estimate the direction of arrival. For example, we can use a pseudo-inverse, such as Least-Squares (LS) to find it: $\mathbf{\Psi} = \text{LS}(\mathbf{U}_2, \mathbf{U}_1)$ [KV96].

The last step is to find the eigenvalues of $\mathbf{\Psi}$ to find the DoA estimates $\tilde{\theta}$ [Gam20]:

$$\tilde{\theta} = \sin^{-1} \frac{\arg(\text{eigenvalues}(\mathbf{\Psi}))}{2\pi}. \quad (2.33)$$

To simplify, we can summarize the ESPRIT algorithm as a sequence of steps:

1. Compute the sample covariance matrix $\mathbf{R}$ based on the collected snapshots (Equation 2.27)
2. Choose the number of signal sources N that we want to estimate.
3. Calculate the $\mathbf{U}_1$ and $\mathbf{U}_2$ from the N principal eigenvectors of the covariance matrix $\mathbf{R}$.

4. Solve the approximate relation from Equation 2.32 using, for example, Least-Square.
5. Calculate the eigenvalues of Ψ and find the DoAs (Equation 2.33).

Similarly to MUSIC, ESPRIT also suffers from similar problems, having trouble with fewer snapshots than targets, not being able to determine the number of targets, Gaussian noise assumption, and problems with MRAs [HY10; TF09; KV96].

2.3.6 Spatial Smoothing

As we observed in the past few subsections, the estimation of the covariance matrix is a key component when estimating DoA. For subspace-based techniques, the DoA estimation is related to having a non-singular covariance matrix which often means having a correlated noise (non-Gaussian). Spatial smoothing is a technique used to improve the accuracy of the direction of arrival estimations. This technique is particularly useful when the noise is correlated across multiple sensors. The basic idea behind it is to average the covariance matrix across smaller sub-arrays in order to reduce the impact of the correlated noise [Gam20].

Spatial smoothing helps to increase the effective number of snapshots used in the DOA estimation, resulting in improved performance. However, it also comes with a trade-off between noise reduction and angular resolution. In other words, we can reduce the noise at the cost of reducing the resolution.

In general, this technique involves dividing the antenna array into smaller sub-arrays, each with its own snapshot. Then, we apply Equation 2.27 to the L snapshots coming from L sub-arrays. This is the same as averaging all covariance matrices coming from the sub-arrays. Figure 2.13 shows an antenna array and its sub-arrays.

MUSIC and ESPRIT with spatial smoothing are used in Chapter 5.

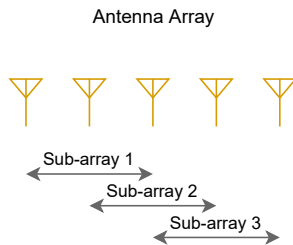


Figure 2.13: An antenna array with 5 elements being divided into separated 3 sub-arrays containing 3 elements each.

2.3.7 Maximum Likelihood Estimation

The Maximum Likelihood Estimation (MLE), also called Deterministic Maximum Likelihood (DML), is a popular parametric algorithm used for DoA estimation. While the spectral and subspace-based algorithms are computationally attractive, they do not always have sufficient accuracy. In particular, for highly correlated signals, their performance may be poor. MLE, on the other hand, provides an efficient and more robust approach at the price of a more computationally intensive algorithm. Instead of relying on a 1-D search, like MUSIC, this approach needs a multidimensional search to find the DoAs [KV96].

The basic idea behind the MLE algorithm is that the algorithm searches through all possible angle combinations that satisfy the signal model to find the one that best fits the signal model [LB22a].

More specifically, the MLE algorithm is a method used to estimate unknown parameters in DoA estimation by comparing the probability of observations given these unknown parameters. It assumes a data model of independent noise sources and deterministic-unknown signals. The likelihood function calculates the probability of all observations given the unknown parameters, including signal parameters, and noise variance. The MLE algorithm calculates the maximum arguments that maximize the likelihood function to estimate these unknowns [KV96].

As an intuitive approach, the MLE algorithm can be summarized as an algorithm that tries to find the minimum value for the residual error $\varepsilon(\theta)$. In a perfect scenario the difference between the residual error and the noise should be zero $\varepsilon(\theta) - \sigma_\eta^2 \mathbf{I} = 0$. Considering that $x(t) = \mathbf{A}(\theta)s(t) + \eta(t)$ (Equation 2.16), we can write the $\varepsilon(\theta)$ as:

$$\varepsilon(\theta) = x(t) - \mathbf{A}(\theta)s(t). \quad (2.34)$$

We can also rewrite the sources $s(t)$ as:

$$s(t) = \mathbf{A}^+(\theta)x(t), \quad (2.35)$$

being $\mathbf{A}^+(\theta)$ the Moore-Penrose pseudo-inverse that can be given as:

$$\mathbf{A}^+(\theta) = (\mathbf{A}^H(\theta)\mathbf{A}(\theta))^{-1}\mathbf{A}^H(\theta). \quad (2.36)$$

Applying the previously shown Equation 2.35, we obtain that the residual error is:

$$\varepsilon(\theta) = x(t) - \mathbf{A}(\theta)\mathbf{A}^+(\theta)x(t). \quad (2.37)$$

Chapter 2: Background

The estimated DoAs $\tilde{\theta} = \tilde{\theta}_1, \dots, \tilde{\theta}_N$ can be found by searching the minimal residual error:

$$\tilde{\theta} = \underset{\theta}{\operatorname{argmin}} \|\varepsilon(\theta)\|^2, \quad (2.38)$$

in which $\varepsilon_{\min} = \min\{\varepsilon(\theta)\}$ is the minimal residual noise obtained after repeating the Equation 2.38 several times to find its minimal value.

From the previous equation, we can intuitively understand that the MLE algorithm searches between all steering matrices, containing all angular combinations, to find the one that produces the least amount of residual error.

Until now, we have considered just a single snapshot. We can generalize and simplify the Equation 2.38 to its most known form [KV96] if the following consideration is done:

$$\frac{1}{L} \sum_{i=1}^L \|x(t) - \mathbf{A}(\theta)\mathbf{A}^+(\theta)x(t)\|^2 = \operatorname{tr}\{(\mathbf{I} - \mathbf{A}(\theta)\mathbf{A}^+(\theta))\mathbf{R}\}, \quad (2.39)$$

where tr denotes the trace of a matrix.

Utilizing the previously shown Equation 2.38, we can obtain an alternative form for the DoA estimation using MLE:

$$\tilde{\theta} = \underset{\theta}{\operatorname{argmin}} \operatorname{tr}\{\mathbf{\Pi}_\mathbf{A}\mathbf{R}\}, \quad (2.40)$$

where $\mathbf{\Pi}_\mathbf{A} = \mathbf{I} - \mathbf{A}(\theta)\mathbf{A}^+(\theta)$.

The interpretation of the previous equation is that the snapshots $x(t)$ are projected onto a model subspace that is orthogonal to all expected signal components, and we evaluate the power measurements of each projection. The energy of the measurement will be the smallest when the projector effectively removes all true signal components ($\tilde{\theta} = \theta$). However, due to a limited number of noisy snapshots, the energy is not perfectly measured, resulting in estimated DoAs that deviate from the true ones [KV96].

In Figure 2.14, a 3D plot is shown where the power of the inverse residual errors (ε^{-1}) are plotted against all possible angles. This plot helps to visualize how maximum likelihood estimation searches for all possible steering matrices that contain all possible DoAs. In this particular case, a 2D MLE search was performed as we have two targets. The peaks in the plot represent the locations where there are sources, which are at the angles of -20° and 5° , in this case.

The multidimensional search done by the MLE algorithm depends on its search grid. The search grid depends on the Field Of View (FOV) and the angular resolution. The field of view represents the angular extent of the search

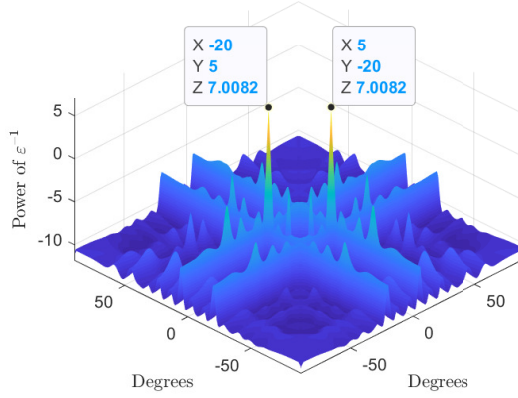


Figure 2.14: A 3D plot containing all the possible angle combinations for 2 targets (-20° and 5°), from -90° to 90° , versus the power of the inverse of the residual error.

grid. Angular resolution refers to the minimum angle within the search grid at which the MLE algorithm can still distinguish between two distinct targets. Considering a certain FOV and a certain angular resolution $\Delta\theta$, we can define the size of a search grid G for MLE as:

$$G = \frac{\text{FOV}}{\Delta\theta} + 1, \quad (2.41)$$

where each element in the search grid is evenly spaced from its neighboring elements.

One of the main disadvantages of using the MLE algorithm to estimate DoAs is its computational complexity, which can grow exponentially with an increasing number of targets. The MLE algorithm requires an exhaustive search over all possible directions to find the angles of the sources. As the number of DoAs increases, the search space grows exponentially, leading to a significant increase in computational complexity. This can make the MLE algorithm impractical for real-time applications or situations where there are many sources of signals to be estimated [LB22a]. Given the search with size G and the number of targets N , the number of searches Θ that the MLE algorithm is required to do is:

$$\Theta_{\text{MLE}} = \binom{G}{N} = \frac{G!}{N!(G-N)!}. \quad (2.42)$$

Chapter 2: Background

For example, considering a FOV=180° and a resolution $\Delta\theta=1^\circ$, a 1D MLE will consider 181 angle combinations, a 2D MLE 16×10^3 , a 3D MLE 94×10^4 , and a 4D MLE 42×10^6 angle combinations.

It is also important to note that this MLE algorithm assumes that the noise is Gaussian such as $\mathcal{N}(0, \sigma_\eta)$. This means that in real-world scenarios, or high-noise environments, this algorithm might not operate as expected.

On the other hand, the maximum likelihood estimation algorithm is robust and reliable. According to Hacker and Yang [HY10], the MLE algorithm is more reliable than other techniques when estimating 2 targets using a single snapshot and a given minimum redundancy antenna array (MRA).

The MLE algorithm is further discussed in Chapter 5, Chapter 6, and Chapter 7.

Finally, we can summarize the MLE algorithm as a sequence of steps:

1. Compute the sample covariance matrix $\mathbf{R}$ as in Equation 2.27.
2. Based on the number of signal sources N , compute all possible steering matrices $\mathbf{A}$.
3. Run equation Equation 2.40 $\binom{G}{N}$ times.
4. Find the steering matrix $\mathbf{A}(\theta)$ that contains the minimum amount of residual $\varepsilon(\theta)$.
5. Find the angles $\tilde{\theta}$ that form such steering matrix $\mathbf{A}(\theta)$.

In the following subsection, we discuss how to estimate the number of targets using snapshots.

2.3.8 Estimating the Number of Targets with MLE

As mentioned previously, several DoA estimators require prior knowledge of the number of targets present in the scene. Therefore, it is essential to estimate the number of sources before or during the DoA estimation using snapshots [Gam20].

However, this task can sometimes be overlooked. Many researchers estimate DoAs expecting the number of targets to be already known, when in reality, retrieving the number of sources can be a non-trivial task itself. For example, if a DoA estimation technique can not distinguish the angles of two or more closely spaced targets, it may not be an appropriate technique to estimate the number of sources

MLE is among the algorithms that can be used for estimating the number of targets. If the background noise is known (or accurately estimated), the maximum likelihood can compare the minimal residual error (Equation 2.38) with the known noise. If the difference is above a certain stipulated threshold, most likely, there is another target in the scene elevating the residual error's estimate.

In other words, if done correctly, $\varepsilon(\theta) - \eta \approx 0$. This means that if $\varepsilon(\theta) - \eta > \mathcal{T}$, where $\mathcal{T}$ is a threshold, probably there is another target in the scene.

We can write the algorithm that allows the MLE algorithm to estimate the number of targets as a sequence of steps:

1. The algorithm assumes that there is one target in the scene and performs a 1D search to find it.
2. If the difference between the residual error and the background noise is bigger than a stipulated threshold, it assumes that there are two targets in the scene instead of one and proceeds to the next step. If the difference is smaller or equal to the threshold, the algorithm stops in this step as it has found both the DoA and the number of targets (1 target).
3. The algorithm assumes that there are two targets in the scene and performs a 2D search.
4. Once again, the algorithm compares the residual error with the noise and decides to continue or to stop.
5. The repetition of assuming the number of targets and calculating the difference continues until the algorithm finds DoAs and the number of targets or until it reaches a pre-established stopping point.

As previously shown in [Equation 2.42](#), if more than 1 target is present, the number of searches increases exponentially. To find the number of targets and their angles, it may be necessary to pass through several iterations of a multi-dimensional search, each more computationally intense than the previous one. For this reason, this method is challenging to use for real-world scenarios with latency constraints.

We discuss the use of the MLE algorithm to estimate the number of sources in [Chapter 5](#) and [Chapter 7](#).

For the following sections, we depart from analytical models and enter the data-driven domain that is extensively used throughout this thesis.

2.4 Deep Learning

Deep Learning (DL) is one of many types of Machine Learning (ML), which is itself a category within the broader field of Artificial Intelligence (AI). The Machine Learning field consists of building algorithms that rely on a gathering of examples (data) to statistically model a certain phenomenon. These examples could be generated (e.g. synthetic data), occur naturally (e.g. weather) or obtained by the usage of sensors (e.g. radars, cameras, lidars) [Bur19].

Although often used as an interchangeable word with Neural Network (NN), Machine Learning is a big field that incorporates many different techniques with various applications, being Neural Networks (and Deep Learning) one of them [Bis06]. Machine learning itself belongs to a broader field of AI, which refers to the development of intelligent machines that can perform tasks that are typically thought to require human intelligence. In Figure 2.15, we show a Venn diagram showing how deep learning is positioned inside the AI domain. For convenience, in this thesis, all these terms are used interchangeably, unless stated otherwise.

It is important to note that deep learning is just one of many techniques and as such, it should not be treated as the ultimate solution to all research questions, but as one of many tools in the hands of a proficient researcher.

This thesis discusses novel techniques and models for applying deep learning to radar data. For that, neural networks are further explained to provide the necessary foundation for the proposed techniques and models.

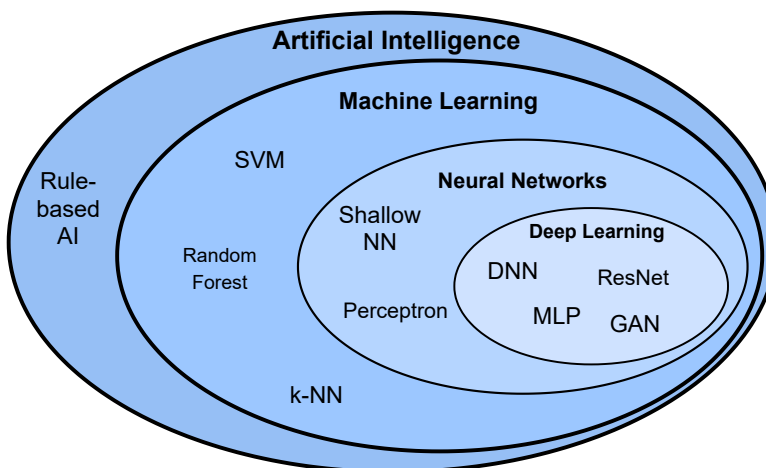


Figure 2.15: A Venn diagram showing the differences between Artificial Intelligence (AI), Machine Learning (ML), Neural Network (NN) and Deep Learning (DL)

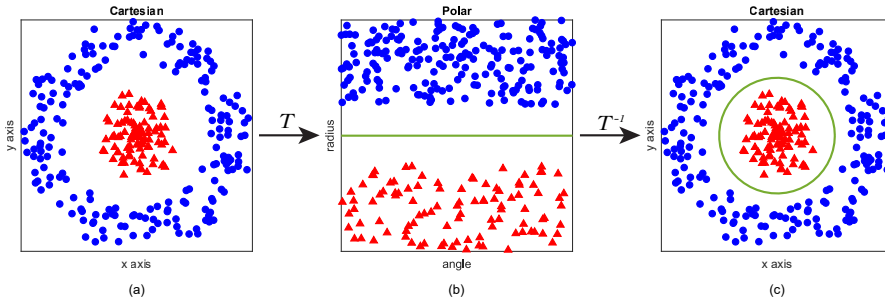


Figure 2.16: An intuitive example of how machine learning works. It is easier to separate the two sets of points (a) if we first make a transformation from Cartesian to Polar. In a Polar system, we can easily draw a straight line to separate the points (b). Afterward, we can make an inverse transformation back to Cartesian (c).

An intuitive example of how machine learning works - and why it is beneficial to use it in some scenarios where data is available - is how to separate two sets of points, as shown in Figure 2.16 (a), using a simple line. As can be noticed, it is impossible to separate the blue circles from the red triangles with a simple straight line. However, if we apply a transformation T that maps all points from their Cartesian form to their Polar form, we can easily separate the two sets of points with a single straight line, as shown in (b). After solving the problem in the easiest system, we can map it back to the original Cartesian form with an inverse transformation T^{-1} , as shown in (c) [GBC16].

In other words, another way to understand intuitively some of the machine learning algorithms, such as neural networks, is to think that these algorithms apply a set of transformations to the data in order to extract features and better understand the similarities between various sets of points, vectors, or matrices.

2.4.1 Deep Learning Tasks

Various sorts of tasks can be done by a deep learning model if sufficient data is provided. Some of these tasks are the current state-of-the-art within some fields. According to Goodfellow et. al. [GBC16], classification, regression, denoising, anomaly detection, clustering, transcription, translation, and others are some of the tasks that can be solved using deep learning. This thesis focuses on classification, regression, denoising, detection, and estimation.

- **Classification:** In this type of task, the machine tries to specify which of the known categories a specific type of data belongs to. Example: the

machine distinguishes between pictures of dogs and cats.

- **Regression:** In this task, the machine tries to predict a numerical value for a given input. Example: predicting future stock prices based on previous records.
- **Denoising:** It is a type of regression. This task tries to recover a clean version of the data, given a corrupted input. Example: cleaning noise from a corrupted music file.
- **Detection:** In most cases, it is a type of classification that tries to identify if there are objects present at the input or not. Example: face detection. Models can be trained to detect faces in images and videos.
- **Estimation:** In most cases, it is a type of regression that tries to predict a true numerical value given a certain type of data. Example: Estimating age in pictures or videos.

2.4.2 Types of Learning

Machine learning algorithms can be classified depending on how they learn with the provided data. Although we mainly use supervised learning, it is important to understand that there are other kinds of learning methods [Bur19].

- **Supervised learning:** In this learning type, the model is trained by showing it examples of what you want it to do. For this, the machine is given data that has already been labeled, targeted, or marked, with the desired answer. Then, it uses this labeled data to learn the relationship between the provided input and output. For example, showing to a computer labeled images of fruits to predict the type of fruit in a new image.
- **Unsupervised Learning:** For this type, the model is trained on unlabeled data to try to find patterns and relationships within the data. The goal is to discover features in the data without being told what these features represent. This is often used to find patterns and group similar data points. For example, grouping similar fruits in a picture full of fruits.
- **Reinforcement Learning:** In this learning process, an agent learns to make decisions in an environment by performing actions. Each action leads to a reward or a punishment, and the goal is to find the best decisions, or policy, that will lead to the most reward and less punishment over time. For example, a computer learns to play a game through repeated trial and error.

2.4.3 Neural Network's Learning

Training is an essential step in the development of all types of neural networks. Here, we provide an overview and a high-level discussion of the main concepts

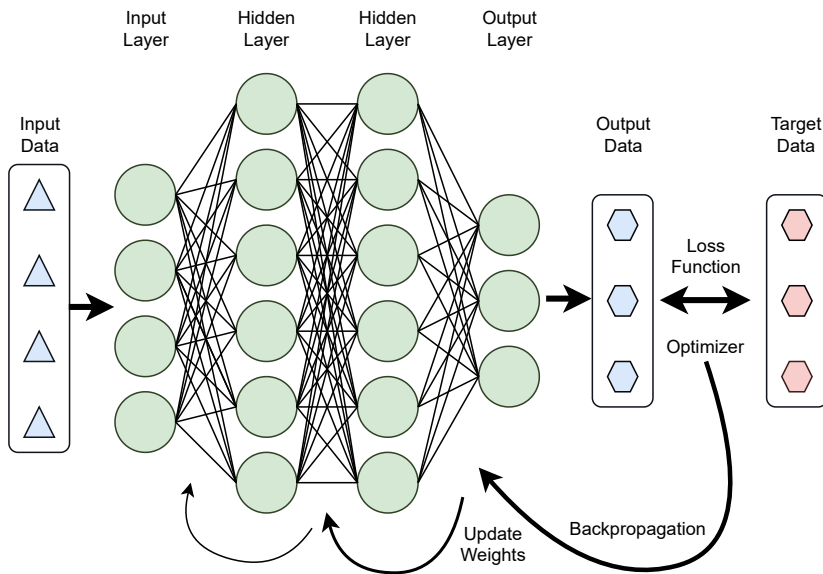


Figure 2.17: An example of a supervised training process used for neural networks.

behind training neural networks.

As the majority of the models used in this thesis rely on some form of supervised learning, understanding it better is essential for later chapters. In addition, supervised learning is the easiest and most intuitive one. Often we humans also learn by seeing pre-existing examples.

During the training of a neural network, a supervised learning algorithm compares the provided label with the output data, generating a loss. The loss function calculates the error between the output values and their corresponding target. In the sequence, the weights of this neural network are updated using an optimizer usually running a backpropagation algorithm. Backpropagation computes the gradient of the loss function, updating the current weights to minimize the loss [Bur19].

In Figure 2.17, we show an example of a multilayer perceptron being trained with a supervised algorithm [Bis06]. The input data passes through the machine and it generates the output data. This data is then compared with its target (labeled) data using a loss function that is used to update the weights of the neural network using an optimizer with backpropagation.

The training process is very important for NNs because it is when the model “learns” which features to extract for its designed task. An intuitive way to think about feature extraction is that each layer of a DL model extracts some sort of

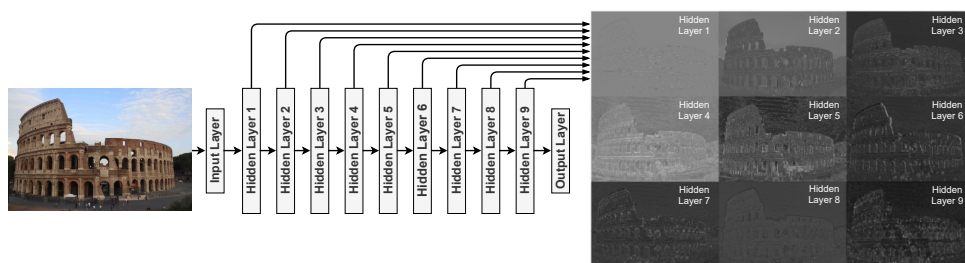


Figure 2.18: An example of a deep neural network and its features extracted in each layer. Since the feature extraction happens in higher dimensions, this is a projection of the features.

features in a higher dimension space. In [Figure 2.18](#), we show an example of feature extraction. Each layer is responsible for the extraction of some important features that help the neural network perform its designed task [\[GBC16\]](#).

Some concepts - such as loss function, backpropagation, optimizer activation function, and batch normalization - are essential to understanding the functioning and training of deep learning models [\[Bur19; GBC16\]](#) and are discussed in the sequence. These concepts are going to be useful moving forward as they help to better understand some of the terminology that will be used from here on.

Although it is outside the scope of this thesis to define every single neural network equation and model, it is important to give a quick review of the basics. For more in-depth information, see [\[Eng19; Ska18; Sch14; GBC16; Agg18; Bur19; Zha+21a\]](#).

Loss Function

The loss function is a crucial component of supervised learning. It measures the error (loss) between the predictions made by the model and the actual target values. The goal of training is to minimize this error by adjusting the weights and biases on each iteration (batch or epoch), normally through backpropagation. Different loss functions are used for different types of problems, such as regression, classification, segmentation, and so on. The choice of loss function depends on the nature of the problem and the desired outputs from the model [\[GBC16\]](#). Mean Square Error (MSE), and Binary Cross-Entropy (BCE) are examples of loss functions used for regression and classification, respectively.

Mean Square Error (MSE) is a vastly used loss function for supervised learning, usually for problems associated with regression. This algorithm works by calculating the average of the squared differences between the predicted and true values [\[Agg18\]](#). Considering K as the number of samples, y as a vector

containing the true values, and $\tilde{y}$ as a vector containing the estimated values, we can define the MSE loss function $\mathcal{L}_{\text{MSE}}$ as:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{K} \sum_{i=1}^K (y_i - \tilde{y}_i)^2. \quad (2.43)$$

Another commonly used loss function for supervised learning is the Binary Cross-Entropy (BCE), but its usage is normally associated with classification problems, where the goal is to predict a binary label. This algorithm works by calculating the difference between the predicted probabilities and the true binary label [Bur19]. In this case, y can just take binary values (0 or 1), and the $\tilde{y}$ values between 0 and 1. We can define BCE loss function $\mathcal{L}_{\text{BCE}}$ as:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{K} \sum_{i=1}^K [y_i \log(\tilde{y}_i) + (1 - y_i) \log(1 - \tilde{y}_i)]. \quad (2.44)$$

Backpropagation

The process of adjusting the parameters in a neural network during training is typically performed using the algorithm known as backpropagation. It is an efficient algorithm for computing the gradients of the loss function with respect to the model's parameters, which are used to update them during each iteration of training [Bur19].

Optimizer

For traditional supervised learning, an optimizer uses the gradients computed by backpropagation to adjust the parameters in a direction that reduces the loss. There are many different types of optimizers, each with its own strengths and weaknesses, such as Stochastic Gradient Descent (SGD), Root Mean Squared Propagation (RMSProp), and Adaptive moment estimation (Adam) [SG71; Hin12; KB15].

Stochastic Gradient Descent (SGD) [SG71] and its variations are popular optimizers for training NNs models for several decades. It is an iterative method that updates the model parameters based on an estimate of the gradient of the loss function. In each iteration, the algorithm randomly selects a small batch of training examples and computes its gradient. Then, the weights are updated by taking a small step in the direction of the negative gradient. The size of the step depends on the learning rate. At the end of several steps, it is expected that the model converged to a minimum point, meaning that probably the NN has learned a set of optimal parameters [Bur19; GBC16]. Considering w as a vector

Chapter 2: Background

containing the weights of a certain neural network, lr is the learning rate, and $\nabla \mathcal{L}(w)$ is the gradient of the loss function $\mathcal{L}$ with respect to the weight w , we can define the naive SGD as:

$$w_{\text{new}} = w - lr \nabla \mathcal{L}(w) . \quad (2.45)$$

Other optimizers, such as RMSProp [Hin12], and Adam [KB15] are widely used for training neural networks, and they are variations of the SGD algorithm. There are plenty of applications that use them as the main optimizer. Although we have used these algorithms, their equations and further discussion are beyond the scope of this thesis [GBC16].

Activation Function

The activation function g is a pre-determined, non-linear function applied element-wise in a neural network. It transforms the output of a neuron, allowing the network to learn complex representations and make more accurate predictions. As its name suggests, an activation function works by controlling how much a certain neuron is activated or not. Some common activation functions include sigmoid, Rectified Linear Unit (ReLU), and hyperbolic tangent (tanh) [Agg18; GBC16; Bur19]. For example, ReLU is widely used between layers of deep learning models.

In Figure 2.19 we show the plot of three commonly used activation functions. The sigmoid function is usually associated with multilayer perceptrons and it has been used for several decades [Agg18]. The sigmoid of a certain input z can be represented as:

$$g_{\text{sigmoid}} = \frac{1}{1 + e^{-z}} . \quad (2.46)$$

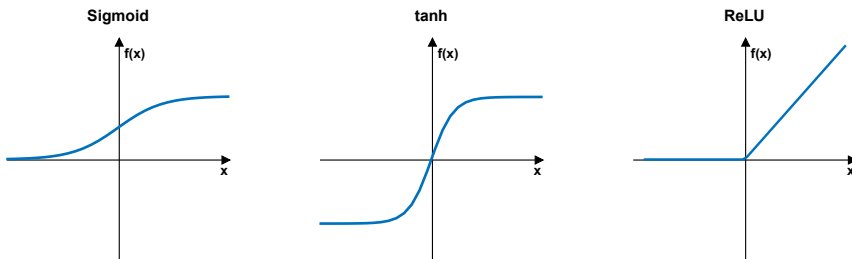


Figure 2.19: Comparison of three activation functions: sigmoid, tanh, and ReLU. Activation functions play an important role in the non-linearity properties of neural networks.

The tanh activation function [Agg18], as expected, is the hyperbolic tangent of a certain input z :

$$g_{\tanh} = \tanh(z) . \quad (2.47)$$

Rectified Linear Unit (ReLU) started outperforming sigmoid-based networks around 2011 [Sch14], becoming the standard activation function for a while [GBC16]. It is still popular and widely used. The ReLU of a certain input z can be defined as:

$$g_{\text{ReLU}} = \max(0, z) . \quad (2.48)$$

The softmax activation function is often used together with the binary cross-entropy loss function, as it brings all outputs of a neural network between 0 and 1, in a probability-like distribution. It can be described as a generalization of the sigmoid function to multidimensional data [Bur19]. The softmax activation function can be defined as:

$$g_{\text{softmax}} = \frac{e^{z_j}}{\sum_{i=1}^I e^{z_i}} , \quad (2.49)$$

where I is the number of classes and z_j is the input to the function for a class j .

Regularization

Regularizations are techniques that are used in machine learning models to prevent or reduce overfitting. Overfitting happens when a model is trained “too well” on a certain training data and it starts “learning the noise” of the data rather than the features of the data itself. In other words, the model starts “memorizing” details of the data instead of learning patterns [GBC16].

Regularization often involves ways to discourage the model from assigning excessive importance to any particular feature, making the model more general. L1, L2 regularization, and dropout are common strategies [Bur19].

Dropout helps to prevent overfitting by randomly turning off a certain percentage of neurons in a layer during training. This forces the model to learn redundant pathways and reduce overfitting [Agg18].

Batch Normalization

Batch normalization is a technique used in deep learning to make the training process faster and more accurate. It involves standardizing the outputs of each layer and helps reduce the internal changes in the network during training. Its goal is to ensure that the inputs have zero mean and unit variance [Agg18].

Chapter 2: Background

In more detail, the activations are normalized by subtracting the mean and dividing by the standard deviation, so that the mean is zero and the variance is one. This normalization helps to prevent the network from producing large, unstable activations that can cause training to become unstable or fail altogether. Additionally, batch normalization may have a regularizing effect.

2.5 Deep Learning Architectures

A deep learning model can be defined as a neural network with more than two hidden layers. There are several architectures for deep learning and the number just keeps growing in this fast-paced field. In this section, we discuss the main architectures that are used in further chapters.

2.5.1 Fully Connected Neural Network

The Fully Connected Neural Network (FCNN), also known as Multi Layer Perceptron (MLP) or Feed Forward Neural Network (FFNN), is one of the simplest and oldest types of NNs, as it dates back to the 60's [Sch14]. However, it is still relevant and useful both within other architectures and when solving simpler tasks. It can be considered as the stepping stone of deep learning.

In this kind of NN, the information always flows forward. Often, every neuron of each layer is connected with all neurons of the following layer. If all neurons on a certain layer are fully connected to each other, this layer is called Fully Connected (FC).

Figure 2.20 shows that each individual neuron outputs the sum of the product between the values that departed from the previous neurons and the connection weight between neurons. The bias is added to the sum and everything is multi-

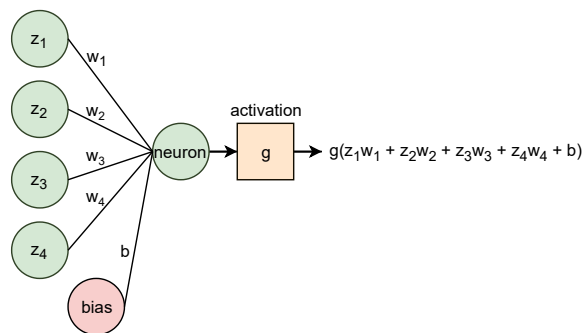


Figure 2.20: An example of a neuron. The output of a single neuron is the weighted sum of the inputs, plus the bias, multiplied by an activation function.

plied by an activation function. In the beginning, before training, all weights in all layers are initialized with a random value.

We can define that for each layer l of the NN, there is a mathematical function f_l that maps the input z to the output $f_l(z)$ of that layer.

More formally [Bur19], instead of using the separate weights, we may use a weight matrix $\mathbf{W}_l$ with the same number of elements as there are connections between neurons within a layer l . The bias constant b_l is a vector containing extra parameters to offset the output, giving it more freedom. The activation function g_l may also change by the layer. The output $f_l(z)$ of a fully connected layer is given as:

$$f_l(z) = g_l(\mathbf{W}_l z + b_l). \quad (2.50)$$

As an essential building block for several neural networks, fully connected layers are used in almost all DL architectures discussed in this thesis.

2.5.2 Convolutional Neural Network

The convolutional layer is another fundamental building block in the architecture of several neural networks. The Convolutional Neural Network (CNN), also called ConvNets, was first introduced in 1979 by Fukushima [Sch14] but just entered the mainstream research world after winning the 2012 ImageNet [KSH12]. Because of this, traditionally, CNNs is widely used for image processing. Several modern NN architectures utilize convolutional layers in them, including the ones we further discuss in this section, such as CAEs, ResNets, and GANs.

In this kind of network, unlike in the FCNN, the weights of each layer are shared (here they are also called filters or kernels), reducing the number of learning parameters without losing much (if any, for many cases) accuracy [Bur19]. At the layer level, this neural network works by convoluting the filter with the input data of a layer producing an output⁹.

The output from CNNs can also be called a feature map as it is expected to contain extracted features from the data. In general, it is expected that each layer will extract different features of the input data. An example of feature extraction using CNN was previously shown in Figure 2.18.

Figure 2.21 shows a typical convolutional neural network used for classification. The input data passes through convolutional layers, activation functions, and pooling operations. Pooling is a way to reduce the parameters of the NN even more by summarizing statistically neighbors' data points [GBC16]. For example, the max pooling operation outputs the max value of neighbors' points. At the end of a typical classification CNN, a few fully connected layers (like

⁹ <https://mllimadeoliveira.github.io/Thesis/2dconv> Example of a 2D Convolution.

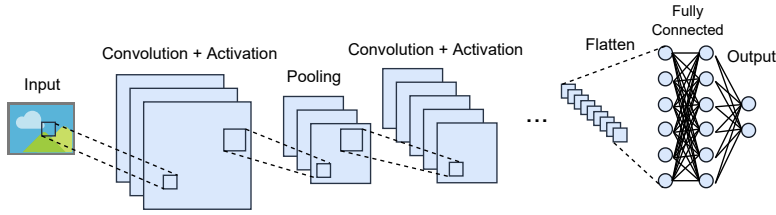


Figure 2.21: A Convolutional Neural Network (CNN) architecture applied for classification. The three most basic parts of a CNN are the convolutional operations, the activation function, and the pooling operation. In the end, some fully connected layers are used for classification.

FCNN) will exist. In this case, the output will be a probability-like distribution. For example, it can be used to classify pictures of cats and dogs.

More formally [GBC16], consider $\mathbf{K}$ the kernel of a 2D CNN of dimensions m, n , and a 2D input data $\mathbf{Z}$ of a certain layer l . The output $f_l(\mathbf{Z})$ from this layer l is the convolution between the input $\mathbf{Z}$ and $\mathbf{K}$, and it has a dimension of i, j :

$$f_l(\mathbf{Z})_{(i,j)} = (\mathbf{Z}_l * \mathbf{K}_l)_{(i,j)} = \sum_m \sum_n \mathbf{Z}_{l(m,n)} \mathbf{K}_{l(i-m, j-n)}. \quad (2.51)$$

It is important to note that, although the size of the input is not explicitly stated in the previous equation, in practice, the input size is known and fixed as the convolutional layers are usually designed to take in inputs of a fixed size.

Although CNNs became famous for their image processing capabilities, the general idea can be expanded to both 1D convolutions and 3D convolutions. The dimensionality of the convolutional layer determines the type of convolution being performed and it is related to the sort of input that the neural network has to deal with. 2D convolutions are commonly used for image-like data, while 1D convolutions [Kir+21] can be used for audio or stock analysis, and 3D for point clouds or videos. For example, Ji et. al. [Ji+13] were among the first works to propose a 3D CNN to extract spatial and temporal features from video data.

The 1D convolutional¹⁰ operation [Kir+21] can be seen as a simplified version of the Equation 2.51. Considering a kernel vector k with m elements, and an input z for a layer certain l , the output $f_l(z)$, with i elements, for this 1D convolution layer would be:

$$f_l(z)_{(i)} = (z_l * k_l)_{(i)} = \sum_m z_{l(m)} k_{l(i-m)}. \quad (2.52)$$

¹⁰ <https://mllimadeoliveira.github.io/Thesis/1dconv> Example of a 1D convolution.

Convolutional layers are widely used in deep learning, forming the basis of networks like convolutional autoencoders, residual networks, and generative adversarial networks, which are discussed in the following subsections.

2.5.3 Convolutional Autoencoder

The original FCNN autoencoder dates back to 1987 [Sch14]. An autoencoder is often described as a kind of NN that is designed to attempt copying its input to its output on an encoder-decoder architecture [GBC16]. A typical autoencoder looks like an hourglass with a bottleneck layer in the middle. The idea is that the bottleneck layer will contain the essential features from the input, but in a lower dimensionality than the original input [Bur19]. In other words, an autoencoder takes the input from a higher-dimensional space to a lower dimension, with the end goal of recreating (approximately) the input. Besides copying, it can be used to denoise, generate new data, to pre-train, and others. For the denoising task, the autoencoder must undo the corruption of its input instead of purely copying it.

A Convolutional AutoEncoder (CAE) is a type of neural network that uses convolutional layers as part of its architecture [Agg18] to perform dimensionality reduction to the input data, therefore, working as an autoencoder. In Figure 2.22, we show a convolutional autoencoder used for denoising an image. CAE can learn features of the data and it tries to reconstruct the original data.

The two main blocks of a CAE are the encoder and the decoder [Agg18]. The role of the encoder is to convert the input data into a compressed latent representation with lower dimensions. The encoder often consists of multiple layers of convolution and pooling operations, gradually decreasing the spatial dimensions of the input, while simultaneously increasing the number of feature maps (e.g., the number of channels). The latent representation refers to a compressed, low-dimensional, and meaningful representation of the input data. The decoder takes the compressed latent representation generated by the encoder and attempts to reconstruct the original (or modified) input. It typically consists

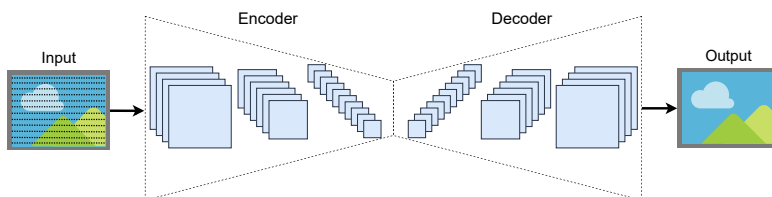


Figure 2.22: A denoising Convolutional AutoEncoder (CAE) with bottleneck for feature extraction and dimensionality reduction.

of a series of upsampling and convolutional layers that gradually increase the spatial dimensions of the input, while decreasing the number of channels to produce the final output.

More specifically, the encoder can be represented mathematically as:

$$h = f_{\text{enc}}(z), \quad (2.53)$$

where f_{enc} is the encoder function that takes the input z and maps it to a lower-dimensional latent space representation h . The decoder, on the other hand, takes the latent representation h and maps it back to the original (or modified) input space z' through the decoder function f_{dec} :

$$z' = f_{\text{dec}}(h). \quad (2.54)$$

This type of neural network is further discussed in [Chapter 3](#) and [Chapter 4](#).

2.5.4 Residual Neural Network (ResNet)

The original Residual neural Network (ResNet) [He+16] was the first 152 layers deep NN, being an order of magnitude deeper than previous architectures, achieving (for the first time) human-level performance of classification [Agg18]. Training such deep architectures was not possible before the integration of the idea of skipping connections. There are a few explanations of why skipping connections improves the performance of NNs. One explanation is that, while training with backpropagation, the gradient might explode or vanish, making it harder to train the earlier layers. Skipping connections might help avoid the vanishing/exploding gradient by creating bypasses in the network.

The building blocks of residual Neural networks rely, usually on 2 convolutional layers (with Batch Normalization) followed by an activation function (ReLU). Then, the input is summed to the output. In [Figure 2.23](#), we show a comparison of: (a) a convolutional layer without skipping connections used in traditional CNNs; (b) ResNet identity block, copying the input to the output in order to train it deeper; (c) ResNet convolutional block, gives an extra convolution to transform the input into the desired shape for the addition operation [Zha+21a].

Although the original ResNet was initially designed to be an image processing NN which won the ImageNet in 2016 [Zha+21a], it can be modified to use 1D or 3D convolutions. A 1D Convolution is a convolution between two vectors instead of two matrices. A 1D ResNet is a Residual Neural Network that relies on 1D convolutions and skip-connections.

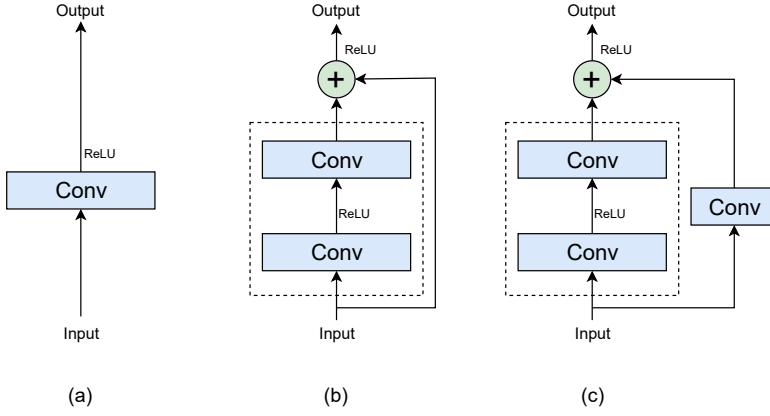


Figure 2.23: Residual neural Network (ResNet) blocks: (a) Just a convolutional layer and no skipping connection; (b) identity block; (c) convolutional block

More specifically [He+16], we can define that a certain convolutional layer is a function f that maps the input z to a feature map F , regarding a set of weights w :

$$F = f(z; w) . \quad (2.55)$$

For an identity block of a ResNet [He+16], we can define that a certain feature map F is obtained through the sum of a function f that maps the input z with the input itself, regarding a set of weights w :

$$F = f(z; w) + z . \quad (2.56)$$

Finally, a convolutional block of a ResNet [Zha+21a] is almost the same as the identity block, but with one extra convolution to transform the input into the desired shape. This means that a new set of weights w_s is introduced:

$$F = f(z; w) + w_s z . \quad (2.57)$$

ResNets are quite used in this thesis. In Chapter 5 we utilize 2D ResNets. In Chapter 6 and Chapter 7 we use 1D ResNets.

2.5.5 Generative Adversarial Network (GAN)

The original Generative Adversarial Network (GAN) was initially proposed by Goodfellow [Goo+14] in 2014. The simplest idea behind GAN, which was based on game theory, is having two neural networks (generator and discriminator)

Chapter 2: Background

competing against each other while improving themselves, in the form of a zero-sum game.

During training, both networks are trained simultaneously. The generator neural network directly produces samples, while the discriminator neural network tries to identify if a drawn sample belongs to the generator or to the training dataset. A probability value is then produced by the discriminator, indicating the likelihood that a given sample is a real training example rather than a fake created by the generator. In short, the goal of the generator is to minimize the discriminator's detection of its data, while the goal of the discriminator is to maximize its own identification between real and fake (generated) data. When a GAN reaches convergence, the generator produces samples that are indistinguishable from real data for the discriminator. At this point, the discriminator outputs a probability of 0.5 for all samples, indicating that it is equally likely that a sample is real or fake. Here, the discriminator is no longer needed and can be discarded [GBC16].

However, GANs can also be problematic as training can be quite unstable. Due to the adversarial nature of the model, it is not uncommon for the generator and discriminator to get stuck in a cycle where the generator produces poor-quality samples that the discriminator can easily identify as fake. Another scenario is that the discriminator may not identify at all the difference between fake and real too early in the training, causing poor-quality samples coming from the generator. In summary, achieving a stable convergence can be a challenge [GBC16].

Besides that, GANs can also suffer from model collapse, when the generator produces samples that are all too similar; high complexity, as GAN models might have too many hyperparameters to be tuned and adjusted for a good performance; black box designed, as it is generally quite hard to understand how each sample is being produced; and ethical considerations, especially considering the growth of deepfake models.

Yet, GANs can be a powerful tool for generating synthetic data and has many applications in areas such as data augmentation, content creation, and anomaly detection [GBC16]. These NNs have been used to generate a wide variety of data, including (but not limited to) images, music, and text.

In **Figure 2.24**, we show the model of a typical most simple type of GAN architecture. The generator receives white noise as input and tries to generate some data to deceive the discriminator. The discriminator, on the other hand, receives a batch containing both real and fake¹¹ data and it needs to distinguish them.

¹¹ The fake images were not created using a GAN technique, but generated using Copilot Image Creator from Microsoft Designer www.bing.com/images/create.

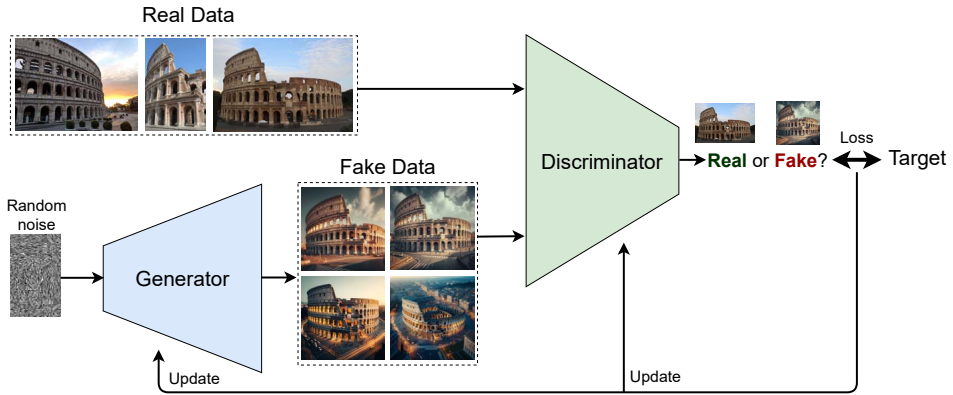


Figure 2.24: An example of a Generative Adversarial Network (GAN) training. The generator tries to fool the discriminator, while the discriminator tries to distinguish between real and generated data.

Being more specific [Goo+14; GBC16], considering $\mathcal{G}(w_g; n)$ as the generator synthetic data sample, where w_g is a parameterized set of weights and biases for the generator, and n is an input noise vector; and considering $\mathcal{D}(w_d; z)$ as the likelihood that the input data z is real rather than fake, where w_d is a parameterized set of weights and biases for the discriminator; the objective of the generator is to minimize the loss function $\mathcal{L}(w_g)$ given as:

$$\mathcal{L}(w_g) = \min(-\log(\mathcal{D}(w_d; \mathcal{G}(w_g; n))), \quad (2.58)$$

while the goal of the discriminator is to minimize the loss function $\mathcal{L}(w_d)$ that maximizes its detection rate:

$$\mathcal{L}(w_d) = \min(y \log(\mathcal{D}(w_d; z)) + (1 - y) \log(1 - \mathcal{D}(w_d; z))), \quad (2.59)$$

where $y = 1$ if the training sample is real, and $y = 0$ if the training sample is fake [Zha+21a].

We utilize a GAN architecture in [Chapter 4](#).

2.6 Advanced Deep Learning Topics

In this section, we further discuss the field of machine learning by briefly discussing the ideas behind some of the advanced techniques that are often used by deep learning engineers to achieve better results when designing new models. For more in-depth information, see [Eng19; Ska18; Sch14; GBC16; Agg18; Bur19; Zha+21a].

2.6.1 Reducing Model Complexity

Deep neural networks can become extremely large and computationally intensive, especially when dealing with high-dimensional data and complex architectures. Although large models can be an excellent starting point as a way to showcase novel techniques, they can be difficult to deploy into embedded devices with limited computational resources and memory. For this, lighter models may be necessary in order to reduce the overall computational footprint. There are a few techniques that can be used to achieve a reduced lighter model:

- **Pruning:** Pruning is a technique to reduce the complexity of a neural network by removing neurons and connections with the smallest impact on the model's accuracy [Agg18]. For example, a model can become lighter and faster by removing the weights of the smallest magnitudes (closer to zero) out of a neural network.
- **Dimensionality Reduction:** Although a more complex input data, with more dimensions, might improve the overall performance, reducing the dimension of the input data and the dimension of the inner layers of the neural network may reduce the size of the model [Bur19]. For example, instead of building a 2D CNN with 2D input data, a 1D convolutional network might be constructed by modifying the input data and layers.
- **Quantization:** Often NNs are built using float-point numbers, which can take up a lot of memory and processing power. Quantization is a technique that reduces the number of bits used to represent each weight, leading to a lighter model that requires less memory and processing power [Agg18].
- **Model Architecture:** While designing a deep learning model, it is important to keep in mind that fewer layers and neurons will always be lighter than a more complex model with many layers and neurons. Often, a more complex NN may not perform better than a lighter one [GBC16]. The complexity of the model needs to reflect the complexity of the problem and the available computer resources. For example, for a given problem, a neural network with 10 layers may be sufficient. This means that a more complex NN with additional layers would not give any better results.
- **Model Distillation:** Distillation is a technique to transfer knowledge from a larger network into a smaller one [HVD15]. A pre-trained larger NN is used as the teacher, and a smaller network as the student. The student is trained to learn the outputs of the layers of the teacher network, rather than the training data itself. By learning the outputs of the larger network, it is effectively compressing the knowledge into a more lightweight model.

2.6.2 Other techniques

Here, instead of having the goal of reducing the neural network, we discuss a few different ways that a deep learning model can be trained, especially when a larger pre-trained model is available.

- **Training from Scratch:** Training a model from scratch involves using a large dataset from the beginning, randomly initiated weights, and no sort of pre-trained information.
- **Transfer Learning:** Transfer learning is a technique where a pre-trained neural network, on a large dataset, is reused as a starting point to solve a similar but different problem [Bur19]. To achieve that, often the first layers of the NN are frozen (they cannot be trained), while the last layers learn new features of the new similar problem. For example, a neural network used for classification in images could be used as a starting point for a problem involving classification in videos.
- **Fine Tuning:** It is considered a type of transfer learning, but the whole network is trained when fine-tuning a model. It is often used by altering some hyper-parameters of the original network such as the learning rate, epochs, or batch size. By decreasing the learning rate and using a small dataset, the fine-tuning process will not rewrite the previously learned features of the original NN as only small steps are taken [Zha+21a]. For example, getting a trained neural network for general image classification and fine-tuning it to identify different dog breeds.

Up to this point, we have discussed the basic concepts of automotive radar, the most used analytical techniques for detection and estimation, and the most important ideas behind machine learning. However, we need to be able to obtain results and compare them in order to build a strong research foundation. For that, in the following section, we depart from deep learning and start discussing the most used metrics for this thesis.

2.7 Metrics

In scientific research, having reliable metrics is crucial to evaluate the quality of a model or an algorithm. Metrics provide a quantitative measure of how well a system is performing, and we can use them to compare different models or algorithms. In signal processing and machine learning, various metrics are used to evaluate the performance of various systems [Ric14; GBC16].

Choosing the right metric for a specific task is essential as different metrics may be more appropriate for different tasks, and no single metric might be sufficient to evaluate the performance of a system fully. In practice, it is often necessary to use multiple metrics and to carefully analyze the results to draw meaningful conclusions.

In this section, we present the concept of a binary classification, signal-to-noise ratio (SNR) as a way to understand the ratio of the signal power to the noise power, the root mean square error (RMSE) as a method to measure the difference between the predicted and actual values of a variable, and general concepts of true positives or false positives.

2.7.1 Binary Classification Metrics

A binary classifier is a model that can only classify the data between two possible groups: True or False. The classification can be correct or incorrect. When a true prediction is correct, it is called a True Positive (TP). When a true prediction is incorrect, it is called a False Positive (FP). When a false prediction is correct, we call it a True Negative (TN) and when it is wrong, we call it False Negative (FN) [Eng19; Ras14].

However, the strength of the binary classification metrics comes from the fact that we can make any sort of multiclass problem or even non-discrete problem into a binary problem by establishing a clear boundary between the classes. For example, we could establish that any number above 10 is a true positive and any number below 5 is a true negative. Any prediction that does not fit the pre-defined boundaries is a false prediction.

In [Figure 2.25](#), we can show a confusion matrix of a binary classifier, with all four possible outcomes: TP, FP, TN, and FN.

Based on the concept of binary classifiers, we can build a few metrics to

		Predicted	
		True	False
Actual	True	True Positive (TP)	False Negative (FN)
	False	False Positive (FP)	True Negative (TN)

Figure 2.25: Binary Classifier

analyze the performance of such classifiers [Eng19; Ras14]:

$$\left\{ \begin{array}{lcl} \text{False Positive Rate} & = & \frac{FP}{FP+TP} \\ \text{False Negative Rate} & = & \frac{FN}{FN+TP} \\ \text{Sensitivity} & = & \frac{TP}{TP+FN} \\ \text{Specificity} & = & \frac{TN}{TN+FP} \\ \text{Accuracy} & = & \frac{TP+TN}{TP+TN+FP+FN} \end{array} \right. \quad (2.60)$$

In the following chapters, some variations of the metrics shown in Equation 2.60 are used to evaluate our proposed techniques.

2.7.2 Signal-to-Noise Ratio

Another fundamental metric in the signal processing field is the Signal-to-Noise Ratio (SNR) as it assesses the quality and the amount of noise of signals in various domains. SNR quantifies the ratio between the desired signal and the background noise or interference present in the environment [Sij+96; WSJ08; KBN14]. A high SNR indicates a lower amount of noise. The SNR formula can be given as:

$$SNR = 10\log_{10}(P_{\text{signal}}/P_n), \quad (2.61)$$

where P_{signal} is the power of the signal and P_n is the power of the noise.

We show an example of SNR in the Figure 2.26. The figure shows how the signal disappears into the noise as the SNR decreases.

On the other hand, Peak Signal-to-Noise Ratio (PSNR) tends to be more used when assessing the quality of images or image-like signals. For this, instead of analyzing the power of the signal, PSNR uses the peak value of the given signal or image [KBN14]. The PSNR also has a direct relationship with the

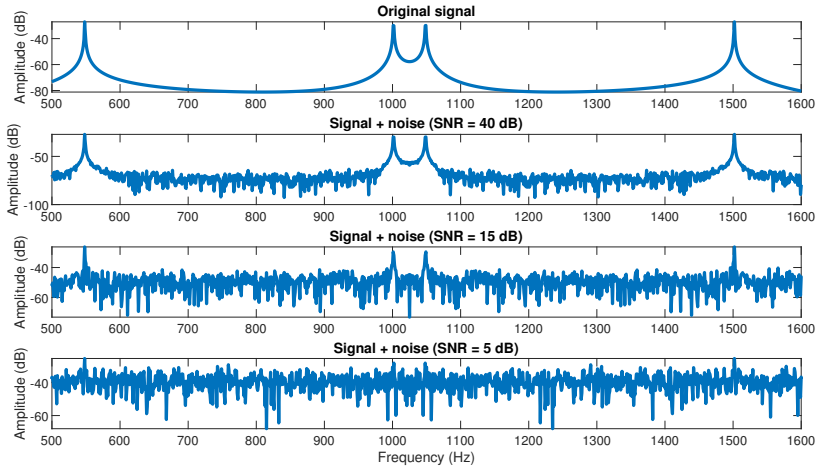


Figure 2.26: An example of SNR. Here, we can show the original signal and different levels of noise that were added to this signal.

Mean Square Error (MSE), which was previously described in the [Equation 2.43](#). MSE may be used as a loss function and as an evaluation metric [\[WSJ08\]](#). The PSNR equation is given as:

$$\text{PSNR} = 10\log_{10}(P_{\max}^2/\text{MSE}), \quad (2.62)$$

where $P_{\max}$ is the maximum possible value (or pixel value) in the image, and MSE is the mean square error between the original image and the reconstructed image.

Both SNR and PSNR measure the relative strength of the original information compared to the noise, making them conceptually similar despite being used in different domains. The key difference lies in how noise is defined: in SNR, it is any unwanted variation in the signal, while in PSNR, it is typically measured as pixel-



Figure 2.27: An example of PSNR. Here, we can show the original image and different levels of noise that were added to this image.

wise differences linked to Mean Squared Error (MSE). Despite this distinction, both metrics serve the same fundamental purpose of assessing how noise degrades information.

An example of various PSNR levels produced by adding Gaussian noise into an image is shown in [Figure 2.27](#). As the noise levels increase, the image disappears.

While PSNR is more important in [Chapter 3](#) and [Chapter 4](#), we utilize SNR for [Chapter 5](#), [Chapter 6](#) and [Chapter 7](#)

2.7.3 Root Mean Square Error

Another common metric for evaluating performance throughout various fields is the Root Mean Square Error (RMSE) [[CD14](#)]. This metric is an expansion of the MSE metric. Considering an error between two variables $|\tilde{y} - y|$, the MSE is the mean of this error squared: $MSE = 1/K \sum_k^K |\tilde{y}_k - y_k|^2$. The main problem with mean square error is that it penalizes large errors as they are squared. To attenuate this problem, we can add a square root to the MSE as it gives us the root mean square error:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{K} \sum_{k=1}^K |y_k - \tilde{y}_k|^2}, \quad (2.63)$$

where K is the number of samples, $\tilde{y}_k$ is the predicted value, and y_k is the true value.

If we add L Monte-Carlos trials so we get to a more precise value, we get to the following RMSE equation:

$$RMSE = \frac{1}{L} \sum_{l=1}^L \sqrt{\frac{1}{K} \sum_{k=1}^K |y_k - \tilde{y}_{k,l}|^2}, \quad (2.64)$$

where K is the number of samples, L is the number of Monte-Carlos trials, $\tilde{y}_{k,l}$ is the predicted value, and y_k is the true value, given a certain sample k and Monte-Carlo trial l .

RMSE is an important metric for DoA estimation and it is used in [Chapter 5](#), [Chapter 6](#) and [Chapter 7](#).

Here, we conclude the discussion of some key concepts used in this thesis, transitioning into more specific topics.

Convolutional Autoencoders for Reducing Noise in RD Maps

ABSTRACT

This chapter discusses the usage of deep Convolutional AutoEncoder (CAE) to remove noise from Range-Doppler (RD) maps of an FMCW radar in situations where there are moving objects such as pedestrians and cyclists. Traditional noise reduction methods like Constant False-Alarm Rate (CFAR) with peak detection algorithm do not work well in highly noisy environments, especially for low-reflectivity objects like pedestrians. We propose using CAEs to overcome these limitations since they can learn to identify patterns and recognize moving objects like humans and bicycles, while traditional methods are not context-aware. The results show that CAEs may improve object detection when compared to CFAR.

In this chapter, we present the use of Convolutional AutoEncoder (CAE) to denoise Range-Doppler (RD) maps containing moving objects, which may help object detection in FMCW radars. The structure of this chapter is as follows. In the introduction, we discuss the limitations of Constant False Alarm Rate (CFAR) based methods for object detection. Sequentially, we present the motivation for using Convolutional AutoEncoders (CAE) to improve object detection in range-Doppler maps. We propose four different systems using CAEs to reduce the noise levels, trained and evaluated with either synthetic or measured data. Furthermore, we present our results, evaluating the performance of the proposed

systems using sensitivity and specificity. Finally, we discuss some limitations and we conclude by giving suggestions for future work.

3.1 Introduction

In this section, we discuss the differences between Constant False-Alarm Rate (CFAR) and Convolutional AutoEncoder (CAE) based denoising and detection in range-Doppler maps using FMCW radars.

As previously stated in [Chapter 2](#), Frequency Modulated Continuous Wave (FMCW) radars provide relative distance and radial velocity measurements of reflected targets and clutter. This information can be used to create a Range-Doppler (RD) map, which is a two-dimensional heat map that shows the distance between the radar and the object on one axis, and their relative radial speed on the other.

CFAR is a class of algorithms that are widely used to assess the noise floor and detect potential objects in Range-Doppler (RD) maps when combined with a peak detection algorithm.

Although we can adjust CFAR-based detection algorithms to various conditions, they might not perform well when noise or clutter levels are high. Additionally, they may require frequent recalibration [[Ric05](#)].

Even though RD maps are often expressed as the normalized received power (in decibels) of reflected objects at specific velocities and ranges, they can be treated as images, allowing various image processing techniques, such as CAE, to be applied to it. A deep convolutional autoencoder is a Convolutional Neural Network (CNN) that can reconstruct images, and its applications can range from pose classification [[SOG18](#)] to noise filtering [[Mor+18](#)] and even multi-spectral image fusion [[AMK19](#)].

Objects within RD maps appear as a consistent clustering of reflected targets with identifiable patterns over time. Context-aware techniques, such as deep convolutional autoencoders, might take advantage of the recurrent patterns of moving objects, such as pedestrians, bicycles, and cars. This means that even in highly noisy situations, it can use its context awareness to differentiate between noise (or clutter) and moving objects, making it possibly more robust than CFAR.

As a related work, Seyfioglu et al. [[SOG18](#)] used CAE to classify 12 different human activities - such as walking, jogging, running, crawling, and sitting - by analyzing micro-Doppler spectrograms obtained from radars. Their study showed that the accuracy of activity classification using CAE was 94.2%, which was 17.3% higher than the accuracy achieved with Support Vector Machines.

In a later study, Fuchs et. al., [[Fuc+21](#)] suggested a way to utilize complex-valued CNNs (CVCNNs) for radar interference reduction. After mitigating the

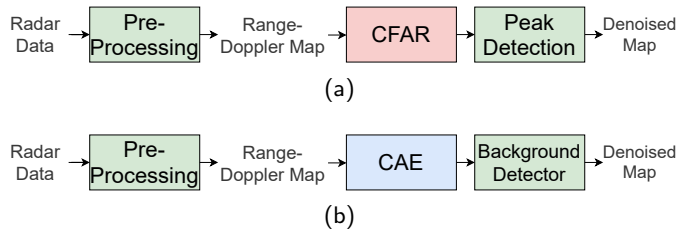


Figure 3.1: A comparison between two pipelines for object detection in range-Doppler maps: (a) the classical processing, and (b) our proposed processing with CAE.

interference the radar signal, they claim that Range-Doppler maps were restored. According to them, by using complex-valued analysis, an inductive bias is introduced, which limits the degrees of freedom in the network. However, while this restriction may seem disadvantageous, it can greatly improve the learning process by enforcing transformations on the signal that conforms to the physical properties of the data and simplifying the problem at hand.

The pipeline showed in (a) of Figure 3.1 is commonly used for object detection in FMCW radars [Tex18; Ed+13], while (b) of Figure 3.1 shows our proposed modifications by incorporating CAE and the foreground detection algorithm.

3.2 Architectures and Configurations

In this section, we discuss the deep learning architectures and models utilized for denoising range-Doppler maps and detecting moving objects. We start by discussing autoencoders and their deep learning equivalent. Then, we show our four proposed systems to denoise RD maps. Furthermore, we discuss the usage of the foreground detection algorithm. Finally, we explain the CFAR-based detection algorithm used to obtain the baseline comparison for our models.

Autoencoders are typically an unsupervised type of algorithm that aims to reduce the dimensions of a higher-dimensional input map to a lower-dimensional one, such that the original input can be reconstructed (with some approximation) from the lower-dimensional representation [TBL18]. In other words, it tries to recreate the input by reducing its dimensions through feature extraction.

A deep Convolutional AutoEncoder (CAE) is a type of autoencoder that uses Convolutional Neural Networks (CNNs) to extract features from matrix-like data, such as images or RD maps.

After experimenting with different parameters such as the number of layers and filter size (further explained in Section 3.4), we selected the CAE architecture shown in Figure 3.2.

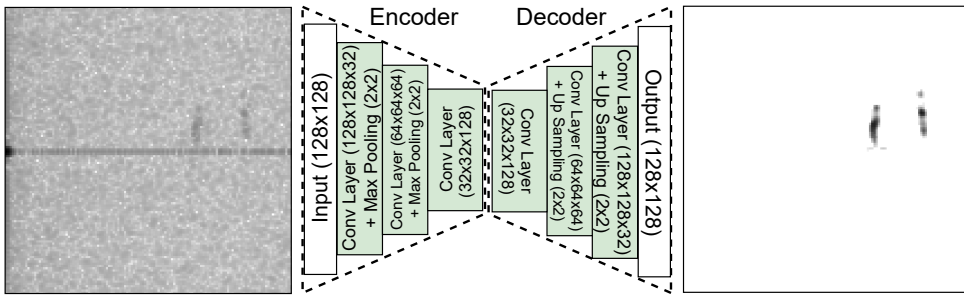


Figure 3.2: A noisy range-Doppler map passing through a 3-layer encoder and 3-layer decoder to obtain a cleaner image.

The input of our CAE architecture is black-and-white images of the RD maps that can be more easily processed using deep learning. For this, we normalized the range-Doppler maps expressed in decibels to an 8-bit grayscale image scale [Bas19].

During the training of our convolutional autoencoder, a noisy input and its corresponding denoised output were provided. As the loss function, we used Mean Squared Error (MSE). As the optimizer algorithm, we used Root Mean Squared Propagation (RMSProp), which aims to minimize the loss function to adjust the weights and biases of the neural network.

In Figure 3.2, we also show a comparison between a noisy and a denoised RD map. Firstly, a noisy range-Doppler map that contains the information of two people walking toward the radar is provided as input to the encoder. The encoder is designed to extract relevant features and shapes from the input. The extracted features are then passed through the decoder, which reconstructs a denoised map in the output, based on the training process.

One advantage of deep NN over analytical methods is their ability to learn patterns. When a person walks, their movements create specific patterns in the RD map that can be identified by humans, such as a line-shaped object that



Figure 3.3: Six frames of a person walking on an RD map over time. The movement starts in frame (a) and ends in frame (f). These frames are not necessarily consecutive.

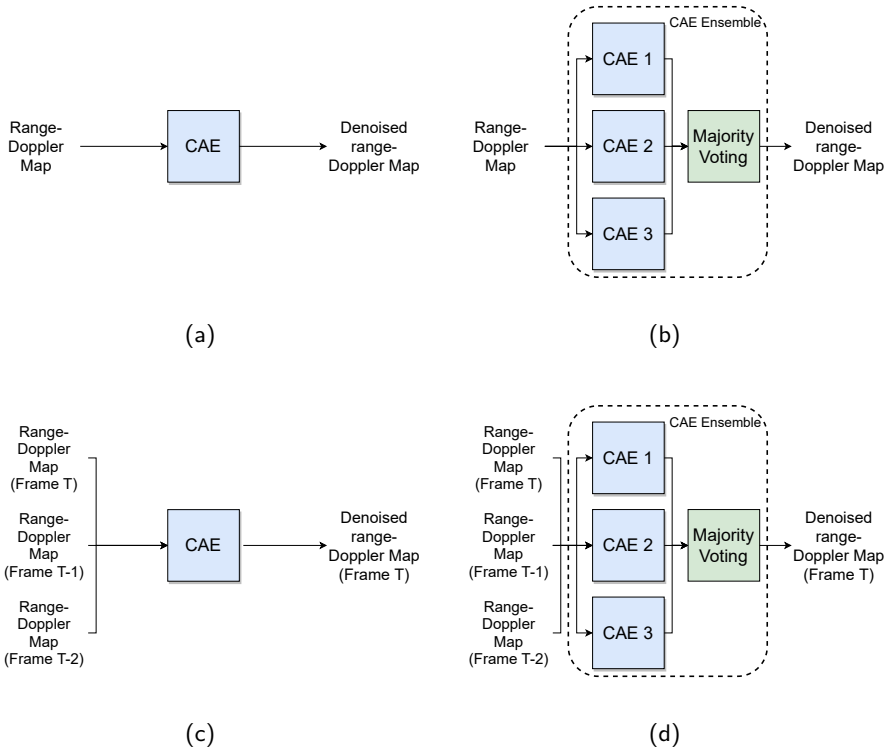


Figure 3.4: The four proposed models using deep learning. (a) Uses one CAE and processes one frame; (b) uses three CAEs and processes one frame; (c) uses one CAE and processes three frames; (d) uses three CAEs and processes three frames.

compresses and stretches¹ (see Figure 3.3), which a neural network can learn.

As weights and biases are usually randomized at the beginning of the training process of a neural network, the resulting trained models are most probably unique. As a result, we can utilize this in our favor by combining multiple models in parallel as it reduces the chances of getting stuck in a local minimum after training. According to Yu et al., [YHB03], combining multiple NN may achieve a higher pattern recognition since the sets of wrongly recognized patterns by different networks do not necessarily overlap. Through some empirical experimentation, we decided to use three parallel networks.

To improve the noise reduction performance of CAE in range-Doppler maps,

¹ <https://mllimadeoliveira.github.io/Thesis/CAE.html> Animations of a black-and-white range-Doppler map and its denoised version by one of our CAEs.

we have also experimented with architectures that incorporate time-series data. Since RD maps are captured over time, multiple time-sequential frames could feed a network to help differentiate a moving object from random Gaussian noise. Zhao et. al. [Zha+17] presented different methods of producing a time series with CNNs, emphasizing their importance in handling temporal data objects obtained through chronological observation. Through some empirical experimentation, we decided to use three sequential frames.

By incorporating the concepts of parallel networks and time series, we proposed and evaluated four systems based on the CAE architecture. All systems are shown in Figure 3.4. The first system is one CAE that processes one frame at a time. The second system uses three CAEs, in parallel, with different weights and biases, also processing one frame at a time. The third system is a time-series CAE that processes the current frame and two previous frames. The fourth system includes three time-series CAEs working in parallel, processing the same three sequential frames. To obtain a single output from the parallel networks, we applied a majority voting algorithm. Any object needs to be detected by at least two networks to be preserved, otherwise, it is considered a false positive and discarded.

While denoising is the most important component of our detection system, it is not the only one. The final step involves performing the detection itself on the denoised maps. To perform this, a simple detection algorithm was used, which applied a foreground detection algorithm based on a mixture model with three Gaussian modes. Foreground detection algorithms [Pic04; SG99], also known as background detection algorithms or background subtraction algorithms, are commonly used for detecting moving objects using stationary cameras. Since the map is completely clean of other objects, a simple foreground detection algorithm is enough. After detecting the object, we calculate the edge pixels and calculate the bounding box. The pipeline of our detection system is shown in Figure 3.5.

During the training of the systems, we observed that they performed better and converged faster when trained gradually. Since the training dataset was not very large, starting with easier scenarios and gradually increasing the difficulty helped the DL model stabilize and adapt more effectively. This approach

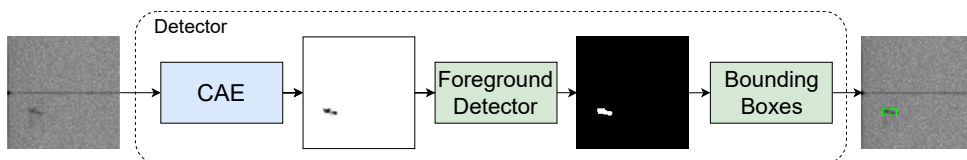


Figure 3.5: The pipeline of our detection system for range-Doppler maps.

promoted smoother weight convergence for the neural network. The training process consisted of first training the system with 1460 frames of synthetically generated data with a PSNR value of 25 dB, then 1610 frames with a PSNR of 10 dB, followed by 1430 frames of 5 dB, and finally 4550 frames ranging from 2 dB to 25 dB. For the experiments using measured data, we re-trained the neural networks with measured data after initial training with synthetic data.

Peak Signal-to-Noise Ratio (PSNR) is a common metric for measuring image quality. It is widely used due to its simplicity and ease of understanding. A low PSNR indicates high noise levels, while a high PSNR value indicates that the retrieved image is close to the original image.

The Ordered-Statistic Constant False Alarm Rate (OS-CFAR) algorithm was used as a baseline comparison to evaluate our algorithm. OS-CFAR is a variation of the CA-CFAR algorithm that sorts the samples in an ascending order to create a new sequence and derives the threshold from this sequence. More precisely (as previously discussed in Equation 2.3.2), the κ th strongest point of the ascending-order vector z is chosen to be the threshold $\mathcal{T}$ [Ric14]:

$$\mathcal{T} = \psi z_{\kappa}, \quad (3.1)$$

where ψ is a parameter that allows the adjustment for the threshold.

3.3 Dataset

In this section, we discuss the datasets used for training and evaluating our NN-based models.

Three measured datasets were created for training and evaluating our systems. The first database was created using an automotive FMCW radar and included

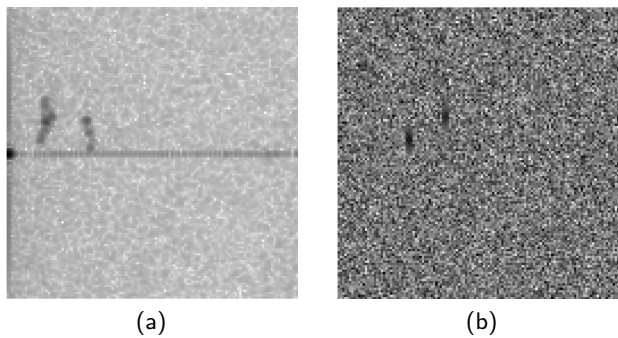


Figure 3.6: RD maps containing two moving objects: (a) measured data, and (b) algorithmically (synthetic) generated data.

data with moving bicycles and pedestrians. This dataset was further divided into two groups. One group was gathered with a still radar while the second group was gathered with a moving radar. The second dataset was created by augmenting the measured data with algorithmically generated noise. The third dataset was created using algorithmically generated synthetic data. This allowed better control of the amount of noise introduced in the maps.

Figure 3.6 shows two Range-Doppler (RD) maps depicting the movements of pedestrians. The image (a) was obtained from gathering data using an automotive FMCW radar, while the image (b) was generated algorithmically.

3.3.1 Measured Radar Data

To gather data, we have utilized the 77 GHz automotive FMCW radar transceiver from NXP to collect data of pedestrians and bicycles moving toward and away from the radar. The radar was configured for short-range, with a maximum velocity resolution of 24 km/h and a maximum distance of 12.8 meters. The raw radar data was processed using 128 chirps, each consisting of 512 samples, and several 2D FFTs to obtain 3782 range-Doppler maps of size 128×128 at a rate of 20 frames per second. Of these maps, 2007 were used for training, and 1775 were used for evaluation. The maps were then converted to black and white images (to be fed into our CAE systems) and all objects were labeled manually.

During our data collection process, we set a maximum of two moving objects in any given frame, in any arrangement. All objects moved either towards or away from the radar, which allowed us to measure the exact relative radial velocity between them. Figure 3.7 shows an example scenario of our data collection, where a cyclist is moving away from the radar and a pedestrian is moving towards it. A challenging point of using measured data is that people are not highly reflective and their reflected power can sometimes be below the noise level.

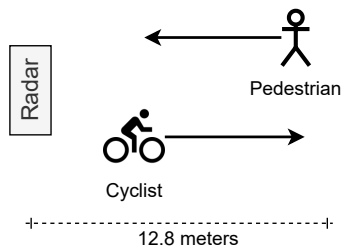


Figure 3.7: An example of a scenario with a cyclist and a pedestrian used during the data collection. Here, the pedestrian walks towards the radar and the cyclist away from the radar

We gathered two groups of measured data. In the first one, the radar was stationary. In the second one, the radar was moving at approximately 4 km/h. The movement of the radar adds clutter noise to the RD maps, as the moving ground can be mistakenly identified as a moving object.

3.3.2 Augmented Data

After collecting and labeling data, we created another dataset containing the same two groups of data by augmenting our existing measured dataset. For that, we digitally injected more Gaussian noise into the measured data. This reduced the Peak Signal-to-Noise Ratio (PSNR) of the data from an average of 9 dB to an average of 7 dB.

3.3.3 Synthetic Data

Throughout our time gathering data, we only collected 3782 range-Doppler maps. For this reason, an algorithm was used to produce synthetic image-like data similar to range-Doppler maps. In this algorithm, we could control noise levels and dataset size, obtaining automatic labeling. The aim was to generate images that resembled RD maps with up to six pedestrian-like objects moving at different speeds while also controlling the noise level. We created 18290 frames of 128 x 128 black-and-white images, mimicking RD maps with noise ranging from 2 to 25 dB. We used 9050 frames for training and 9240 for evaluation.

3.4 Experiments and Results

In this section, we discuss the results of three sets of experiments. Before introducing the experiments, we first discuss our metrics. In the first experiment, we evaluate different convolutional autoencoder architectures regarding their number of parameters and layers using synthetic data. In the second experiment, we evaluate the performance of our four CAE systems under various noise levels using synthetic data. In the third and last experiment, we evaluate the performance of the same systems using gathered radar data.

3.4.1 Metrics

For the following experiments, PSNR [Nas+14] was used as a metric to measure the amount of noise in each scenario. As previously discussed in [Chapter 2](#), Peak Signal-to-Noise Ratio (PSNR) and Signal-to-Noise Ratio (SNR) are both widely used to evaluate the quality of a signal or an image. However, they are typically applied at different stages of signal processing and differ in their purpose. SNR is generally used before detection, assessing the ratio of signal power to noise

		Predicted	
		True	False
Actual	True	 TP	 FN
	False	 FP	 TN

Figure 3.8: Binary classifier used to calculate sensitivity and specificity in our experiments.

power during signal acquisition or pre-processing. In contrast, PSNR is more often applied after detection, especially in image and video compression, where it evaluates the ratio between the maximum possible signal power and the noise affecting the quality of the processed signal. Expressed in decibels (dB), PSNR is often applied for evaluating the fidelity of a reconstructed signal.

As our goal is to evaluate the quality of denoising later in the signal processing pipeline, PSNR is a more appropriate metric than SNR.

For the image processing field, the PSNR can be calculated as:

$$\text{PSNR} = 10\log_{10}(\text{max}^2/\text{MSE}) , \quad (3.2)$$

where max is a constant value that refers to the maximum possible pixel value in the image, and MSE is the mean square error between the noisy image and the noise-free image.

Another metric that we have used to evaluate the denoised RD maps was measured by calculating the number of objects correctly detected in their original positions (true positive) and the number of frames without detected objects that were not present in the original map (true negatives). **Figure 3.8** contains an example of True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) [Eng19]. Here, we defined as a TP when an existing object is correctly detected, as FN when an existing object is not detected, as FP when an object that does not exist is detected, and as TN when the absence of an object is correctly detected. As previously discussed in **Chapter 2**, we can calculate the

sensitivity and specificity as:

$$\begin{aligned} \text{Sensitivity} &= \frac{\text{TP}}{\text{TP} + \text{FN}} \\ \text{Specificity} &= \frac{\text{TN}}{\text{TN} + \text{FP}} \end{aligned} \quad (3.3)$$

It is important to note that, since these metrics are binary, if any part of the object is within the labeled bounding boxes, we consider it a successful detection.

3.4.2 Evaluating Different CAE Architectures

In the first experiment, the aim was to determine the combination of parameters (such as the number of layers, filter size, and depth) that would create a cost-efficient neural network that would not waste computational resources. In this experiment, each newly trained architecture was evaluated using 1250 synthetic frames and a fixed amount of noise.

The results of the first experiment are presented in [Table 3.1](#). The table shows the number of convolution layers, the number of trainable parameters, sensitivity, and specificity of each architecture. Sensitivity represents the percentage of objects correctly detected by the model compared to the ground-truth map, while specificity represents the percentage of frames where the absence of objects was correctly identified.

We selected one architecture to be our base CAE for their subsequent experiments. This chosen CAE, marked in blue color in [Table 3.1](#), has six convolutional layers, a 3×3 convolutional filter size, and 632,710 trainable parameters. It was chosen because it was lighter than using eight or ten layers while still being more

Table 3.1: Experiment results obtained using different architectures.

Model	Number of Convolutional Layers	Filter Size	Number of Parameters	Sensitivity	Specificity
CAE	10	3x3	10,957,702	97.5%	82.2%
CAE	8	3x3	2,698,118	97.7%	86.9%
CAE	6	2x2	281,670	96.3%	10.8%
CAE	6	3x3	632,710	98.0%	84.8%
CAE	6	3x3	159,686	97.6%	74.8%
CAE	6	3x3	2,518,790	98.6%	81.5%
CAE	6	4x4	1,124,166	97.4%	89.3%
CAE	6	5x5	1,756,038	97.0%	82.4%
CAE	4	3x3	116,102	95.5%	80.2%

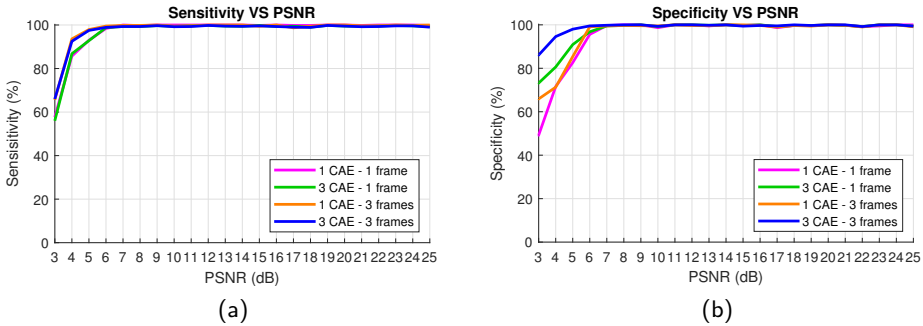


Figure 3.9: The performance of our proposed systems over various noise levels versus two different metrics: (a) Sensitivity (the percentage of correctly detected objects), (b) Specificity (the percentage of frames without a false positive).

robust than four layers, leading to a better cost-benefit ratio for this specific application. However, depending on the sort of application, another configuration could have been chosen.

3.4.3 Performance Over Various Noise Levels

After testing different CAE architectures, the second experiment was set to evaluate the performance of four CAE systems under various noise levels. The four systems evaluated were: (i) one CAE processing one frame at a time, (ii) three CAEs working in parallel, processing one frame at a time, (iii) one time-series CAE processing three frames, and (iv) three time-series CAEs working in parallel, processing three frames. Each system was trained with 9050 frames of the synthetic dataset and evaluated with 9240 frames. The PSNR varied from 3 to 25 dB.

Given the results presented in Figure 3.9 and Table 3.2, we may conclude that the systems that process three frames (time series) were slightly better than those that process only one frame in terms of the percentage of correctly

Table 3.2: Mean performance of distinct configurations over various noise levels and using synthetic data.

Model	Mean Sensitivity	Mean Specificity
1 CAE - 1 frame	93.9%	94.2%
3 CAE - 1 frame	93.6%	96.7%
1 CAE - 3 frame	95%	95.5%
3 CAE - 3 frame	94.6%	98.4%

Table 3.3: Performance over various configurations using measured and augmented data.

(a) Stationary radar with measured data.

Model	Sensitivity	Specificity
OS-CFAR	69.7%	90%
1 CAE - 1 frame	91.7%	87%
3 CAE - 1 frame	89.6%	90%
1 CAE - 3 frames	91.3%	86.3%
3 CAE - 3 frames	91.9%	88.5%

(b) Moving radar with measured data.

Model	Sensitivity	Specificity
OS-CFAR	54.7%	43.4%
1 CAE - 1 frame	87.6%	47.8%
3 CAE - 1 frame	83.4%	63.9%
1 CAE - 3 frames	87.1%	67.6%
3 CAE - 3 frames	84.8%	75.4%

(c) Stationary radar with augmented data.

Model	Sensitivity	Specificity
OS-CFAR	45.4%	60.9%
1 CAE - 1 frame	86.6%	68.9%
3 CAE - 1 frame	85.2%	87.1%
1 CAE - 3 frames	85.3%	63.6%
3 CAE - 3 frames	85.3%	88.6%

(d) Moving radar with augmented data.

Model	Sensitivity	Specificity
OS-CFAR	40.5%	21.8%
1 CAE - 1 frame	83.4%	37.4%
3 CAE - 1 frame	82.9%	64.1%
1 CAE - 3 frames	85.7%	29%
3 CAE - 3 frames	84.6%	64.6%

detected objects (sensitivity). However, when it comes to correctly identifying the absence of objects (specificity), the systems that use parallel networks were more reliable. Although all systems perform well, we believe that the most robust solution would be the one using three parallel CAEs and three sequential frames, provided that extra computational power is available. As shown in [Table 3.2](#), this system had the second-best correct detected rate (94.6% of objects) and almost always identified the absence of objects (98.4% of frames).

3.4.4 Performance Over Measured Measured Radar Data

In the third and last experiment, we evaluated the same systems, but using measured radar data instead of synthetic data. The data were divided into four groups: stationary radar with measured data, moving radar with measured data, stationary radar with augmented data (measured data with algorithmically elevated noise), and moving radar with augmented data. The models were trained with a dataset containing 2007 frames obtained with a stationary radar and 885 frames obtained with a moving radar. They were evaluated using 890 frames obtained with a stationary radar and 885 frames obtained with a moving radar.

From the results in [Table 3.3](#), it can be concluded that although OS-CFAR algorithm can work well in some situations, it is not robust enough to work well in adverse conditions where it is challenging to differentiate between an object and noise. In such situations, our models have shown greater robustness. After evaluating all four systems plus CFAR with all four groups of data, we conclude

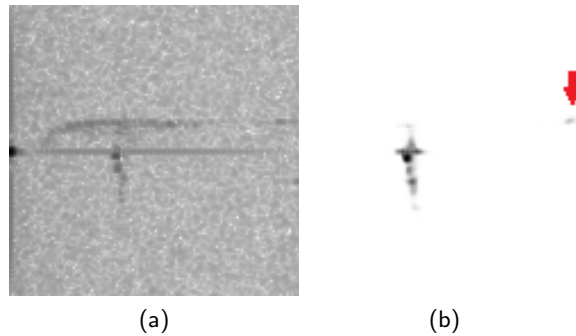


Figure 3.10: Range-Doppler maps of a moving object when the radar itself is moving: (a) the original black-and-white map (b) the denoised output from one of our deep learning systems. The red arrow shows a non-existing object.

that our preferred detection system would be the one that uses three parallel networks and three sequential frames as it makes fewer mistakes in general. As shown in Table 3.4, it has a good mean sensitivity of 86.7% and the highest mean specificity of 79.3%. In comparison, OS-CFAR algorithm has a mean sensitivity of 52.5%, and a mean specificity of 54.1%.

The difference in results between synthetic and measured data may be due to the more predictable patterns in synthetic data. Additionally, moving radars produce strong reflections from clutter, which do not follow a Gaussian distribution. As shown in Figure 3.10, it is harder to denoise a range-Doppler map when the speed of the moving radar is similar to the speed of the moving object. For example, Figure 3.10 (b) shows that even though the deep learning method learned the moving patterns of the radar, it can still introduce some “ghosts” in the reconstructed image, as pointed by the red arrow.

Table 3.4: Mean performance of various models using measured and augmented radar data.

Model	Mean Sensitivity	Mean Specificity
OS-CFAR	52.5%	54.1%
1 CAE - 1 frame	87.3%	60.3%
3 CAE - 1 frame	85.3%	76.3%
1 CAE - 3 frames	87.3%	61.7%
3 CAE - 3 frames	86.7%	79.3%

3.5 Conclusion

In this chapter, we evaluated the use of Convolutional AutoEncoders (CAE) to reduce noise levels in Range-Doppler (RD) maps. Various architectures and configurations of CAE were considered to determine the most viable system. Due to the content awareness of our proposed deep learning models, our results indicate that using our models was superior to using an analytical OS-CFAR-based method in terms of correctly detecting objects (sensitivity) and correctly identifying the absence of objects (specificity).

When using synthetic data, our system that uses three neural networks in parallel and processes three frames at a time seems to be the most robust in terms of sensitivity and specificity, being our preferred solution if sufficient computational power is available. When evaluating the detection systems with our measured and augmented data, our systems outperformed the CFAR-based detection algorithm, especially for scenarios with a higher amount of noise. Also for measured and augmented data, our system uses three networks and processes three frames is the most balanced model. For example, it has the best mean specificity (79.3%), while having a similar mean sensitivity (86.7%) to the other data-driven models (the best mean sensitivity is 87.3%).

Generating Synthetic RD Maps Using Deep Learning

ABSTRACT

In this chapter, we present the creation of synthetic Range-Doppler (RD) maps of moving objects of FMCW radars using Generative Adversarial Networks (GANs) and deep Convolutional AutoEncoders (CAEs). The synthetic data is based on measured RD maps of pedestrians and cyclists and is generated using a Neural Network (NN) model instead of mathematical functions or heavy radar simulations.

Using the synthetic data created with our neural networks has several advantages such as automatic labeling, easy creation of large datasets, and minimal computational power requirement after training. The quality of our synthetic data was evaluated by training a detection system using only synthetic data and evaluating it using measured data.

In this chapter, we present an approach for generating synthetic FMCW range-Doppler maps that mimic actual RD maps derived from processing measured radar data with pedestrians and bicycles. First, we introduce the topic by briefly describing our pipeline and presenting some related work. Subsequently, we discuss the data manipulation used within this chapter. Furthermore, we discuss the usage of GAN and CAE, and how they were applied in the scope of this chapter. Moreover, we present our evaluation metrics, some synthetic data generated by our system, and a comparison between a detector trained with our synthetic data versus a detector trained purely using measured data.

This chapter is based on the published work [LB20b].

4.1 Introduction

In this section, we discuss the different approaches to obtain synthetic data, GAN models, and how our approach generates new synthetic RD maps.

As previously discussed in [Chapter 3](#), Range-Doppler (RD) maps can be obtained by processing FMCW radar raw data through FFTs. If treated as images, it is possible to apply deep-learning-based image processing techniques to them.

While the image processing field has access to a large number of annotated datasets (that have been extensively used for deep learning training), this is not the case for radar processing. Annotating large radar datasets is a tedious, expensive, and time-consuming task. Therefore, synthetic data can be valuable as it can help researchers to save time, labor, and money.

As discussed in [Subsection 1.2.5](#), we categorize synthetic data generation into three main groups throughout this thesis: mathematical, simulation-based, and generative models. The first type, the mathematical approach [[Jor+22](#)], uses well-known physical equations to create data for specific situations. This approach is efficient but has limitations, especially when dealing with complex or non-linear systems. Clutter and external environment properties might be hard to incorporate in such a model. The second type, the simulation-based approach [[Dud+10](#); [Jor+22](#)], models real-world elements like wave reflections and signal paths in a virtual setting, allowing for detailed control over object and environment properties. This is often built using some sort of engine such as Unreal [[QY16](#)]. However, this approach requires a lot of computing power, which can make it prohibitive for generating large quantities of data. The third type, the generative model approach [[GBC16](#)], is the focus of this chapter. It uses deep learning algorithms to produce data. Once trained, this approach can be less computationally intensive than the simulation-based approach and may deliver more complexity than the mathematical approach.

Recent advancements in the image-processing field brought us new types of generative neural networks able to produce generative synthetic data, being the Generative Adversarial Network (GAN) [[Goo+14](#)] one of the first of its kind. GANs are known for their ability to create new data from noise and can be used for tasks such as image generation and anomaly detection in medical images [[Pan+19](#)].

In short, we present in this chapter a method to produce synthetic range-Doppler maps with moving targets. As shown in [Figure 4.1](#), the so-called Generator-Predictor uses Generative Adversarial Networks (GANs) for the generation and Deep Convolutional Autoencoders (CAEs) for future predictions. The generator creates the first frame, and the predictor produces the next four sequen-

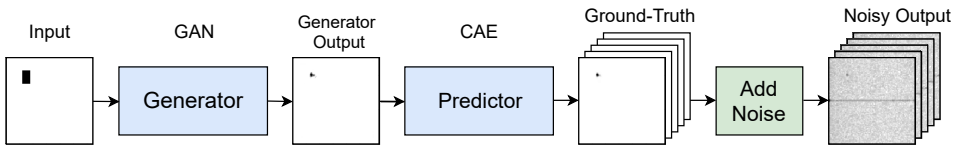


Figure 4.1: Pipeline of our Generator-Predictor used for generating synthetic range-Doppler maps.

tial frames. The generated RD maps are then augmented with noise, resulting in pairs of ground-truth and noisy maps. More precisely, our method uses a small labeled dataset to create synthetic datasets. The key advantage of our method is the ability to generate a large amount of annotated RD maps to complement (or substitute) small or incomplete datasets [Won+16].

The quality of our synthetic radar data was evaluated by training a detection system using an artificial neural network and a foreground detection algorithm, similar to the detection system we described in Chapter 3. During the evaluation, we trained a detection system using exclusively our generated data and tested it using unseen measured radar data. The results indicate that detectors trained solely with our synthetic data have a similar performance to those trained solely with measured FMCW radar data.

As a related work, Erol et. al. [EGA19] used GANs to generate signals of eight different human activities. Although they did not take time-series into account, their trained classifier achieved an accuracy of 82.56%.

In another study, Park et. al. [PLK22], inspired by other works including [LB20b], proposed a method for improving the performance of a range-Doppler map classifier through the usage of their Generative Adversarial Network (GAN)-based data augmentation model. The GAN was trained on a collection of RD maps from Unmanned Aerial Vehicles (UAVs) and was able to generate synthetic maps for a specific input class. They claim that their model was able to achieve a higher classification performance using synthetic data, particularly in situations where the training data was limited.

4.2 Dataset

In this section, we discuss the data acquisition and the processing steps we used to prepare the data before training our deep learning models.

The first step before producing synthetic data is gathering some FMCW radar data. For this, we used the same dataset as in Chapter 3 [LB20a]. In short, we have used a 77 GHz FMCW radar transceiver from NXP, configured to use 128 chirps, each comprising 512 samples, a bandwidth of 1.8 GHz, a maximum speed

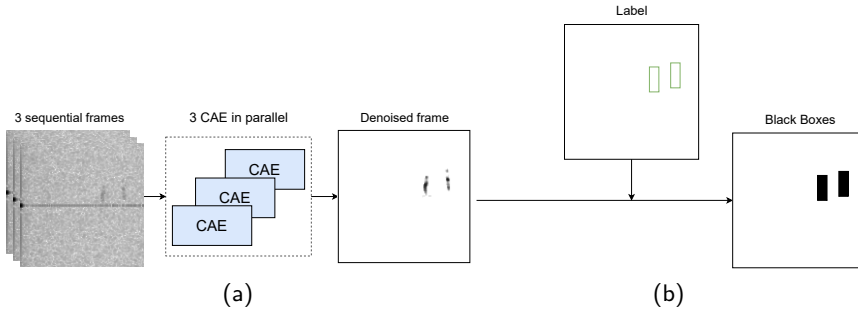


Figure 4.2: (a) Three sequential noisy range-Doppler maps passing through three Deep Convolutional Autoencoders (CAEs) to obtain one clean map. (b) The clean denoised map and the bounding boxes are added together so we can generate the black boxes used for the GAN training.

setting of 24 km/h, a maximum distance setting of 12.8 meters, and capturing rate of 20 frames per second. With this, we obtained 2845 range-Doppler maps of 128×128 bins that were converted to a black-and-white image, as previously discussed in [Subsection 2.1.2](#). As in [Chapter 3](#), we used a maximum of two moving objects per map. We used 1898 RD map for training, 547 maps for the evaluation, and 50 maps to produce the noise addition.

After processing the dataset, we passed it through the denoising neural network of [Chapter 3](#) to obtain the ground-truth maps. For this, we used the model that utilizes three parallel Convolutional Autoencoders (CAEs) to process three sequential frames [[LB20a](#)] and produce a single clean frame containing only the moving objects, as shown in [Figure 4.2](#).

In the sequence, all frames passed through a visual inspection to ensure that our neural networks effectively cleaned each frame. To ensure that the frames were completely clean, we utilized the bounding boxes of our labels to keep the relevant objects and deleted any false positives.

To expand our training dataset, we performed data augmentation by mirroring some frames, and by combining moving objects from different frames into a single frame. Augmenting our dataset gave us a total of 4008 frames for training.

4.3 Architecture and Configurations

In this section, we present and discuss the three parts that form our Generator-Predictor: generator, discriminator, and predictor. In short, the generator and predictor are training simultaneously in a competitive scheme. The predictor

uses autoencoders to predict future frames based on the first estimation from the generator. After training, the discriminator is no longer needed.

4.3.1 Generator and Discriminator: Generative Adversarial Networks

In recent years, there has been significant research focused on the application of generative networks [AMB21]. One of the pioneer architectures was the Generative Adversarial Networks (GANs) initially introduced by Goodfellow et. al. [Goo+14], who proposed a framework involving two competing networks. The first network, known as the discriminator, aims to distinguish between real and fake data. The second network, known as the generator, tries to trick the discriminator by attempting to generate results that look authentic. Although the initial iterations may seem like random noise, as the training progresses, the generated data tends to gradually approach the quality of the real data used for comparison. This concept is often linked to a zero-sum game in game theory [Zha+19]. In this MinMax game, the discriminator seeks to maximize its ability to correctly identify real and fake data, while the generator aims to minimize the detection probability of fake data. As previously discussed in Subsection 2.5.5, we can summarize GAN equations as:

$$\begin{aligned}\mathcal{L}(w_g) &= \min(-\log(\mathcal{D}(w_d; \mathcal{G}(w_g; n))) \\ \mathcal{L}(w_d) &= \min(y \log(\mathcal{D}(w_d; z)) + (1 - y) \log(1 - \mathcal{D}(w_d; z))) ,\end{aligned}\tag{4.1}$$

where the objective of the generator is to minimize the loss function $\mathcal{L}(w_g)$ that minimizes the detection probability, and the goal of the discriminator is to minimize the loss function $\mathcal{L}(w_d)$ that maximizes its detection rate. $\mathcal{G}(w_g; n)$ is the generator's data sample, w_g is the weights and biases for the generator, n is an input noise vector, $\mathcal{D}(w_d; z)$ is the likelihood that the input data z is real rather than fake, w_d is the weights and biases for the discriminator, $y = 1$ if the training sample is real, and $y = 0$ if the training sample is fake [Zha+21a].

Unlike many generative models that utilize pure noise as input [GBC16], we have opted to develop a GAN that would enable us to specify the starting position of the moving object by providing a black box representing the desired position. This black box was generated using the annotated bounding boxes from labeling, as shown in Figure 4.2. This approach is similar to other studies where boxes were utilized to indicate desired positions, such as in the generation of aerial images of cars [She+19].

We used a similar version of GAN proposed by Goodfellow et. al. back in 2014 [Goo+14] instead of a more advanced version of GAN, such as Conditional

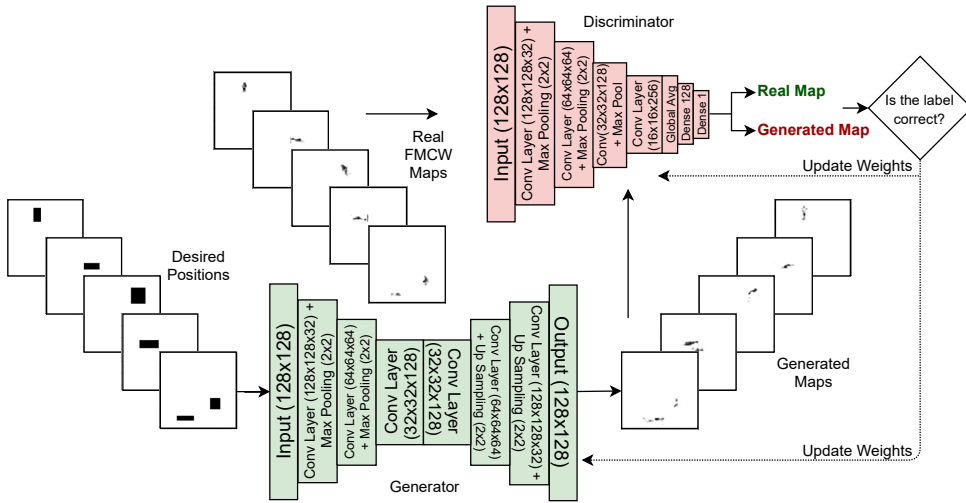


Figure 4.3: The training of our Generative Adversarial Networks (GANs) architectures. The generator receives images with black boxes as the input. The output of the generator is the generated synthetic RD maps. The generator tries to fool the discriminator. The discriminator receives as input both generated (fake) and measured (real) RD maps, as it tries to distinguish between them.

GAN [Wan+18] or Style-GAN [KLA21] that can generate realistic images¹, as our problem is simpler and mostly involves the creation of simple moving objects. However, it is well known that the most simple GAN architectures have convergence issues and vanishing gradient problems [Pan+19], but since our data was not complicated and the neural networks small, these were less problematic for us as we could find convergency after a few attempts.

For training the generator and discriminator, we used Binary Cross-Entropy (BCE) as the loss function, and Root Mean Square Propagation (RMSProp) as the optimizer, with a learning rate of 0.00025 for the discriminator and 0.0002 for the generator.

Our proposed generative adversarial networks' architecture is shown in Figure 4.3. It includes five input samples with specified positions for the moving objects represented as black boxes. Additionally, five denoised measured (real) radar range-Doppler maps are included to train the discriminator. The output of the generator has five generated (fake) RD maps that were produced in an attempt to deceive the discriminator after fifty thousand training iterations. Upon visual inspection, we observed that the generated (fake) maps resemble their

¹ <https://this-person-does-not-exist.com/en> is a website that generates realistic people that do not exist using Style-GAN.

measured (real) counterparts.

4.3.2 Predictor: Deep Convolutional Autoencoder

As previously stated in [Chapter 3](#), a Convolutional Autoencoder (CAE) is a type of Convolutional Neural Network (CNN) that functions as an autoencoder. It is specifically designed to reconstruct input data by extracting relevant features. The CAE architecture consists of an encoder and a decoder. The encoder is responsible for extracting the important features from the input, while the decoder is responsible for reconstructing the input from the extracted features. Through supervised learning, it can learn to distinguish between noise and moving objects in a range-Doppler map [[LB20a](#)].

While in the previous chapter convolutional autoencoders were used only for denoising, in this chapter we have employed CAEs for various purposes. Firstly, they were also used to denoise Range-Doppler (RD) maps to generate our ground-truth dataset. Secondly, CAEs were utilized as predictors within the Generator-Predictor. Lastly, they were integrated into the detection systems for evaluation purposes.

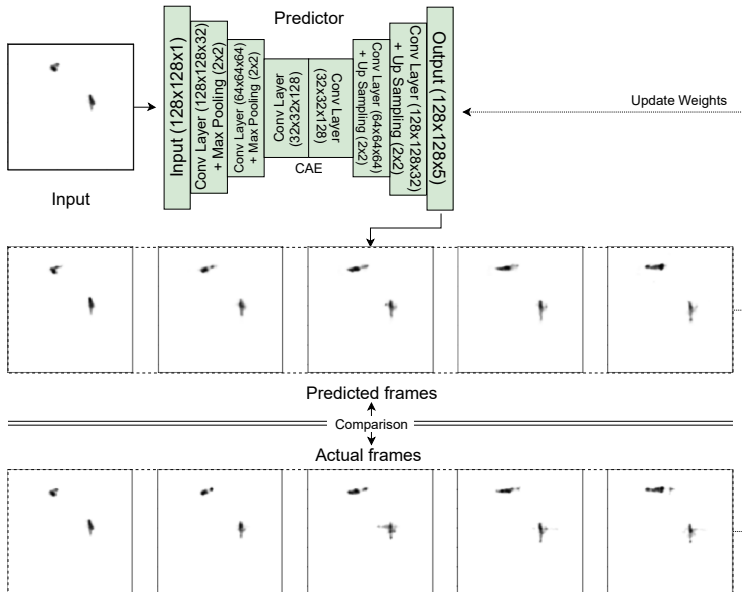


Figure 4.4: Training a Convolutional Autoencoder (CAE) architecture used as a predictor. During training, the predictor receives one frame of FMCW radar range-Doppler map and outputs five frames.

In the predictor architecture, a single RD map is provided as input, while the target consists of five sequential RD maps representing future frames. The decoder component of the CAE is responsible for reconstructing the current frame and predicting the subsequent four frames. For training, Mean Squared Error (MSE) was employed as the loss function, and RMSProp was used as the optimizer.

An example of the training of the predictor and its architecture is shown in [Figure 4.4](#). In the example, the encoder receives a single RD map containing two moving objects as input. The encoder then extracts essential features and shapes from the map. These features are subsequently analyzed by the decoder, which aims to reconstruct and predict the movement patterns of the two objects based on their training. A visual inspection showed that the prediction for the four future frames was similar enough to the expected outcome.

4.3.3 Generator-Predictor System

After training the generator and predictor separately, we combined them to create a unified system (as shown previously in [Figure 4.1](#)).

Our Generator-Predictor takes a white map with black boxes as the desired positions as input. The black boxes, which represent moving objects, are generated by an algorithm with varying positions and shapes. Our system then generates five synthetic frames capturing the movement of objects. Subsequently, noise is introduced to the generated frames. The noise is created by randomly selecting pixels from any of the 50 extra measured range-Doppler maps that do not contain moving objects. The resulting output of our system is unique for each run and closely resembles RD maps obtained from measured radar data processing. It is worth noting that we are interested in both the noise-free and the noisy data, as they can be used together to train other neural networks.

[Figure 4.5](#) shows six examples of the output generated by our Generator-Predictor. A visual inspection reveals that the four examples from the right-hand side show more satisfactory results compared to the two on the left-hand side. When combining two networks in sequence, the errors from both networks can accumulate and potentially introduce unknown disturbances in the output. Typically, the last predicted frame tends to contain more errors. However, it is noteworthy that the network has learned how to capture the compressing-stretching effect resulting from the varying speed between the torso and legs of a walking pedestrian.

4.4 Evaluation

In this section, we discuss the results of our experiment and our used metrics.

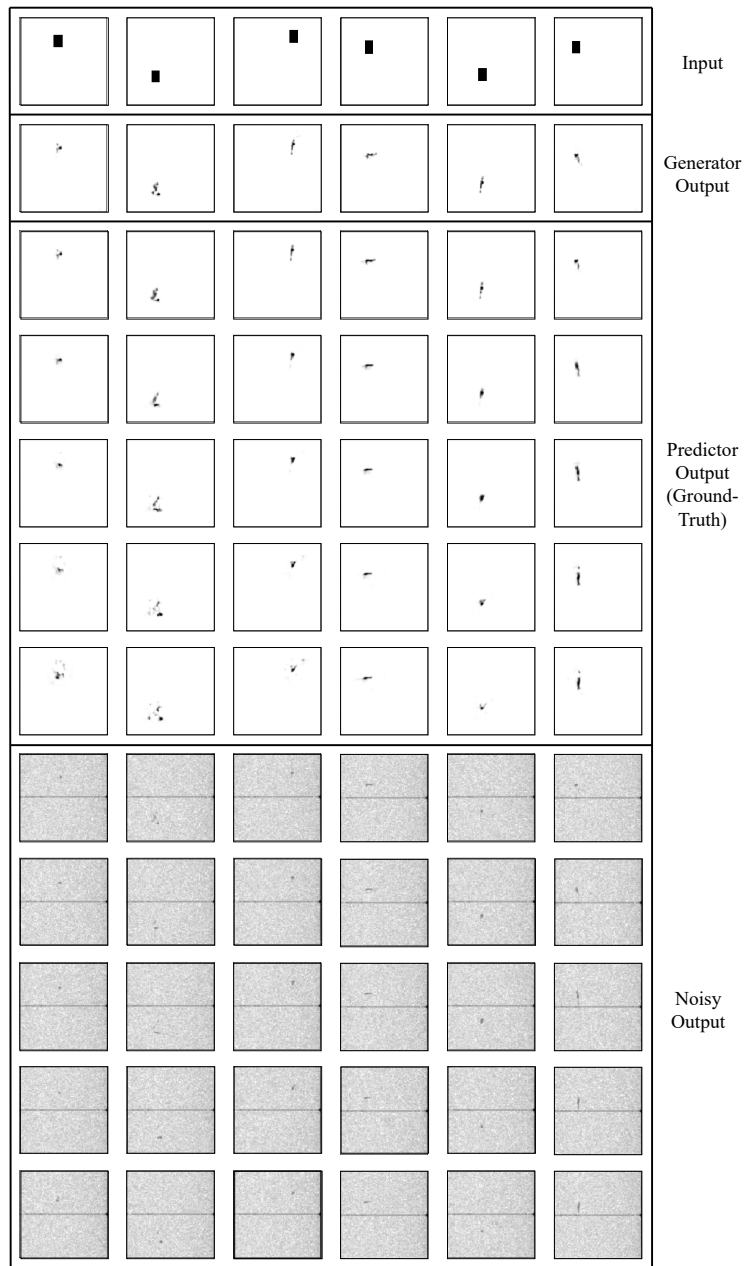


Figure 4.5: Six results from our Generator-Predictor. The inputs are the black boxes. The generator outputs a single generated frame. The predictor outputs the current frame plus four predicted future frames. Finally, radar noise is added to all five frames.

Chapter 4: Generating Synthetic RD Maps Using Deep Learning

In this experiment, we compared the performance of detectors trained using our synthetic and measured data. The detection systems follow the same architecture previously shown in Figure 3.5, using a denoising convolutional auto-encoder and a foreground detection algorithm. Similarly to Chapter 3, we measured our performance using sensitivity (percentage of correct detection).

For evaluation, we generated 8000 groups of five time-sequential frames of synthetic RD-like maps with noise using our Generator-Predictor. Additionally, as a comparison, we used 547 groups of sequential manually labeled frames of FMCW radar data that were not used for training the Generator-Predictor.

We evaluated the performance of a detection system trained solely with our synthetic data, having no direct exposure to measured radar data. At the output of the detector, bounding boxes are generated. When trained exclusively with synthetic data and evaluated with measured data, the detection system achieved a sensitivity of 95.2%.

To check if there would be an improvement in the performance if measured data was added to the training of the detection system, we also fine-tuned it with 1891 measured RD maps and achieved a sensitivity of 96.5%.

To provide a comprehensive performance comparison, we trained alternative detection systems using various dataset sizes of solely measured data, ranging from 10 to 4008 groups of sequential frames, as shown in Table 4.1.

The detection system trained only with our synthetic data showed superior performance (sensitivity of 95.2%) over some detection systems trained with small measured RD maps, specifically those with up to 1300 groups of sequential frames (sensitivity of 95.1%). Even when compared to a system trained on all 1898 frames (sensitivity of 96.4%), the one trained on synthetic data performed

Table 4.1: Results using different measured radar dataset sizes, in comparison with our synthetic data.

Measured Training Dataset Size	Sensitivity	Measured Training Dataset Size	Sensitivity
10	86.0%	500	95.0%
20	87.5%	800	94.5%
40	88.4%	1000	94.0%
80	85.3%	1300	95.1%
100	92.1%	1600	96.3%
150	92.4%	1898	96.4%
200	91.6%	4008*	96.5%
250	91.3%	0**	95.2%
300	91.5%	1898**	96.5%

*Augmented Data **Synthetic Data

at a similar level. Additionally, our results show that using synthetic data and fine-tuning it with measured data achieves performance comparable to that of systems trained with augmented datasets (96.5% for both scenarios).

The results show the potential benefits of using our Generator-Predictor to generate synthetic data. Particularly when working with limited datasets, synthetic data can be advantageous. The detection system trained with our synthetic dataset demonstrated good generalization when evaluated with measured data, showing the effectiveness of our synthetic data for training other neural networks.

It is important to note that our synthetic data also has certain limitations. The integration of two neural networks in our approach led to some differences in the generated moving objects when compared to their measured counterparts. Additionally, the generated data is limited to the type of moving objects present in the training dataset. For example, since our system was trained on a dataset containing only pedestrians and bicycles, it could not be used to generate cars. Moreover, in our current architecture, it is not possible to select between pedestrians and bicycles, even though the shape of the bounding boxes is different, which provides some level of information about the desired object to the generator. Lastly, we did not evaluate whether the moving objects' predicted trajectory was realistic.

4.5 Conclusion

In this chapter, we have discussed an approach for generating synthetic FMCW radar Range-Doppler (RD) maps using a combination of Generative Adversarial Networks (GANs) and Convolutional Autoencoders (CAE). Our goal was to create a large dataset of synthetic RD maps that closely resemble the RD maps obtained from processing measured radar data. Through experimentation, visualization, and evaluation, we have shown the effectiveness and potential benefits of our approach. The benefits include rapidly building a large dataset, automatically having annotated data, avoiding losing too much real-world generalization, and being computationally efficient after training.

The results of our experiment with a detection system trained solely on our synthetic data and evaluating it using measured radar data showed that it achieved a detection sensitivity of 95.2%. Furthermore, we trained additional detection systems using varying amounts of measured data. The result of training a detection system using measured and augmented data was a sensitivity of 96.5%, which is only slightly better (1.3%) than training using solely our synthetic data, and the same of training with synthetic data and fine-tuning with measured data. This suggests that our synthetic data can generalize well and

Chapter 4: Generating Synthetic RD Maps Using Deep Learning

provide comparable performance to measured radar data, particularly in scenarios where the availability of datasets is limited.

ResNet for Single-Snapshot DOA Estimation

ABSTRACT

In this chapter, we discuss the usage of Residual Neural Networks (ResNets) for estimating the Direction Of Arrival (DoA) and the number of sources of MIMO radars that use a Minimum Redundancy antenna Array (MRA). Specifically, we focus on the scenario where only one snapshot is available. In such cases, most techniques tend to deliver a poor estimation, whereas the Maximum Likelihood Estimation (MLE) algorithm still provides decent performance but at a high computational cost.

On the other hand, our ResNet-based direction of arrival estimator appears to be a viable alternative solution for this problem, as it seems to outperform the MLE algorithm in some scenarios.

As previously shown in [Figure 1.10](#), after detecting the targets in Range-Doppler maps, we estimate their number and angles. Therefore, this chapter advances the FMCW radar pipeline by shifting from RD maps to the problem of estimating the direction of arrival.

In this chapter, we discuss an approach for single-snapshot DoA estimation and the estimation of the number of sources using a residual neural network. Our ResNet-based estimator is trained with covariance matrices as input, and it can estimate multiple sources.

Firstly, we define our scope in the introduction. In the sequence, we discuss the signal model, the maximum likelihood estimation algorithm, and a source

estimation algorithm. Furthermore, we detail our ResNet-based DoA estimator's architecture. Finally, we present our evaluation metrics and the results of three sets of experiments, comparing the performance of the MLE algorithm with our data-driven method.

5.1 Introduction

In this section, we discuss the scope of our DoA estimator, the usage of the maximum likelihood estimation algorithm for estimating DoAs, and the pipeline of our ResNet-based estimator.

The Direction Of Arrival (DoA) estimation has been widely studied and there are many proposed approaches to solve it [Van02; SS90]. In this chapter, we focus on two main problems: estimating multiple DoAs using a single snapshot, primarily using a Minimum Redundancy antenna Array (MRA), and estimating the number of sources¹. Since we mostly consider MRAs, we primarily compared our results with the results obtained by using a Maximum Likelihood Estimation (MLE) algorithm, as MUSIC and ESPRIT do not perform well for this particular scenario [VBC14]. For example, as previously shown in Figure 1.7, using MRAs with the MUSIC algorithm might result in large side lobes that may hide targets.

As previously discussed in Subsection 2.3.7, the maximum likelihood estimation [Van02; HY10; SS90] is a reliable algorithm used for estimating the DoA. The main idea behind this algorithm is that it exhaustively tries all possible combinations of angles on a search grid that would satisfy the provided signal model, ultimately selecting the combination that yields the minimum residual error for the received snapshot. Nevertheless, the MLE algorithm suffers from the fact that its computational complexity increases exponentially with the number of sources. Furthermore, according to Häcker and Yang [HY10], MLE is the best-performing algorithm for DoA estimation when only a single snapshot is available.

Recently, other researchers have shown applications of Neural Networks (NN) for DoA estimation. In the research of both Bialer et. al. [BGT19] and Fuchs et. al. [FWG19], deep Fully Connected Neural Networks (FCNN) were used for estimating DoAs using a single snapshot. Both works used a neural network with regression to solve the problem. Bialer et. al. [BGT19] used Root Mean Squared Error (RMSE) as their loss function and could solve the DoA problem up to four angles. They also claim to be able to estimate the number of sources. Fuchs et. al. [FWG19] used both the average and distance between two sources as their

¹ In the literature for the estimation of the direction of arrival, the words 'target' and 'source' might be used interchangeably, as for this kind of application, it does not matter if the object is transmitting or reflecting.

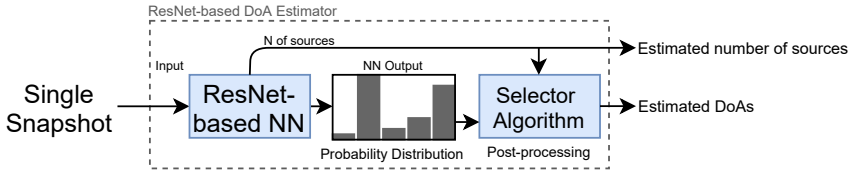


Figure 5.1: The pipeline of our ResNet-based DoA estimator that uses a ResNet-based neural network.

loss function and solved the DoA problem for two sources. Elbir [Elb20] also used regression and various Convolutional Neural Networks (CNN) in parallel to achieve a MUSIC-like spectrum and solve the DoA problem for up to six distinct angles.

Even more recently (after the publication of [LB22b] and [LB22a]), Youn et. al. [You+22] used unfolding neural networks to estimate DoAs. This approach considers the DoA estimation problem as an inversion problem, applying a variation of the LISTA algorithm to it, using unfolding deep neural networks (called CC-LISTA), to estimate the angles.

Unlike many previous approaches, our ResNet-based DoA estimator utilizes classification instead of regression to find the most probable DoAs. Furthermore, although the DoA estimation problem is connected to the problem of estimating the number of sources from snapshots, many researchers tend to consider that the number of sources is already known when estimating DoAs. In reality, this is rarely the case.

Figure 5.1 provides a pipeline for our ResNet-based DoA estimator. Our estimator is composed of two modules: a ResNet-based neural network and a selector post-processing algorithm. The outputs of our estimator are the estimated DoAs and number of sources. It is important to note that the number of sources can also be selected manually if desired.

5.2 Signal Model

In this section, we discuss the snapshot model and the maximum likelihood estimation algorithm.

As previously discussed in Section 2.2, for M antennas and N far-field sources,

where $0 \leq N \leq M - 1$, the received snapshot² $x \in \mathbb{C}^M$ can be represented as:

$$x = \mathbf{A}s + \eta, \quad (5.1)$$

where $s \in \mathbb{C}^N$ is the signal emitted by the sources, $\eta \in \mathbb{C}^M$ is the added Gaussian noise, and $\mathbf{A} \in \mathbb{C}^{M \times N}$ is the steering matrix. The matrix $\mathbf{A}$ contains the directions of arrival $\theta = \theta_1, \dots, \theta_N$, and it was previously shown in [Equation 2.14](#).

5.2.1 Maximum Likelihood Estimation

As previously discussed in [Subsection 2.3.7](#), Maximum Likelihood Estimation (MLE) is a commonly used algorithm for DoA estimation for a single-snapshot problem, having a good performance at it [[HY10](#)]. It works by calculating all possible angle combinations (within a search grid) to find the one that best minimizes the residual error.

As previously shown in [Equation 2.37](#), the residual error vector $\varepsilon \in \mathbb{C}^M$ is the difference between the received snapshot and its approximation. We can calculate the residual error as:

$$\varepsilon = x - \tilde{\mathbf{A}}\tilde{\mathbf{A}}^+x, \quad (5.2)$$

where $\tilde{\mathbf{A}}$ is a possible estimated steering matrix, and $\tilde{\mathbf{A}}^+$ is its pseudo-inverse. The Moore–Penrose pseudo-inverse of the matrix was previously given in [Equation 2.36](#).

After repeating [Equation 5.2](#) several times for various $\tilde{\mathbf{A}}$, we can find its minimal value as the minimal residual error: $\varepsilon_{\min} = \min\{\varepsilon\}$. In other words, the MLE algorithm minimizes the residual error ε by trying various matrices $\tilde{\mathbf{A}}$ with different DoA combinations. When the residual error is minimal, $\tilde{\mathbf{A}} = \mathbf{A}$. Therefore, as shown in [Equation 2.38](#), the estimated DoAs $\tilde{\theta} = \tilde{\theta}_1, \dots, \tilde{\theta}_N$ can be calculated using the minimal residual error:

$$\tilde{\theta} = \arg_{\theta} \{||\varepsilon_{\min}||^2\}. \quad (5.3)$$

As previously shown in [Equation 2.42](#), the number of times [Equation 5.2](#) needs to be repeated to find the minimal residual error that yields the direction of arrival estimation depends on the search grid ([Equation 2.41](#)) of the MLE algorithm and the number of sources. As previously discussed in [Subsection 2.3.7](#), the size of the search grid typically depends on the Field Of View (FOV) and angular resolution. FOV refers to the angular extent of the search grid. Angular

² In contrast to [Chapter 2](#), from this chapter onwards, we opted to simplify the notations. This means that variables such as θ and t are sometimes omitted. For example, instead of using $x(t) = \mathbf{A}(\theta)s(t) + \eta(t)$, we used $x = \mathbf{A}s + \eta$.

Algorithm 1 Estimating the number of sources using the MLE algorithm.

```

    ▷ Assumes the initial value for the estimation of the number of sources  $\tilde{N}$ .
    ▷ Find the residual error  $\varepsilon_{\min}$ , and the DoA  $\tilde{\theta}$  for a  $\tilde{N}$ -D MLE search.
    ▷ The snapshot  $x$  is the input for the MLE algorithm.
     $\tilde{N} = 1$ 
     $[\varepsilon_{1\min}; \tilde{\theta}_1] = \text{MLE}_{1\text{D}}(x)$ 
    if  $\|\eta - \varepsilon_{1\min}\|^2 > \mathcal{T}$ , then
         $\tilde{N} = 2$ 
         $[\varepsilon_{2\min}; \tilde{\theta}_1, \tilde{\theta}_2] = \text{MLE}_{2\text{D}}(x)$ 
        if  $\|\eta - \varepsilon_{2\min}\|^2 > \mathcal{T}$ , then
             $\tilde{N} = 3$ 
             $[\varepsilon_{3\min}; \tilde{\theta}_1, \tilde{\theta}_2, \tilde{\theta}_3] = \text{MLE}_{3\text{D}}(x)$ 
            if  $\|\eta - \varepsilon_{3\min}\|^2 > \mathcal{T}$ , then
                 $\tilde{N} > 3$ 
            end if
        end if
    end if
    end if
    
```

resolution is the smallest angular separation in the search grid where the MLE algorithm can still differentiate between two distinct targets.

In a scenario with a Field of View (FOV) of 180° , resolution of 1° , and 3 targets, the MLE algorithm would need to run Equation 5.2 nearly a million times. This means that this algorithm is computationally intense when estimating the DoA for multiple sources.

As explained in Algorithm 1, the MLE algorithm can also estimate the number of sources $\tilde{N}$. Initially, the algorithm assumes the presence of a single source, setting $\tilde{N} = 1$. Then, it employs a 1D MLE search to estimate the DoA that minimizes the residual error. If the difference between the minimum residual error and the known noise power is larger than a certain threshold, probably more than one source is present in the snapshot. Subsequently, the algorithm proceeds to assume the existence of two sources and it runs a 2D MLE search. If the minimum residual error is still larger than the noise power, it assumes that three sources exist. This iterative procedure continues for a 3D MLE search. However, the algorithm will not run further than a 3D MLE search. If more than 3 sources are present, the algorithm will report three sources and DoAs.

In other words, if $\|\eta - \varepsilon_{\min}\|^2 < \mathcal{T}$, then we can consider that $\eta \approx \varepsilon_{\min}$ and that we found the number of sources and their angles. The threshold $\mathcal{T}$ is typically set to the power of the noise in the snapshot but can be tuned based on experimentation with the necessary data.

5.3 Deep Learning Architecture and Data

In this section, we discuss the data manipulation and architecture of our ResNet-based neural network used to estimate the direction of arrival and the number of sources.

5.3.1 ResNet-Based Neural Network Data

ResNet was originally developed for image recognition [He+16]. Images can be represented as 3D matrices, with the third dimension for colors. We used a similar concept to process the input snapshot vector x .

Similar to Elbir [Elb20], we utilized the covariance matrix $\mathbf{R} \in \mathbb{C}^{M \times M}$ as the input for our neural network. However, as previously discussed in [Subsection 2.3.3](#), for real-world applications, obtaining the complete covariance matrix may not be possible. Therefore, we approximate the covariance matrix as:

$$\mathbf{R} = xx^H, \quad (5.4)$$

where x^H is the conjugate transpose (or Hermitian transpose) of the snapshot x .

As a form of regularization, we decided to normalize our neural network's input. Since our architecture contains real-valued weights, we need to process the complex covariance matrix $\mathbf{R} \in \mathbb{C}^{M \times M}$ as a real-valued input matrix $\mathbf{X} \in \mathbb{R}^{M \times M \times 2}$. For that, we separate the real and imaginary parts of $\mathbf{R}$, store them both separately in the third dimension of the 3D matrix $\mathbf{X}$, and normalize. The constant c_0 is used to normalize the input, keeping all values of $\mathbf{X}$ between -1 and 1.

$$\begin{aligned} \mathbf{X}_{i,j,1} &= \text{Re}\{\mathbf{R}_{i,j}\}/c_0 \\ \mathbf{X}_{i,j,2} &= \text{Im}\{\mathbf{R}_{i,j}\}/c_0 \end{aligned} \quad (5.5)$$

where $i = 1, \dots, M$ and $j = 1, \dots, M$.

The input layer of our neural network depends on the number of antennas. For the MRA experiments, we have used an input with size $8 \times 8 \times 2$. For the ULA experiment, the input size was $16 \times 16 \times 2$.

The output layer of our neural network consists of a fully connected layer, where each neuron corresponds to a specific angle bin. In other words, each neuron represents one possible angle in a grid. The value of each neuron represents the probability of a source being present at the corresponding angle. It is important to note that this probability measures how strongly the NN “believes” in the presence or absence of a target at a specific angle bin rather than being a mathematical representation of the actual probability of the problem.

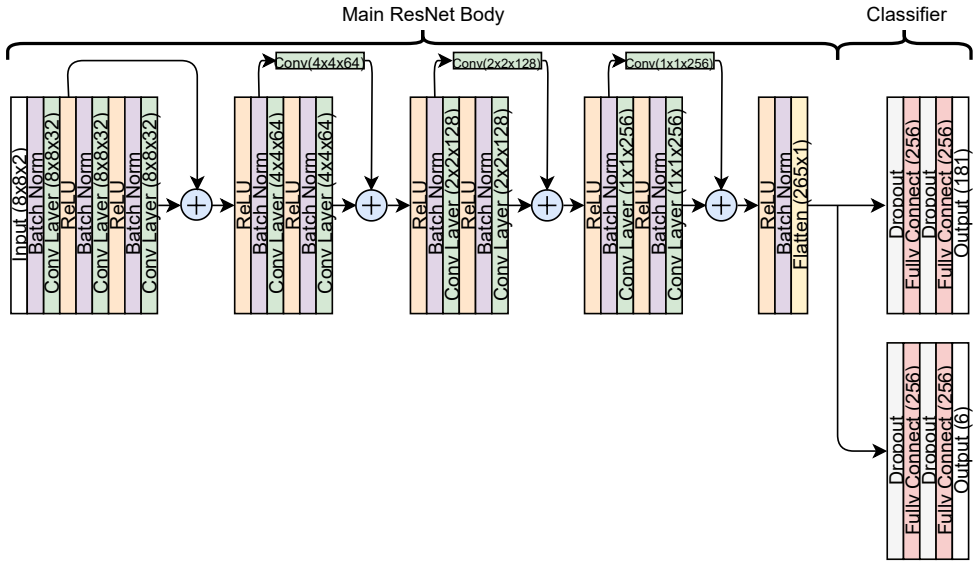


Figure 5.2: Our ResNet-based neural network architecture for estimating DoAs and the number of sources. In this architecture, we have two distinct outputs (classifiers): one with 181 neurons for the probability distribution, and another with 6 neurons to indicate the estimation of the number of sources. Both outputs share the same main body.

Unlike some other works [BGT19; FWG19], our NN does not perform a regression but a classification. With this approach, we do not need to modify our architecture when dealing with different numbers of sources N . On the other hand, post-processing is required to analyze the probability distribution from the network. Additionally, retraining is required when there are changes in the desired search grid. Despite these limitations, we observed better performance using classification-based NNs when compared to the regression-based ones.

The number of neurons in the output layer of the network depends on the desired grid, whereas the grid size depends on the FOV and the angular resolution. Consider that the output of this NN is a vector with evenly spaced elements. As previously shown in Equation 2.41, the size of the grid has $G = (\text{FOV}/\text{resolution}) + 1$ elements (neurons).

As we wanted to test different scenarios for different antenna arrays, we used different FOV and resolution for the MRA and the ULA experiments. For our MRA experiments, we used an FOV of 180° and a resolution of 1° , resulting in 181 neurons in the output layer. For the ULA experiment, we used an FOV of 20° and a resolution of 0.1° , resulting in 181 neurons in the output layer.

5.3.2 ResNet-based Neural Network Architecture

Our ResNet-based neural network architecture is a smaller and modified version of the original ResNet architecture proposed by He et al. [He+16]. As shown in Figure 5.2, our architecture includes 1 identity block, 3 convolution blocks, and 2 fully connected layers. Identity and convolution blocks were previously discussed in Subsection 2.5.4). We have found this lighter architecture to have a lower error rate when estimating DoAs when compared to heavier Fully Connected Neural Networks (FCNNs) and Convolutional Neural Networks (CNNs).

The main body of our ResNet-based NN contains convolutional layers (within the identity and convolutional blocks) that are responsible for the feature extraction. The classifiers consist of fully connected layers, which interpret the extracted features and classify them into different classes (or angle bins). The output of the main classifier of our ResNet-based NN architecture is the probability distribution v , where each output neuron has a value between 0 and 1. A value closer to 1 indicates a higher probability of a target being present at the corresponding angle bin.

The use of a classification instead of a regression allowed the neural network to have more freedom when estimating the direction of arrival. In addition, using softmax (as previously discussed in Equation 2.4.3) as the activation function helped to enforce sparsity in the probability distribution v .

5.3.3 Estimating Number of Sources

Besides estimating the direction of arrival, our architecture can also estimate the number of sources present in the received snapshot, as seen in Figure 5.2. Instead of employing a separate neural network or a different solution, we introduced a new classifier with additional fully connected layers to estimate the number of sources, using the same features extracted by the main body of the ResNet-based NN for both classifiers. We call this secondary path as the number of sources estimator. This classifier was not trained simultaneously with the direction of arrival estimator classifier.

The number of sources estimator has 6 neurons in the output layer. This allows it to estimate the number of sources as 0, 1, 2, 3, 4, or more than 4 sources. It is worth mentioning that this estimator takes into account the possibility of the absence of sources ($\tilde{N} = 0$), and when the number of sources is above 4, they are classified as $\tilde{N} > 4$.

5.3.4 Post-Processing - Selector Algorithm

After obtaining the probability distribution v of the NN's output, we use a post-processing step for selecting the most probable DoAs. This step picks the $\hat{N}$

most probable angles from the probability distribution v for when the number of sources is previously known ($\hat{N} = N$), or for when it is estimated ($\hat{N} = \tilde{N}$). Initially, the most probable angles $\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_{2\hat{N}}$ contain the estimated angles $\tilde{\theta} = \tilde{\theta}_1, \dots, \tilde{\theta}_{\tilde{N}}$. In other words, initially, $\tilde{\theta} \in \hat{\theta}$. After the post-processing, all inadequate angles are deleted, and our result is that $\tilde{\theta} = \hat{\theta}$. The post-processing pseudo-algorithm is shown in [Algorithm 2](#).

In short, we can summarize the post-processing steps³ as:

1. Get the $2\hat{N}$ highest values of v as the most probable angles $\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_{2\hat{N}}$.
2. Delete any angle that is at least 100 times smaller than the highest peak value. In other words, delete values that are $100v(\hat{\theta}_i) < \max\{v(\hat{\theta})\}$.
3. Try finding $\hat{N}$ peaks in $v(\hat{\theta})$.
4. If the remaining size of $\hat{\theta}$ is bigger than $\hat{N}$, delete the weakest angles until reaching $\hat{N}$.
5. If the size of $\hat{\theta}$ is smaller than $\hat{N}$, use the most probable angles that are in the vicinity of the peaks and belong to the $2\hat{N}$ most probable angles.

In summary, [Algorithm 2](#) finds peaks in the distribution and uses these as initial estimates, adjusting the estimation if necessary based on the angles surrounding each peak. The algorithm defines $\hat{\theta}_i$ as a peak if its value is greater than those of the adjacent elements.

[Figure 5.3](#) shows our ResNet-based NN output and the post-processing. It shows the probability distribution v , the $2\hat{N}$ most probable angles $\hat{\theta}$, and the estimated angles $\tilde{\theta}$. In this example, there are four sources, therefore $2\hat{N} = 8$. The correctly estimated DoAs were -33° , 40° , 41° , and 44° .

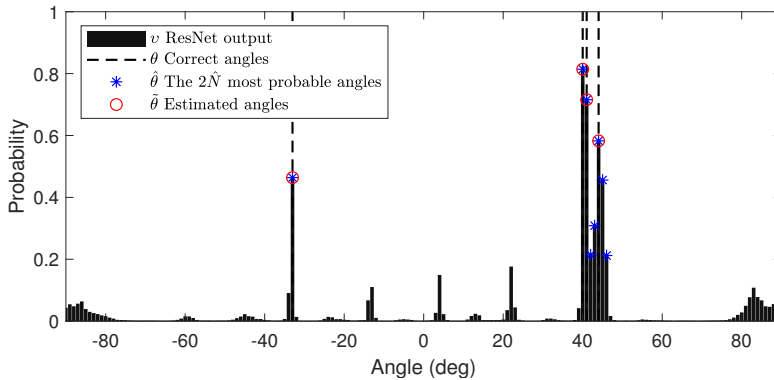


Figure 5.3: An example of the probability distribution and the post-processing of our ResNet-based estimator.

³ <https://mllimadeoliveira.github.io/Thesis/Paper3.html> Animation containing an example with the steps of our post-processing algorithms.

Algorithm 2 Post-Processing for Selecting the Most Probable Angles.

```

▷ Set  $\hat{N}$ :
if Number of sources is known then
     $\hat{N} = N$ 
else if Number of sources is estimated then
     $\hat{N} = \tilde{N}$ 
end if
Get the  $2\hat{N}$  highest values  $\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_{2\hat{N}}$  of  $v$ 
▷ Delete weak estimations:
if  $100v(\hat{\theta}_i) < \max\{v(\hat{\theta})\}$  then
    Delete  $\hat{\theta}_i$ 
end if
▷ Find peaks:
if  $v(\hat{\theta}_i) > v(\hat{\theta}_{i+1})$  AND  $v(\hat{\theta}_i) > v(\hat{\theta}_{i-1})$  then
     $\text{peaks}_i = \hat{\theta}_i$ 
end if
▷ Delete excess:
if  $\text{size}(\text{peaks}) > \hat{N}$  then
    Delete the weakest  $[\text{size}(\text{peaks}) - \hat{N}]$  peaks
end if
▷ Get more:
if  $\text{size}(\text{peaks}) < \hat{N}$  then
    ▷ Add strongest points in the vicinity of a peak:
    Add  $\hat{\theta}_{i+1}$  OR  $\hat{\theta}_{i-1}$ 
end if
▷ Now  $\hat{\theta} = \tilde{\theta}$ 
  
```

The complete ResNet-based DoA estimator starts with our ResNet-based NN and ends with the final estimation of the post-processing selector algorithm, as previously shown in [Figure 5.1](#).

5.4 Evaluation and Results

In this section, we discuss our used metrics and the results of three sets of experiments. We begin by presenting our metrics: Root Mean Square Error (RMSE), $\text{sensitivity}_{0.5}$ and sensitivity_5 . Furthermore, in the first set of experiments, we evaluated different deep learning architectures to determine the most effective one. In the second set of experiments, we compared our ResNet-based DoA estimator with the MLE algorithm in a MRA scenario. In the last experiment,

we change the scenario for ULA.

5.4.1 Metrics

Similar to other researchers [BGT19; FWG19; Eib20], we also utilized RMSE as a metric to evaluate our DoA estimation. As the name suggests, the root mean square error is calculated by taking the square root of the squared difference between the estimated value and the correct value. More precisely our RMSE is calculated as:

$$\text{RMSE} = \frac{1}{L} \sum_{l=1}^L \sqrt{\frac{1}{K \times N} \sum_{k=1}^K \sum_{n=1}^N (\theta_{n,k} - \tilde{\theta}_{n,k,l})^2}, \quad (5.6)$$

where K is the number of samples, N is the number of sources, L is the number of Monte-Carlos trials, $\tilde{\theta}_{n,k,l}$ is the estimated angle, and $\theta_{n,k}$ is the actual angle, given a certain angle n , sample k and Monte-Carlo trial l . For the situation where the number of sources is unknown, N is replaced by $\tilde{N}$.

When estimating target angles from a continuous range ($\theta \in \mathbb{R}^N$) using a discrete search grid, a basic error arises because true angles rarely align perfectly with grid points. This error creates a minimum achievable RMSE for any estimator constrained to the grid, even if the estimator were ideal. For a large, random set of angles, this minimum RMSE, reflects the average error from grid spacing alone. This lower bound ($\text{RMSE}_{\min}$) depends only on grid resolution and represents the typical deviation for large datasets with random distributions due to grid discretization:

$$\text{RMSE}_{\min} = \frac{\text{resolution}}{\sqrt{12}}. \quad (5.7)$$

Applying Equation 5.7 for our situation where we use a resolution of 1° , we can find that $\text{RMSE}_{\min} = 0.2887^\circ$.

Although RMSE is commonly used, it does not represent well certain aspects. Equation 5.6 considers that θ and $\tilde{\theta}$ have the same number of elements. However, if the estimated number of sources is incorrect ($\tilde{N} \neq N$), we would not be able to apply Equation 5.6 anymore. Therefore, we have to adapt the equation to consider such a scenario. For this, our RMSE algorithm searches for the pair of $\tilde{N}$ and N that produces the smallest difference, given a certain angle n , sample k , and Monte-Carlo trial l .

Furthermore, another limitation of Equation 5.6 is that a higher RMSE may be due to a few angles that are far from the actual values, while a lower RMSE could be due to many angles that are closer but still incorrect. In most cases, the difference between a target being incorrectly estimated by 15 or 50 degrees

is insignificant, as both errors are above a certain threshold and are considered completely wrong. To address this issue, we have also used two sensitivity-based metrics. As previously discussed, we can calculate sensitivity with Equation 3.3.

Equation 5.8 shows how to find the True Positive (TP) and False Negative (FN) to calculate the sensitivity. We define $\text{sensitivity}_{0.5}$ and sensitivity_5 as two distinct metrics, where the way they calculate TP and FN varies. For $\text{sensitivity}_{0.5}$, an estimation can be considered a true positive if the absolute difference between the correct DoA and its estimation is lower or equal to half the resolution. For sensitivity_5 , an estimation can be considered a true positive if the absolute difference between the correct DoA and its estimation is lower or equal to five times the resolution. In other words, $\text{sensitivity}_{0.5}$ and sensitivity_5 represent the percentage of angles that are correctly estimated within half the resolution and within 5 times the resolution, respectively:

$$\begin{aligned}
 \text{Sensitivity}_{0.5} &: \text{TP} = |\theta - \tilde{\theta}| \leq 0.5 \times \text{resolution} \\
 \text{Sensitivity}_{0.5} &: \text{FN} = |\theta - \tilde{\theta}| > 0.5 \times \text{resolution} \\
 \text{Sensitivity}_5 &: \text{TP} = |\theta - \tilde{\theta}| \leq 5 \times \text{resolution} \\
 \text{Sensitivity}_5 &: \text{FN} = |\theta - \tilde{\theta}| > 5 \times \text{resolution}
 \end{aligned} \tag{5.8}$$

5.4.2 Experiment Results Using Various NNs

For the first set of experiments, we tested different NN-based architectures for estimating the DoA. For this, we used a small dataset to speed up the training and evaluation process. Although this approach doesn't fully leverage the neural networks' capabilities, it provides a baseline for the performance we can achieve with comprehensive training on a larger dataset. For this, we have used 8 antennas in an MRA.

By applying the previously discussed Equation 2.20 to our smallest antenna array configuration, we find that the spacing between antennas equals:

$$D = [0, 1, 4, 6, 13, 14, 17, 19] \times \lambda/2, \tag{5.9}$$

where λ is the radar's wavelength.

In addition, we have also used a single snapshot, a resolution of 1 degree, a FOV of 180 degrees, and 2 targets. The NNs were evaluated using 10 thousand data points and 100 Monte-Carlo trials. For this experiment, we assumed that the number of sources was known - thus, we manually input it into the selector algorithm.

From the results in Table 5.1, we conclude that our selected ResNet-based estimator outperforms both the Fully Connected Neural Network (FCNN) and the

Table 5.1: MLE algorithm compared to different NN architectures and configurations.

NN-based DoA Estimator	Number of Layers	Sensitivity _{0.5} for 2 angles with SNR = 30 dB
MLE _{2D}	-	88.9 %
FCNN	8 FC Layers	82.29 %
CNN	4 Conv. + 2 FC Layers	79.7 %
ResNet	12 Conv. + 2 FC Layers	90.1 %

Convolutional Neural Network (CNN) based DoA estimators. It is important to note that despite having more layers, our ResNet-based estimator is less computationally intense than a CNN-based estimator and on par with the FCNN-based estimator.

5.4.3 Experiment Results Using MRA

For the second set of experiments, we tested our ResNet-based DoA estimator against the MLE algorithm in various scenarios, using the same MRA configuration as previously described in Equation 5.9. However, in this experiment, the ResNet-based neural network was trained using a large dataset of algorithmically generated data containing 40 million data points. This dataset contains 0 to 5 targets, normalized amplitudes varying from 0.5 to 1, and SNR ranging between 0 and 30 dB. Although 40 million data points may seem a large number, they represent only a small fraction of all possible scenarios. Thus, during training, the neural network had to learn to generalize.

The evaluation set was split into two subsets: one that assumes the number of sources was known, and another that does not make such assumptions.

Assuming the number of sources was known

In this first subset of the second set of experiments, we assumed that the number of sources was known and did not attempt to estimate it when evaluating the estimators.

Due to the computational complexity of the MLE algorithm, we made some adjustments when building our comparison by reducing its FOV and limiting the evaluation data as the number of sources increased. For the MLE algorithm, the experiment with 1 source had a FOV of 160° and 10,000 data points. With 2 sources, the FOV was 160°, and we used 10,000 data points. For 3 sources, the FOV was 150°, and we used 8,000 data points. For 4 sources, the FOV was 100°, and we used 5,000 data points. For the ResNet-based estimators, the FOV was always set to 180° regardless of the scenario.

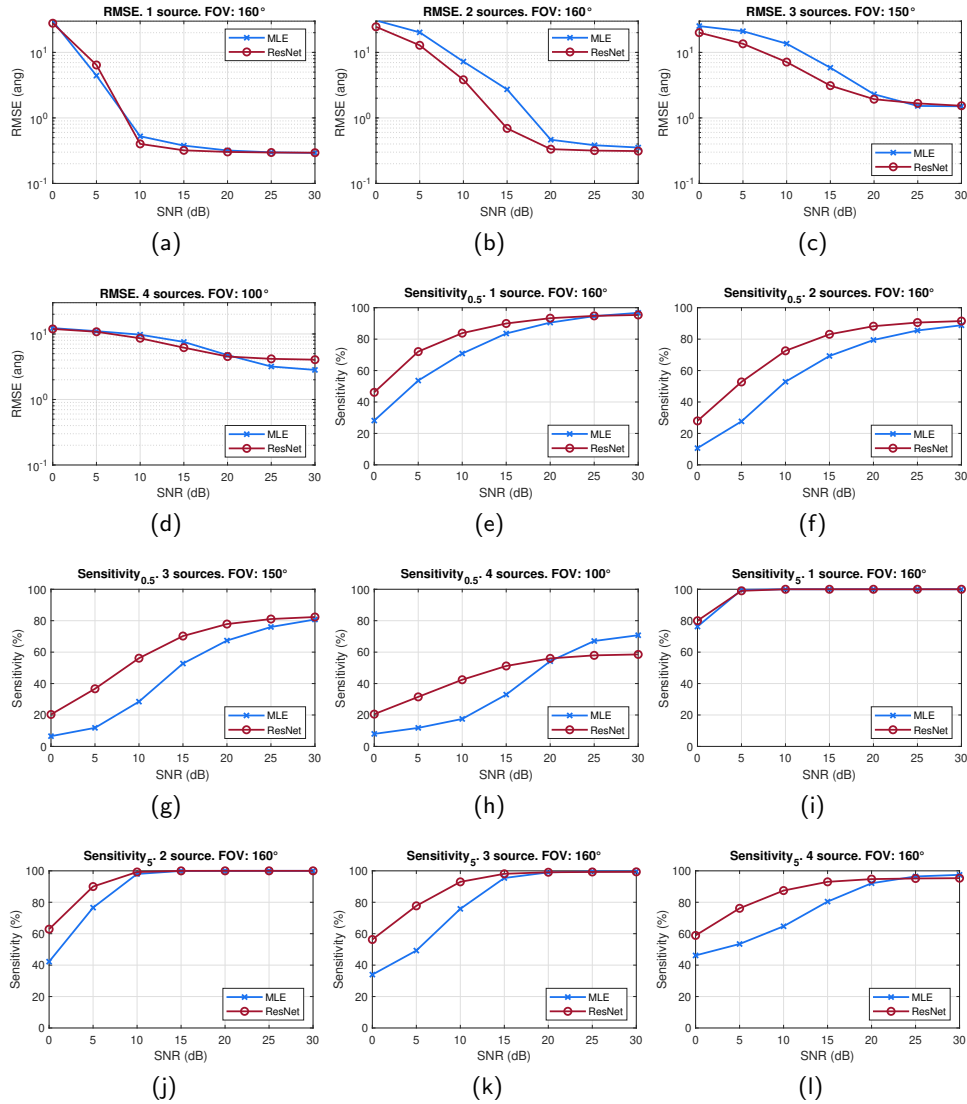


Figure 5.4: (a)-(d) RMSE vs. SNR, (e)-(h) Sensitivity_{0.5} vs. SNR, (i)-(l) Sensitivity₅ vs. SNR. The results of our experiments between our proposed ResNet-based estimator and the MLE algorithm, using 1 to 4 sources. For this experiment, the number of sources was previously known.

Figure 5.4 shows the results for this experiment. Our proposed deep learning model performs well in a single-snapshot MRA environment. Our ResNet-based estimator showed comparable performance to maximum likelihood estimation for one source. It is interesting to note that for one source, both methods get close to the theoretical RMSE lower bound (0.288°), achieving 0.292° for the MLE algorithm and 0.294° for our estimator. For two sources, our ResNet-based estimator slightly outperformed the MLE algorithm. For three and four sources, our estimator was better than the MLE algorithm for low SNR levels (high noise) and slightly worse for high SNR levels (low noise). Overall, our estimator performs well under high noise levels, and it is similar to the MLE algorithm when noise decreases.

Furthermore, Figure 5.4 also shows the $\text{sensitivity}_{0.5}$ and sensitivity_5 for different scenarios. The trends observed for the RMSE results are consistent with the ones obtained with $\text{sensitivity}_{0.5}$ and sensitivity_5 , except for the case of three sources with high SNR. In this scenario, the neural network has a higher $\text{sensitivity}_{0.5}$ than the MLE algorithm, but due to having a lower sensitivity_5 , it has a slightly lower RMSE.

Assuming the number of sources was unknown

In this second subset of the second set of experiments, we assumed that the number of sources was unknown, requiring the DoA estimators to estimate it.

Due to the computational complexity of the MLE algorithm, it was only feasible to run Algorithm 1 for up to three sources. We also had to limit the search grid and FOV of the MLE. The experiment with 1 source had a FOV of 160° and 10,000 data points. With 2 sources, the FOV was 150° , and we used 8,000 data points. For 3 sources, the FOV was 100° , and we used 3,000 data points. For the ResNet-based estimators, the FOV was always set to 180° regardless of the scenario.

Figure 5.5 shows the RMSE and the percentage of correctly estimated sources for various FOVs and sources. We observed that our ResNet-based estimator showed a marginal improvement in RMSE performance over the MLE algorithm when the number of sources was unknown compared to when it was known (Figure 5.4). Yet, when estimating the number of sources, our ResNet-based estimator significantly outperforms Algorithm 1 and MLE.

This mismatch between RMSE performance and the estimation of the number of sources is a result of the way RMSE is calculated, as discussed in Subsection 5.4.1. This happens because RMSE only considers whether any of the estimated angles are similar to the actual angles. In other words, if the correct DoAs is a subset of the estimated DoAs $\theta \in \tilde{\theta}$, the RMSE would be zero. For

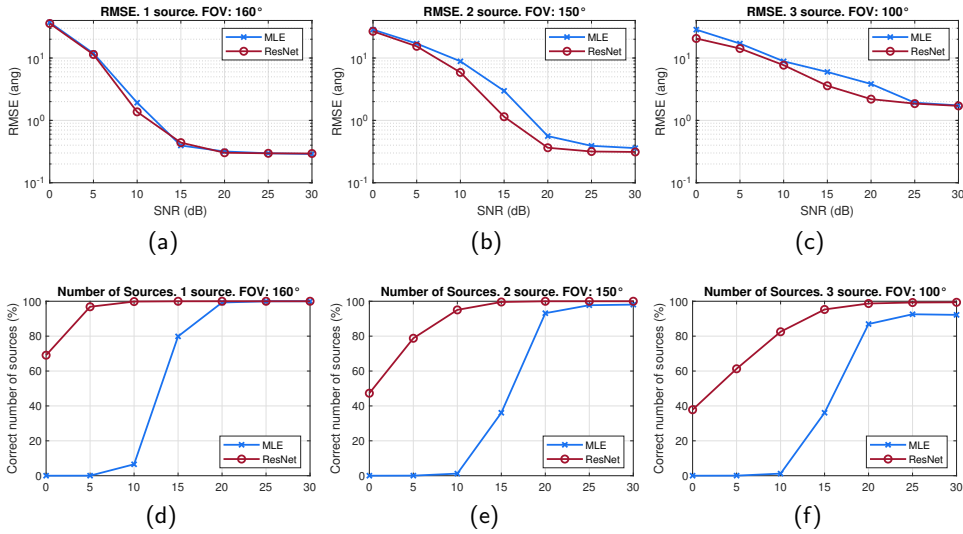


Figure 5.5: (a)-(c) RMSE vs. SNR, (d)-(f) Number of sources vs. SNR. In this experiment, the number of sources is unknown and it is estimated together with the DoAs. Here, we vary the number of sources between 1 and 3.

example, if $\theta = [-10^\circ, 30^\circ]$ and $\tilde{\theta} = [-10^\circ, 30^\circ, 60^\circ]$, $\text{RMSE} = 0$, but $\tilde{N} \neq N$.

5.4.4 Experiment Results Using ULA

In the final experiment, we aimed to determine whether our ResNet-based DoA estimator would perform as well in an ULA configuration as it did with MRA. For this, we attempted to replicate the results of Bialer et al. [BGT19], using a resolution of 0.1 degrees and a FOV of 20 degrees, but were unsuccessful. As a result, we conducted our comparison using the MLE algorithm, the RootMUSIC algorithm (a variation of the MUSIC algorithm) with spatial smoothing, and the ESPRIT algorithm with spatial smoothing [Van02]. We previously discussed MUSIC, ESPRIT, and spatial smoothing in Section 2.3.

In this experiment, we retrained our neural network with 12 million data points, 0 to 5 targets, normalized amplitudes varying from 0.5 to 1, and SNR ranging between 0 and 30 dB. Our new antenna model includes 12 uniformly spaced antennas, a field of view of 20 degrees, and a resolution of 0.1 degrees. This means that this new ResNet-based neural network has an output layer with 201 neurons, in contrast with the 181 neurons in our previous NN.

By applying the previously discussed Equation 2.17 to our uniform linear array

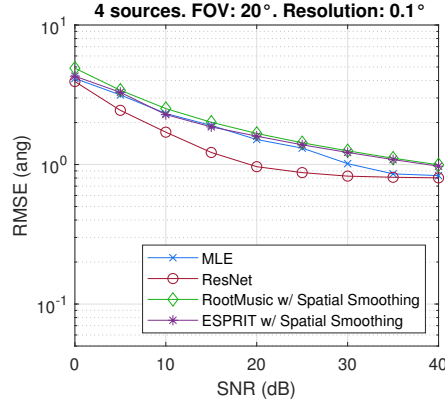


Figure 5.6: RMSE vs. SNR for ULA, comparing MUSIC, ESPRIT, MLE and our ResNet-based DoA estimator.

configuration, we find that the spacing between antennas equals:

$$D = [0, 1, 2, 3, 4, \dots, 11] \times \lambda/2, \quad (5.10)$$

where λ is the radar's wavelength.

For this experiment, we considered the number of sources to be known and we evaluated the challenging case with 4 DoAs to demonstrate the performance of our ResNet-based DoA estimator. Figure 5.6 shows the results of this experiment. Here, our ResNet-based estimator outperformed all three other algorithms, although the MLE algorithm had a comparable performance for high SNR.

5.5 Conclusion

In this chapter, we introduced an approach to address the direction of arrival estimation problem and the number of sources estimation problem using a residual neural network. We discussed various scenarios involving different fields of view, resolutions, and antenna configurations. We evaluated both Minimum Redundancy Linear Arrays (MRA) and Uniform Linear Arrays (ULA).

The results of our ResNet-based direction of arrival estimator show promise. It performed better than the maximum likelihood estimation algorithm, particularly in high-noise scenarios and when estimating the number of sources. This means that our estimator's performance is comparable (sometimes superior) to the MLE algorithm for estimating the DoAs and superior when estimating the number of sources. Our unproved hypothesis of why our model can sometimes have a better performance than the MLE algorithm is that the Gaussian noise assumption does not hold for all scenarios. Since the NN can learn different noise

Chapter 5: ResNet for Single-Snapshot DOA Estimation

distributions, it could potentially outperform the MLE algorithm in some noisy scenarios.

As with any neural network, the main drawback of our ResNet-based estimator is its computational intensive during the training process. Another drawback is the post-processing algorithm that selects peaks without determining if they correspond to targets or not.

Fusion of ResNet and MLE for DoA Estimation

ABSTRACT

This chapter introduces a DoA estimation model named Deep-MLE as a possible solution to the single-snapshot direction of arrival estimation problem. Deep-MLE combines a deep Neural Network (NN) with the classical analytical Maximum Likelihood Estimation (MLE) algorithm. By combining these two methods, we can overcome some of their limitations, such as computational complexity from the MLE algorithm and incorrect predictions from the NN. For that, Deep-MLE utilizes the MLE algorithm to partially correct the estimations made by the neural network, while having a reduced computational complexity when compared to MLE. The results showed that Deep-MLE performs better than using solely MLE or NNs for DoA estimation.

In this chapter, we introduce an approach called Deep-MLE for the problem of estimating the Direction of Arrival (DoA). Unlike the model presented in [Chapter 5](#), which utilized a 2D ResNets-based neural network and a selector algorithm, this approach takes a different direction by combining a lighter 1D ResNet-based neural network architecture together with the MLE algorithm. With this combination, we can achieve a higher performance with a lower computational complexity when compared to the previously discussed models.

We first start this chapter by discussing the scope of this chapter in the introduction. Next, we expand parts of the signal model introduced in [Chapter 5](#). Furthermore, we discuss the architecture of our Deep-MLE, giving the details of

its neural network and reduced MLE algorithm. Finally, we present two sets of experiments and their results. Unlike the previous chapter, this chapter does not consider the estimation of the number of targets.

6.1 Introduction

Building on [Chapter 5](#), in this section, we continue the discussion of Direction of Arrival (DoA) estimation [[RM19](#); [Abo+17](#)] and introduce our fusion model called Deep-MLE. In the sequence, we present two related works.

In this chapter, we go further into the usage of Maximum Likelihood Estimation (MLE). Here, we have used it for comparison as well as an estimator within our proposed model. Our choice comes from the fact that according to Hacker and Yang [[HY10](#)], the MLE algorithm is one of the most reliable analytical techniques to estimate the DoA when using a single snapshot and MRA.

As previously discussed, the MLE algorithm works by testing all possible angle combinations and finding the one that gives the least amount of residual error [[SS90](#)]. However, the computational complexity of MLE grows exponentially with the number of sources, making it a computationally intensive algorithm to use.

Deep-MLE is a fusion model that combines a data-driven method with an analytical one. For this, it uses a 1D Residual Neural Network (1D ResNet) based neural network architecture and the Maximum Likelihood Estimation (MLE) algorithm. Deep-MLE takes a single snapshot from an eight-antenna MRA and estimates the unknown target angles. As shown in [Figure 6.1](#), firstly, the pre-processed snapshot passes through the 1D ResNet-based neural network. From the output of the neural network, we pre-select the most probable angles. Then, the MLE algorithm within our model estimates the output angles of Deep-MLE based on the pre-selected set, significantly reducing its computational complexity by up to ten thousand times compared to the traditional MLE algorithm.

Other researchers also have utilized other combinations of data-driven techniques with signal-processing techniques to improve their models. Liu et. al.

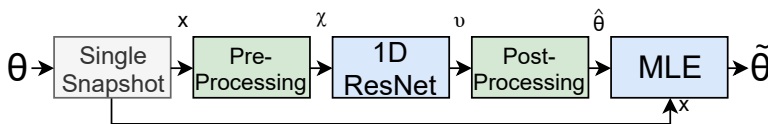


Figure 6.1: The processing pipeline of our Deep-MLE model.

[Liu+21], fed a single snapshot into several parallel Convolutional Neural Networks (CNNs) to estimate the spatial spectrum of MUSIC. In a similar work from another author, Elbir [Elb20], a deep learning method was used to learn the MUSIC spectral representation, creating a MUSIC/DL hybrid. In their work, they presented a deep learning approach for the direction of arrival (DoA) estimation called DeepMUSIC. This model also consists of multiple Convolutional Neural Networks (CNNs) that were trained to classify signals based on their angular spectrum. Each CNN is responsible for a specific subregion of the spectrum, learning its corresponding MUSIC spectrum.

6.2 Signal Model

In this section, we expand the signal model previously presented in Chapter 5 by introducing normalized variable amplitudes and the computational complexity for MLE and neural networks.

As previously presented, the snapshot is given as $x = \mathbf{A}s + \eta$, where s is the sources' signal, η is the added Gaussian noise, and $\mathbf{A}$ is the steering matrix. The matrix $\mathbf{A}$ contains the directions of arrival $\theta = \theta_1, \dots, \theta_N$. Expanding the previously discussed Equation 2.14, this chapter also considers the normalized amplitude $A = A_1, \dots, A_N$, where $0 < A_i \leq 1$:

$$\mathbf{A}(\theta) = \begin{bmatrix} A_1 e^{j2\pi\lambda^{-1}D_1(d)\sin(\theta_1)} & \dots & A_N e^{j2\pi\lambda^{-1}D_1(d)\sin(\theta_N)} \\ \vdots & & \vdots \\ A_1 e^{j2\pi\lambda^{-1}D_M(d)\sin(\theta_1)} & \dots & A_N e^{j2\pi\lambda^{-1}D_M(d)\sin(\theta_N)} \end{bmatrix}, \quad (6.1)$$

in which λ is the wavelength of the transmitting signal, $d = \lambda/2$, and the positions of each antenna of the array are given as $D(d) = [0, 1, 4, 6, 13, 14, 17, 19]d$ (Equation 2.20).

6.2.1 Maximum Likelihood Estimation

As previously discussed, the Maximum Likelihood Estimation (MLE) algorithm works by trying all possible steering matrices with different angle combinations that satisfy the Equation 2.38. It is important to note that this maximum likelihood estimation is not the only possible MLE algorithm. Other versions of MLE might have different trade-offs between performance and complexity [RB97; ZHL07].

Moreover, as previously shown in Equation 2.42, the number of times Θ that the MLE algorithm needs to perform Equation 2.38 to find the angle combination with the least residual error is given by the following combination:

$$\Theta = \binom{G}{N}, \quad (6.2)$$

where G is the grid size and N is the number of sources. The search grid size G is a function of the FOV and resolution: $G = (\text{FOV}/\text{resolution}) + 1$. For instance, if $\text{FOV} = 180^\circ$ and the resolution is 1° , then $G = 181$, and for $G = 181$ with $N = 4$, the number of searches is $\Theta = \binom{181}{4} = 43,252,665$.

6.2.2 Computational Complexity

The computational complexity of the MLE algorithm depends the most on the pseudo-inverse operation, which most implementations rely on the Singular Value Decomposition (SVD) matrix operation and has a computational complexity of $O(M^3)$ [Cou08]. As MLE computes the pseudo-inverse Θ times, its computational complexity, denoted as O_{MLE} , can be estimated as:

$$O_{\text{MLE}}(M^3\Theta), \quad (6.3)$$

where M is the number of antennas in the array.

Using eight antennas ($M = 8$) and the same grid as before ($G = 181$), it is possible to calculate the number of operations for different numbers of targets. The number of operations for 1D MLE (one target) is 9.2×10^4 , 2D MLE is 8.3×10^6 , 3D MLE is 4.9×10^8 , and 4D MLE is 2.2×10^{10} . In short, the MLE algorithm might become impractical for real-time applications if more than two targets are present.

On the other hand, the computational complexity of neural networks depends on the type and number of layers. For a convolutional layer, the complexity O_{conv} depends on the number of filters (also called channels) of its input and output (C_{in} and C_{out} respectively), the input size (I_{in}), kernel filter size Kf , and the number of strides S used in convolution:

$$O_{\text{conv}}(C_{\text{in}}C_{\text{out}}\frac{I_{\text{in}}}{S}Kf). \quad (6.4)$$

For a fully connected layer, the computational complexity O_{fc} is determined by just the input and output sizes (I_{in} and I_{out} respectively):

$$O_{\text{fc}}(I_{\text{in}}I_{\text{out}}). \quad (6.5)$$

Table 6.1: Comparison between different architectures.

NN-based DoA Estimator	Number of Operations	Number of Layers	Sensitivity for 2 angles with SNR = 30 dB
MLE _{2D}	8.3×10^6	-	88.9 %
FCNN	3.3×10^6	8 FC Layers	82.29 %
CNN	2.5×10^7	4 2D Conv. + 2 FC Layers	79.7 %
2D ResNet	5.1×10^6	12 2D Conv. + 2 FC Layers	90.1 %
1D ResNet	1.68×10^6	12 1D Conv. + 2 FC Layers	90.1 %

All NNs were trained with a reduced dataset at this stage.

6.3 Deep Learning Architecture

The choice of using a 1D ResNet-based NN as the neural network within Deep-MLE is a result of experimenting with various neural network architectures while trying to improve DoA estimation. Table 6.1 expands the previously discussed Table 5.1 by introducing the number of operations.

The results of Table 6.1 show that the 1D ResNet has the smallest number of operations among the tested neural networks while offering the same sensitivity as the 2D ResNet.

As previously discussed in Chapter 5, He et. al. [He+16] proposed the original ResNet model. However, in contrast to their work, our 1D ResNet-based NN is a small ResNet model that utilizes 1D convolutions¹ instead of 2D convolutions². Figure 6.2 shows the architecture of our 1D ResNet-based NN.

With the knowledge of the used 1D ResNet architecture (Figure 6.2), we can apply Equation 6.4 and Equation 6.5 to each layer of our proposed NN and we

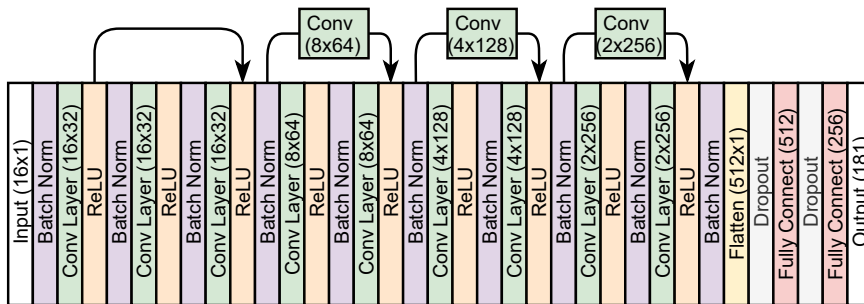


Figure 6.2: Our 1D ResNet-based neural network architecture for selecting DoAs candidates within Deep-MLE.

¹ <https://mllimadeoliveira.github.io/Thesis/1dconv> Example of a 1D convolution.

² <https://mllimadeoliveira.github.io/Thesis/2dconv> Example of a 2D Convolution.

can obtain that the number of operations of our 1D ResNet-based neural network is 1.68×10^6 . Unlike MLE, the number of operations remains constant and does not depend on the number of targets.

6.4 Deep-MLE

In this section, we discuss all the components of Deep-MLE and how they relate to each other. As shown in [Figure 6.1](#), we can break down the Deep-MLE components as: pre-processing, 1D ResNet-based neural network, post-processing, and MLE.

In short, firstly, the single snapshot x passes through a pre-processing step to prepare it for the NN and becomes χ . In the sequence, χ passes through the neural network that outputs the probability distribution v . In the post-processing, some angles $\hat{\theta}$ are selected from v and input into the MLE algorithm. The output of the MLE algorithm is the DoA estimation $\tilde{\theta}$. Essentially, Deep-MLE inputs x and outputs $\tilde{\theta}$. The remainder of this section provides additional details on the components of the Deep-MLE.

The initial step in our Deep-MLE is the pre-processing step. As previously stated, unlike the original ResNet paper [\[He+16\]](#), our 1D ResNet takes a vector as input rather than a matrix. Moreover, we decided to use weights with real numbers instead of complex numbers to facilitate the training. Therefore, we needed to modify the received snapshot to be a real vector. For that, our neural network's input is a vector containing both the real and imaginary parts of the snapshot x concatenated. More precisely, the input of our ResNet, $\chi \in \mathbb{R}^{2M}$, is:

$$\begin{aligned}\chi_i &= \text{Re}\{x_i\}/c_0 \\ \chi_{i+M} &= \text{Im}\{x_i\}/c_0\end{aligned}\quad (6.6)$$

where $i = 1, \dots, M$, and x_i represents the i -th element of the snapshot. It is important to note that χ has a size of $2M$, as we concatenate both the real and imaginary parts of the snapshot x into a single vector. As we used 8 antennas, our χ had 16 elements. Moreover, to normalize the input, we used a constant c_0 , which was empirically determined. This constant ensures the input is between -1 and 1 .

Similarly to the previously discussed neural network in [Chapter 5](#), the final layer of the neural network is fully connected. The number of neurons in this layer depends on the desired search grid. For a FOV of 180 degrees and a resolution of 1 degree, the number of neurons in the output layer is 181.

The output from our 1D ResNet-based NN is the probability distribution v , with each neuron's value ranging between 0 and 1. Similarly to [Chapter 5](#), this

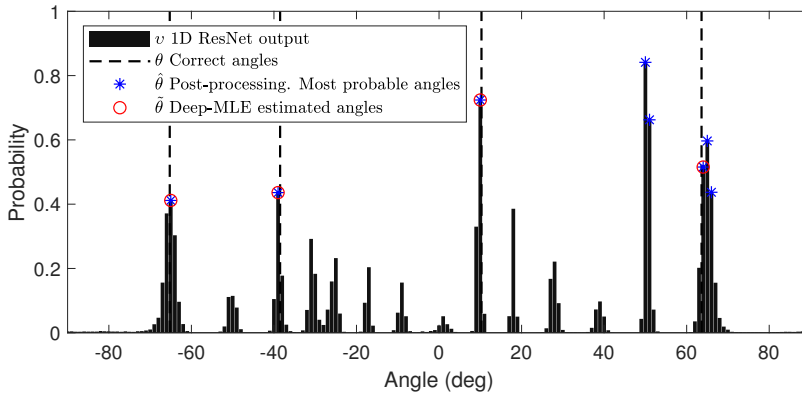


Figure 6.3: An example of the probability distribution, the post-processing step, and MLE of our Deep-MLE for DoA estimation.

probability reflects the NN's “confidence” in the presence or absence of a target at a specific angle, rather than representing the actual probability of the problem.

Furthermore, the post-processing step selects the $2N$ most probable angles of the probability distribution v . In other words, $\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_{2N}$. It is important to note that the number of selected angles varies based on the number of targets N . For instance, in the case of estimating the DoAs of 3 targets, the post-processing step would choose the 6 most probable angles. We have chosen the number $2N$ empirically, based on a balance between performance and computational complexity for Deep-MLE.

Finally, in the last step, the MLE algorithm receives as input the $2N$ most probable angles ($\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_{2N}$), and outputs the best angle combination ($\tilde{\theta} = \tilde{\theta}_1, \dots, \tilde{\theta}_N$) that would result in the minimum residual error (Equation 5.3) for a certain number of sources N .

Figure 6.3 shows how our Deep-MLE works. In the presented scenario with 4 targets ($N = 4$), the vertical black dashed lines represent the actual DoAs ($\theta = \theta_1, \dots, \theta_4$) in the snapshot, the black bars represent the NN's probability distribution (v), the blue asterisks represent most probable angles selected by the post-processing step ($\hat{\theta} = \hat{\theta}_1, \dots, \hat{\theta}_8$), and the red circles represent the output of the MLE algorithm ($\tilde{\theta} = \tilde{\theta}_1, \dots, \tilde{\theta}_4$), and thus the output of our Deep-MLE.

The previous example shows that despite the high probability of a target at 50° , it was not selected as a correct DoA by the MLE algorithm. In this case, our Deep-MLE estimated the correct DoAs, using a fraction of the computational cost of a comparing MLE algorithm running all possible angle combinations for 4 targets.

Comparing Figure 6.3 with Figure 5.3, we observe the differences between

Table 6.2: Comparison of the needed number of operations across various techniques and number of targets.

(a)			(b)		
Model Type	Number of Targets	Number of Operations	Model Type	Number of Targets	Number of Operations
MLE _{1D}	1	9.2×10^4	1D ResNet	-	1.68×10^6
MLE _{2D}	2	8.3×10^6	Deep-MLE _{1D}	1	1.68×10^6
MLE _{3D}	3	4.9×10^8	Deep-MLE _{2D}	2	1.68×10^6
MLE _{4D}	4	2.2×10^{10}	Deep-MLE _{3D}	3	1.69×10^6
			Deep-MLE _{4D}	4	1.72×10^6

our Deep-MLE and our previously discussed ResNet-based DoA estimator. Deep-MLE is more robust as it can better correct inaccurate estimations from the neural network.

The computational complexity of Deep-MLE results from combining the computational complexities of the 1D ResNet-based NN and the MLE algorithm, but with a smaller search grid size for MLE. Instead of a search grid size of $G = (\text{FOV}/\text{resolution}) + 1$ of the regular MLE algorithm, the MLE inside of Deep-MLE has a grid size of $G = 2N$. Therefore, the computational complexity of Deep-MLE is given as:

$$O_{\text{dMLE}}(O_{\text{ResNet}} + M^3 \binom{2N}{N}), \quad (6.7)$$

where the number of operations of the 1D ResNet-based neural network is 1.68×10^6 . Hence, the number of operations for estimating the DoAs of 4 targets drops from 2.2×10^{10} (MLE) to 1.72×10^6 (Deep-MLE).

Table 6.2 shows the number of operations needed for MLE, 1D ResNet and Deep-MLE for up to four targets.

6.5 Evaluation and Results

In this section, we discuss the results of two sets of experiments after briefly discussing our metrics. In the first set of experiments, we evaluate the performance of different models by varying the SNR levels. In the second set of experiments, we evaluate the performance of different models by varying the normalized amplitude of one of the targets.

This chapter uses the same metrics we previously discussed in Chapter 5. Root Mean Square Error (RMSE) (Equation 5.6), and two sensitivities (Equation 5.8). Sensitivity_{0.5} is the percentage of angles that are correctly estimated

within half the resolution, and sensitivity_5 is the percentage of angles that are correctly estimated within 5 times the resolution.

For the following experiments, we trained our 1D ResNet using 50 million generated data points. Each data point contained 0 to 5 sources, with SNR (Signal-to-Noise Ratio) ranging between 0 and 30 dB, normalized signal amplitude varying from 0.5 to 1, and resolution of 1 degree. While 50 million data points might seem like a large amount of data, it represents an insignificantly small portion of all combinations of targets, SNR, and amplitudes. Hence, there is a need for the neural network to learn how to generalize its training data.

6.5.1 SNR Experiment

In this first set of experiments, we evaluate various models by calculating their RMSE, $\text{sensitivity}_{0.5}$, and sensitivity_5 for different SNR levels.

For this, we have used three models: A 1D ResNet-based DoA estimator with a selector algorithm (similar to the previously discussed model in [Chapter 5](#)), a complete MLE algorithm that tests all possible angle combinations, and our proposed Deep-MLE.

The experiment results are shown in [Figure 6.4](#). We can note that our proposed Deep-MLE tends to follow the ResNet performance for low SNR while following the performance of MLE for high SNR. At times, Deep-MLE also outperformed both of the other techniques.

For one source, Deep-MLE had the best performance for low SNR, and a similar performance to the other models for high SNR. For two sources, Deep-MLE also had the best performance for low SNR, and the second best performance for high SNR. For three sources, Deep-MLE always had the best performance. For four targets, Deep-MLE closely followed the performance of ResNet for low SNR and the performance of MLE for high SNR.

It is interesting to note, that when observing the sensitivity_5 , most of the time, Deep-MLE performed better than the other models, even though, at times, 1D ResNet had better $\text{sensitivity}_{0.5}$.

Furthermore, if we define Low-SNR as the SNR between 0 dB and 10 dB, and High-SNR as the SNR between 15 dB and 30 dB, and if we take the difference of the averages RMSE for Low-SNR and High-SNR, we can create yet another way to interpret our results. The difference of the averages RMSE for Low-SNR between Deep-MLE and the MLE algorithm is -3.4° and between Deep-MLE and the 1D ResNet-based estimator is -0.3° . The difference of the averages RMSE for High-SNR between Deep-MLE and the MLE algorithm is -0.06° and between Deep-MLE and the 1D ResNet-based estimator is -0.2° . Hence, we can state that, on average, Deep-MLE has a better performance than the other tested models.

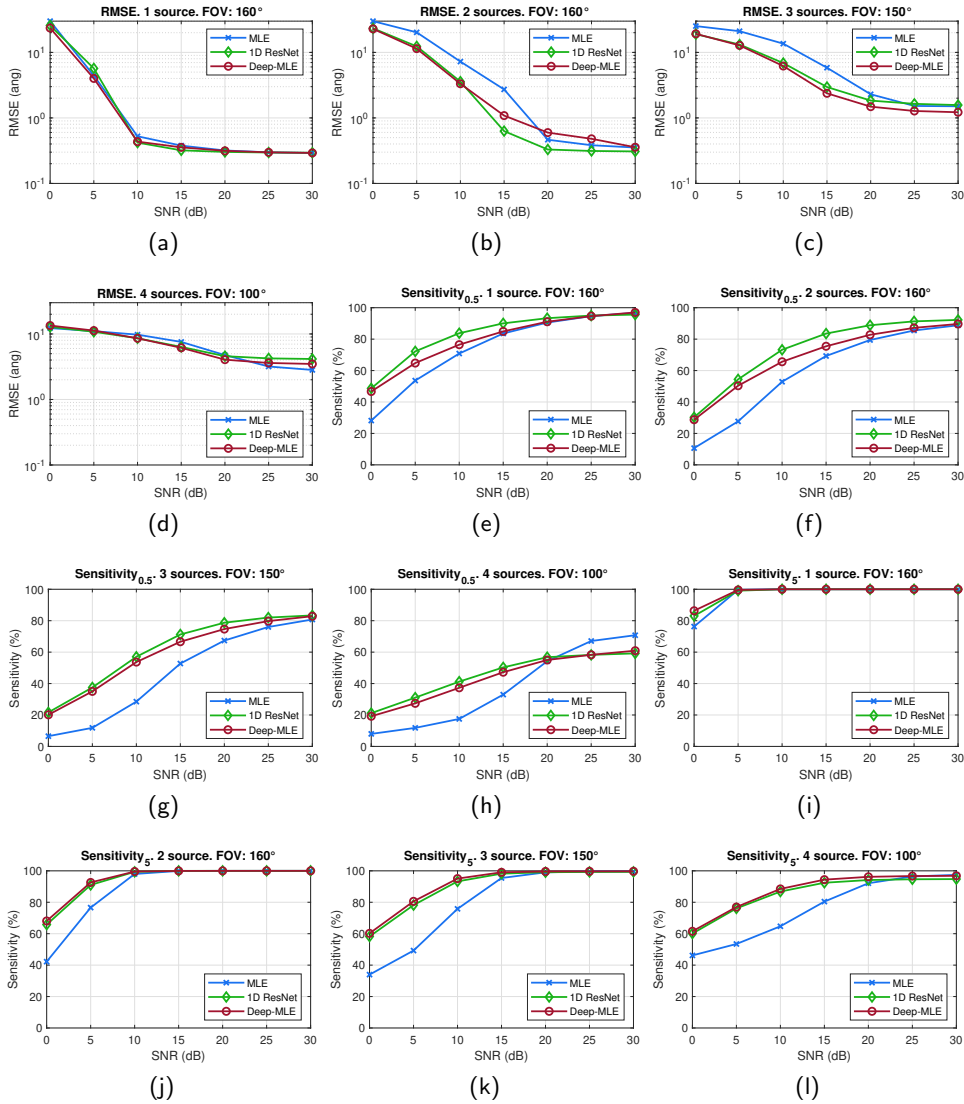


Figure 6.4: (a)-(d) RMSE vs. SNR, (e)-(h) Sensitivity_{0.5} vs. SNR, (i)-(l) Sensitivity₅ vs. SNR. The results of our experiments using MLE, 1D ResNet-based model, and our Deep-MLE. We have used 1 to 4 sources, and the number of sources was known.

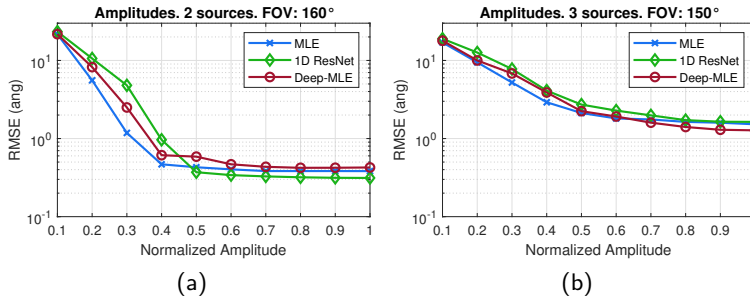


Figure 6.5: RMSE vs. changing the amplitude of 1 source. Scene SNR = 25dB. The number of targets was known. This experiment compares the RMSE performance of MLE, 1D ResNet-based model, and our Deep-MLE. (a) 2 sources (b) 3 sources

6.5.2 Amplitude Experiment

Up to this point, we have always considered that all sources have the same unitary normalized amplitude. In this second set of experiments, we tested the impact of changing the amplitude for the angle estimation. For this, we have created an experiment with two sources and an experiment with three sources, in which only one of them has a varying normalized amplitude. In other words, we have varied the normalized amplitude of one source from 0.1 to 1 and kept the other sources with a constant normalized amplitude of 1.

Figure 6.5 shows the result of this experiment. Deep-MLE had the second-best performance when one of the sources was much weaker. For two targets and high normalized amplitude, Deep-MLE performed closely to MLE. For three targets and high normalized amplitude, Deep-MLE performed better than the other two models.

6.6 Conclusion

In this chapter, we described and evaluated Deep-MLE, a fusion model that combines the analytical MLE algorithm with a 1D Residual Neural Network (ResNet) to estimate the Direction of Arrivals (DoAs).

The results of our experiments revealed that our fusion model Deep-MLE, on average, outperforms (in terms of RMSE) both MLE and 1D ResNet while providing a lower computational complexity than MLE.

We evaluated our approach using a synthetically generated dataset containing different Signal-to-Noise Ratio (SNR) levels, number of sources, and normalized amplitudes.

The disadvantage of such an approach is that, like many other data-driven methods, it requires training and a dataset that accurately represents most scenarios. In addition, it requires 3 megabytes of memory for storing the NN-trained weights that might not be available in embedded systems.

Fusion Model for Antenna Imperfection Mitigation

ABSTRACT

In this chapter, we discuss the effect of antenna gain and phase imperfections in a Minimum Redundancy Array (MRA), and their effect on DoA estimation and number of sources estimation. To mitigate the antenna imperfections, we use a model that uses a deep neural network together with the Maximum Likelihood Estimation (MLE) algorithm called Deep-MLE.

The novelty of this version of Deep-MLE lies in the fact that it learns the antenna gain and phase errors as a function of the direction of arrival instead of applying a single compensation value for all angles.

In this chapter, we build upon the discussions of our two previous chapters and we expand our scope by introducing antenna gain and phase imperfections mitigation into a data-driven DoA estimation model, allowing us to evaluate our model using measured data. This model is an updated version of Deep-MLE discussed in [Chapter 6](#), and can estimate both the direction of arrival and the number of sources, as also discussed in [Chapter 5](#).

For this, we first start this chapter with the introduction, where the scope of this chapter is discussed. In the sequence, we expand the previously discussed signal model by introducing antenna gain and phase imperfections. Next, we discuss the updated architectures of our Deep-MLE and its reduced-size version. In the sequence, we discuss the used data (synthetic and measured). Finally, we present three sets of experiments and their results.

This chapter is based on the published work [[LB23](#)].

7.1 Introduction

Expanding on the techniques presented in [Chapter 5](#) and [Chapter 6](#), we further discuss the estimation of the Direction of Arrival (DoA) and the number of sources by incorporating antenna imperfection into our Deep-MLE model, a fusion approach that combines a neural network with the Maximum Likelihood Estimation (MLE) algorithm.

In this chapter, we introduce antenna gain and phase imperfections to Deep-MLE. The incorporation of antenna imperfections is important when using measured data as perfect antenna models may be too simplified for real-world applications. By considering the criticism of Chung et. al. [[CPK22](#)], where they point out a lack of experimental validation with measured data as a frequent oversight in research for DoA estimation using deep learning, we evaluated Deep-MLE with both synthetic and measured data.

To compensate for the antenna imperfection, our Deep-MLE learns that the gain and phase errors of the antenna as functions of the direction of arrival. Consequently, our model does not use a single value to compensate for all angles - which is commonly applied in combination with the MLE algorithm. [[TF09](#)]

One disadvantage of Deep-MLE is the memory requirement. It requires 3 megabytes of storage. To improve the memory requirement, we introduced reduced-size versions of our deep-learning models to fit a memory of 256 kilobytes.

As a related work, the researchers of [[Yan+22](#); [MZS22](#)] have introduced fusion models that use deep learning and MUSIC to estimate DoAs, but they do not consider antenna imperfections. Liu et. al. [[LZY18](#)] proposes a NN that deals with array imperfections during the DoAs estimation, but they do not test their models with measured data. In the work of Chung et. al. [[CPK22](#)], they claim that previous researchers on data-driven DoA estimation lack experimental validation. This could cause a mismatch between the evaluation with synthetic and measured data.

7.2 Signal Model

In this section, we expand the signal model, introduce the antenna gain and phase errors, and discuss their equations.

As previously discussed in [Chapter 5](#) and [Chapter 6](#), the snapshot equation is defined as $x = \mathbf{A}s + \eta$, with s is the sources' signal, η denoting added Gaussian noise, and $\mathbf{A}$ being the steering matrix that includes the directions of arrival $\theta = \theta_1, \dots, \theta_N$.

However, this definition of the snapshot equation relies on the assumption of a Gaussian noise distribution, which rarely is the case for real antennas. Imper-

fections introduce divergences from this ideal scenario, as the received snapshot often does not have a zero-mean Gaussian error distribution.

In real-world antennas, gain and phase errors are caused by imperfections in the hardware and incorrect modeling of the actual position of the antennas. We previously showed an example of a gain error in [Figure 1.8](#).

Calibration techniques are often applied to correct these errors [[CS96](#); [LL18](#); [KV96](#)]. These calibrations are applied before the DoA estimator and, therefore, do not take the angle dependency of the gain and phase errors into account. [[TF09](#)]

The gain error is $\alpha(\theta) \in \mathbb{R}^M$ and the phase error is $\varphi(\theta) \in \mathbb{R}^M$. Although these variables are a function of the DoA θ , for brevity, the gain and phase errors may also be represented as α and φ , omitting the θ .

Considering a matrix $\mathbf{E}(\alpha, \varphi) \in \mathbb{C}^{M \times M}$ as the so-called error matrix that contains all the gain/phase errors for each antenna, the snapshot equation becomes:

$$x = \mathbf{E}(\alpha, \varphi) \mathbf{A} s + \eta. \quad (7.1)$$

For brevity, we may also use $\mathbf{E}$ instead of $\mathbf{E}(\alpha, \varphi)$.

The error matrix ($\mathbf{E}$) is a diagonal matrix, in which the gain (or amplitude) and phase errors are on its main diagonal when the mutual coupling is zero or small. As mutual coupling is out of our scope, we consider its effects to be negligible.

Because the gain and phase errors are angle-dependent (DoA-dependent) in a non-linear way, these errors are usually measured and not available as analytical expressions.

$$\mathbf{E} = \begin{bmatrix} (1 - \alpha_1(\theta)) e^{j\varphi_1(\theta)} & \dots & 0 \\ \dots & \dots & \dots \\ 0 & \dots & (1 - \alpha_M(\theta)) e^{j\varphi_M(\theta)} \end{bmatrix}, \quad (7.2)$$

in which $\alpha = \alpha_1, \dots, \alpha_M$ are the estimated gain errors, $\varphi = \varphi_1, \dots, \varphi_M$.

As previously discussed in [Equation 2.38](#), the MLE algorithm used for DoA estimation is given as $\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \|x - \mathbf{A} \mathbf{A}^+ x\|^2$, where the goal of the algorithm is to find the best angle combination that minimizes the residual error.

Moreover, the maximum likelihood estimation algorithm can also be used to find the gain and phase errors [[CS96](#)]. We can rewrite the MLE equation for DoA estimation, where the angles θ are known and we are trying to find the best gain and phase combination that minimizes the residual error:

$$\tilde{\alpha}, \tilde{\varphi} = \underset{\alpha, \varphi}{\operatorname{argmin}} \|x - (\mathbf{E} \mathbf{A})(\mathbf{E} \mathbf{A})^+ x\|^2, \quad (7.3)$$

in which $\tilde{\alpha} = \tilde{\alpha}_1, \dots, \tilde{\alpha}_M$ are the estimated gain errors, $\tilde{\varphi} = \tilde{\varphi}_1, \dots, \tilde{\varphi}_M$ are the estimated phase errors, the DoAs are known, and the steering matrix (**A**) is known.

7.3 Deep Learning Architectures

In this section, we discuss the architectures of our updated Deep-MLE. We have developed three distinct variations of the model. The first model, named Deep-MLE model A, is a direct update of the architecture discussed in Chapter 6, trained to mitigate antenna imperfections with synthetic data. The second model, named Deep-MLE model B, was developed to work with measured data. It has an additional layer and incorporates the estimation of the number of targets within its architecture. The third model, named Deep-MLE model C, is a scaled-down version of the first model and is termed “reduced-size”, distinguishing it from the “full-size” Deep-MLE models (A and B).

7.3.1 Full-Size Deep Learning Architectures

Similar to the architecture discussed in Chapter 6 and as shown in Figure 7.1, our updated Deep-MLE models work as follows. Initially, the received snapshot is pre-processed to make it a suitable input for our neural network as it only uses real numbers. During the pre-processing, the real and imaginary parts of the snapshot are separated and then concatenated into a single array that can be processed by our 1D ResNet-based neural network.

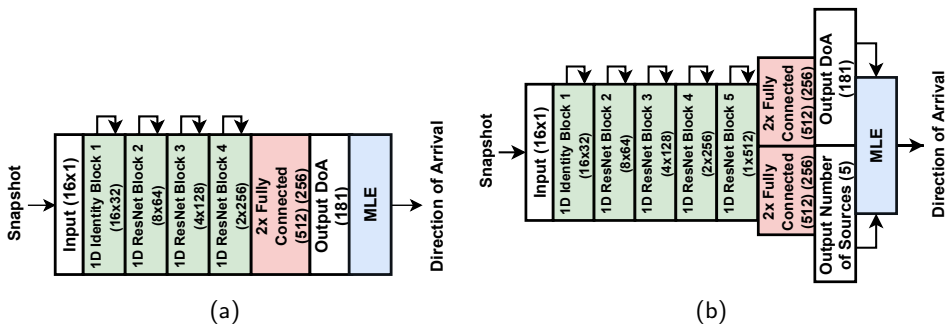


Figure 7.1: The architecture of the full-sized Deep-MLEs (a) Deep-MLE model A estimates DoAs and mitigates antenna imperfections using synthetic data. (b) Deep-MLE model B has an additional layer, estimates DoAs and the number of sources, and mitigates antenna imperfections using measured data.

In the sequence, the input array progresses through the ResNet-blocks (previously discussed in [Subsection 2.5.4](#)), where feature extraction occurs. Each block consists of batch normalization, ReLU activation, and three 1D convolutional layers. The fully connected block contains two sequences of fully connected layers with 512 and 256 neurons, respectively. The output of the NN is the probability that a target exists at a certain angle, with 181 angle bins (FOV of 180° and resolution of 1°).

Similar to what we discussed in [Chapter 5](#), for our Deep-MLE model B, the output of the ResNet-blocks bifurcates into two, each containing two sequences of fully connected layers. The second output is the estimation of the number of sources present in the scene.

Finally, the eight most probable angles and the most probable number of sources are fed into the MLE algorithm to finalize the direction of arrival estimation. The number of pre-selected angles that are fed to the MLE algorithm is fixed to eight, unlike in [Chapter 6](#), which depended on the number of sources.

7.3.2 Reduced-Size Deep Learning Architecture

Deploying neural networks on low-cost embedded systems is challenging because of their constrained memory, storage, and processing power. To address this, we decreased the memory requirement of the 1D ResNet-based neural network from 3.4 MB to under 256 KB without significantly reducing performance.

To do so, we have implemented two strategies. Firstly, we decreased the number of neurons used in both the convolutional and fully-connected layers. The second strategy was to quantize our reduced-size neural network to operate in 8-bit mode, thereby saving storage space.

Through trial and error, we concluded that we could reduce the number of filters used by the 1D convolutions by half and fix the number of fully connected neurons to 128 without excessively degrading the performance of our reduced-size Deep-MLE model C. [Figure 7.2](#) shows the reduced-size model, where the reduced number of neurons is marked in red.

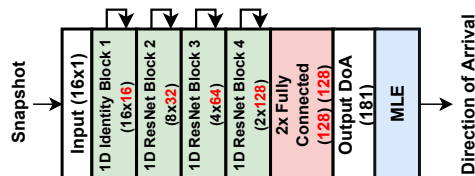


Figure 7.2: Deep-MLE model C is our reduced-size Deep-MLE architecture based on the model A but with fewer neurons (marked in red) and quantized.

After reducing the number of filters in our proposed architecture, we have reduced the storage requirement from 3.4 MB to 821 KB. This was still around three times our desired size. To induce a further reduction in the storage requirements, we quantized our already lighter neural network.

As previously discussed in [Section 2.6](#), quantization reduces the bits needed for each weight, creating a lighter model that uses less memory and processing power.

After passing through a quantization process, where we stored the weights using 8-bit precision instead of the standard 32-bit precision, the reduced-size neural network had a storage requirement of just 224 KB, which was below our desired size.

7.4 Data

In this section, we discuss two data types: synthetic and measured. Synthetic data helped us to more easily build and train data-driven models that are aware of antenna imperfections. Meanwhile, measured data allowed us to evaluate our model in real-world scenarios, decreasing the lack of experimental validation for DoA estimation using deep learning [\[CPK22\]](#).

7.4.1 Synthetic Data

To simulate antenna imperfections in synthetic data, we incorporated phase and gain error curves, as shown in [Figure 7.3](#). For this experiment, we used eight distinct gain/phase error curves (one for each antenna) taking into account the array orientation which is angle-dependent [\[Som+99\]](#). These curves were obtained through simulation. Typically, these curves are obtained through experimentation inside an anechoic chamber, where the position of the targets is known [\[RB23\]](#).

As it is challenging to accurately estimate all imperfections, we introduced approximations into the gain and error curves during the training to better represent the reality that precise antenna models are not fully known. Specifically, we trained the models with the error curves (b) and (d) from [Figure 7.3](#) and evaluated the models using the error curves (a) and (c) from [Figure 7.3](#).

7.4.2 Measured Data

While antenna imperfections are typically obtained in a laboratory environment using an anechoic chamber, this was not our case. The only information available to us was the snapshots, along with the estimated number of targets and their estimated DoAs. Although we refer to it as “ground truth”, not all DoAs were precisely known, but rather considered reliable estimates. Using this data, we

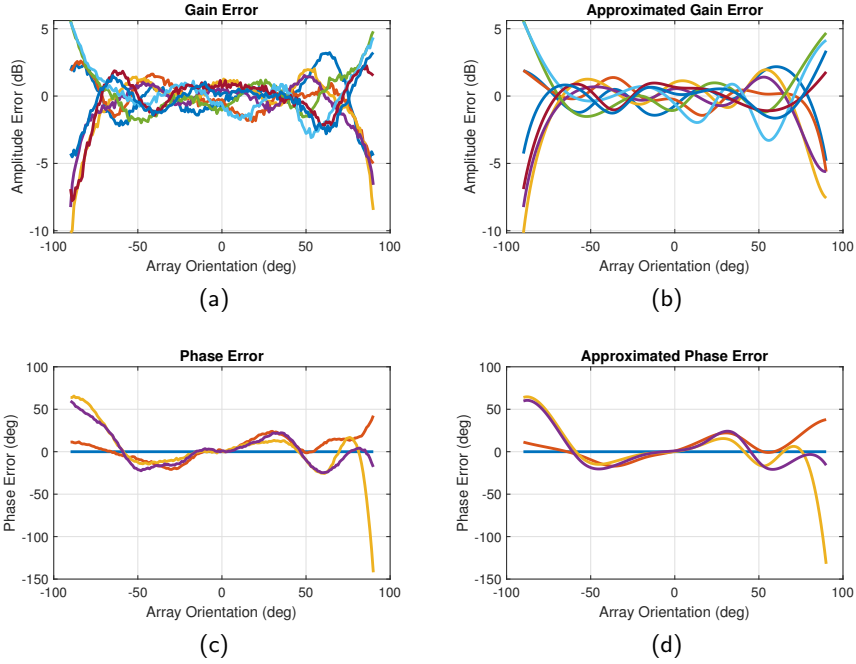


Figure 7.3: The deviation errors when compared to an ideal antenna. (a) and (c) represent the actual simulated imperfection, while (b) and (d) are approximations of these imperfections as we do not normally know the precise imperfection. These imperfections were used to train and evaluate our Deep-MLE models A and C.

estimated the gain ($\tilde{\alpha}$) and phase ($\tilde{\varphi}$) errors for different angles, following Equation 7.3. For this, we first estimated the antenna imperfections in our training data (snapshots) before using them to train our data-driven models.

However, computing the gain/phase errors for all eight antennas would increase the number of variables to estimate to sixteen. To simplify, we chose to estimate a single value for the gain and for the phase errors per known DoA.

Therefore, when estimating the error matrix $\mathbf{E}$ from our available measured data, we reduced the number of parameters to estimate to just two: the mean gain error $\mu_{\tilde{\alpha}}(\theta)$ and the mean phase error $\mu_{\tilde{\varphi}}(\theta)$. This simplification is based on our assumption that similar antennas within the same radar system may have similar imperfections.

In other words, we estimated only one mean gain and one mean phase error per angle for all antennas when using measured data. To compensate for this approximation, we assume that the gain and phase errors for each antenna ($\tilde{\alpha}_i(\theta)$)

and $\tilde{\varphi}_i(\theta)$, respectively) follow Gaussian distributions given by:

$$\begin{aligned}\tilde{\alpha}_i(\theta) &\sim \mathcal{N}(\mu_{\tilde{\alpha}}(\theta), \sigma_{\tilde{\alpha}}) \\ \tilde{\varphi}_i(\theta) &\sim \mathcal{N}(\mu_{\tilde{\varphi}}(\theta), \sigma_{\tilde{\varphi}}),\end{aligned}\tag{7.4}$$

where $i = 1, \dots, M$, and $\mathcal{N}(\mu, \sigma)$ is a normal distribution with mean μ and standard deviation σ . It is important to note that the mean μ is angle-dependent, but the standard deviation σ is fixed and does not depend on the DoAs.

Figure 7.4 shows the estimated deviation error curves, determined by fitting the points (red dots) obtained by solving Equation 7.3 for different DoAs of our “ground truth”. Moreover, our error matrix $\mathbf{E}$ is obtained by substituting $\tilde{\alpha}_i(\theta)$ and $\tilde{\varphi}_i(\theta)$ from Equation 7.4 into α and φ of Equation 7.2.

The solid blue line of Figure 7.4 represents $\mu_{\tilde{\alpha}}(\theta)$ and $\mu_{\tilde{\varphi}}(\theta)$ of the normal distributions $\mathcal{N}(\mu_{\tilde{\alpha}}(\theta), \sigma_{\tilde{\alpha}})$ and $\mathcal{N}(\mu_{\tilde{\varphi}}(\theta), \sigma_{\tilde{\varphi}})$ respectively, while the dashed blue lines represent two times the standard deviation ($2\sigma_{\tilde{\alpha}}$ and $2\sigma_{\tilde{\varphi}}$). Although the mean (μ) of the normal distribution is angle-dependent (θ), we used a fixed standard deviation of $\sigma_{\tilde{\alpha}} = 0.6$ dB for the gain and $\sigma_{\tilde{\varphi}} = 0.4$ degrees for phase. The standard deviation values were found empirically by testing different models using various values. This is why the dashed blue lines always follow the shape of the solid blue line. The blue area in the figure represents where we can find 95% of the errors used during training.

After obtaining the gain and phase errors as an angle-dependent Gaussian distribution, we can update the error matrix $\mathbf{E}$ and obtain the snapshot equations used during the training of Deep-MLE model B.

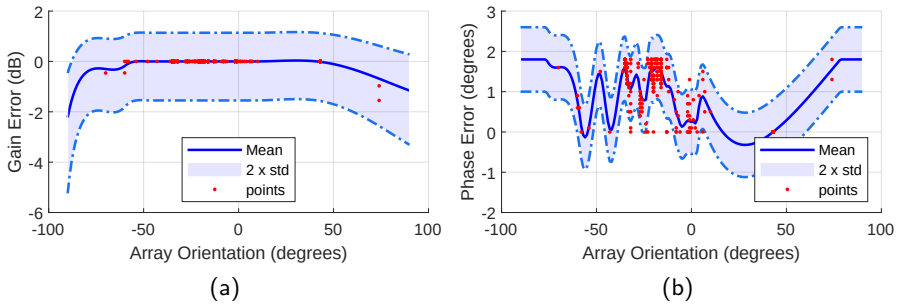


Figure 7.4: Estimated errors used for training Deep-MLE model B. (a) Gain, and (b) Phase errors. The red dots represent the estimated errors, the solid line is the obtained curve by fitting the dots and it represents the mean of a normal distribution for each angle. The dash-dotted line represents 2 times the standard deviation. 95% of the gain and phase errors are within the blue area.

7.5 Evaluation, and Results

In this section, we discuss three sets of experiments using two types of data after briefly discussing our metrics. In the first set of experiments, we evaluate the performance of our full-size Deep-MLE model A using synthetic data and antenna imperfections. The second set of experiments evaluates the performance of the reduced-size Deep-MLE model C using synthetic data and antenna imperfections. The third set of experiments evaluates the performance of the full-size Deep-MLE model B using measured data.

As previously discussed in [Chapter 5](#) and [Chapter 6](#), we evaluated our models using Root Mean Square Error (RMSE), and sensitivity. The sensitivity is defined here as the percentage of DoAs that were correctly estimated within 1 time the resolution or within 5 times the resolution.

7.5.1 Synthetic Data: Full-Size Neural Network

This first set of experiments was set to assess the performance of our updated version of the Deep-MLE of [Chapter 6](#) under the influence of antenna imperfections, using synthetic data with approximately known antenna imperfections ([Figure 7.3](#)) and a known number of sources. For this experiment, our Deep-MLE

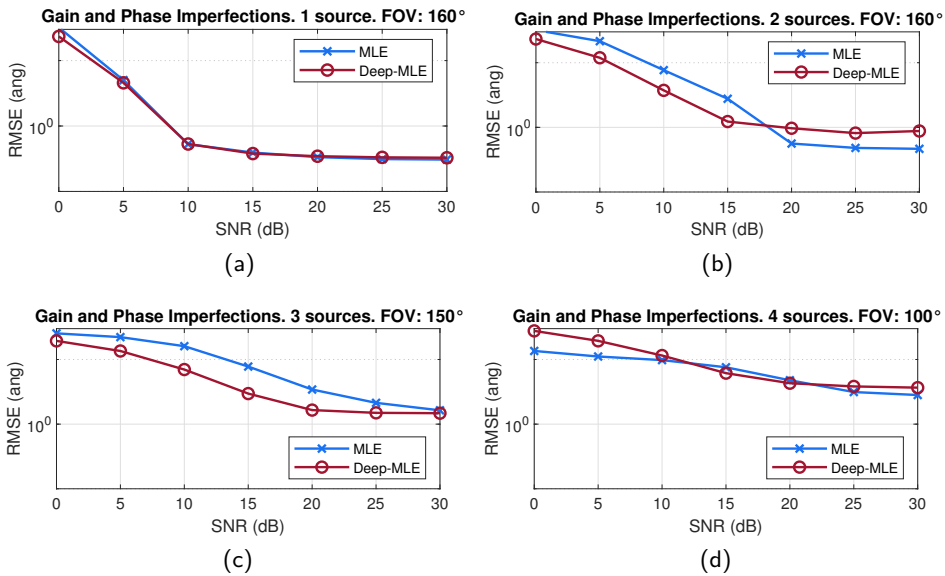


Figure 7.5: RMSE vs. SNR for DoA estimation using gain and phase errors. For this experiment, the number of sources was known and 1 to 4 sources were used.

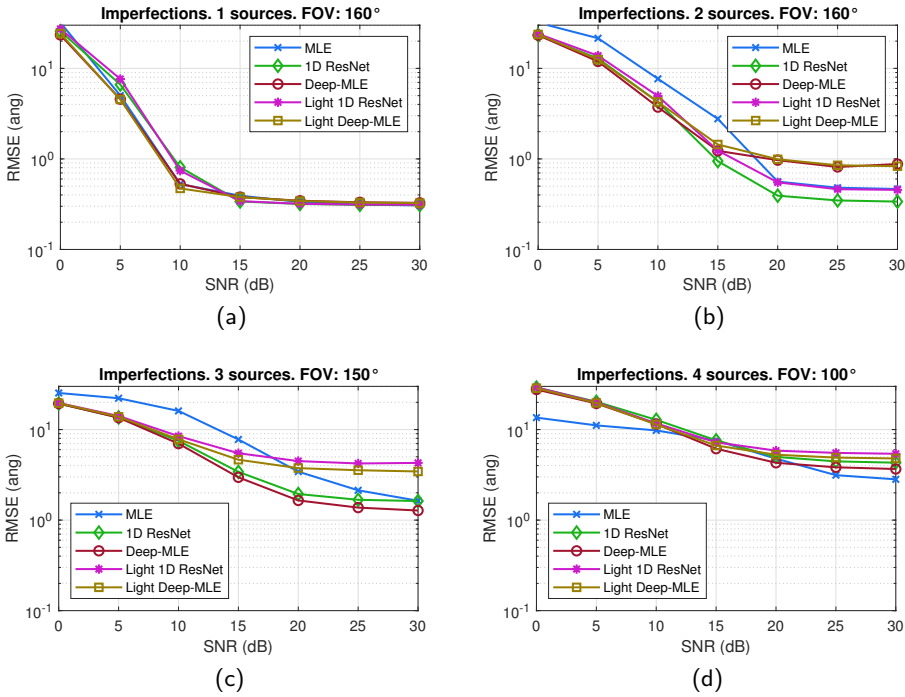


Figure 7.6: RMSE vs. SNR for DoA estimation using gain and phase errors and evaluating with various reduced-size and full-size models. For this experiment, the number of sources was known and 1 to 4 sources were used.

model A was trained with 40 million synthetically generated snapshots containing antenna imperfections, as described in Equation 7.1. This experiment did not consider the estimation of the number of sources.

The results, shown in Figure 7.5, demonstrate that our Deep-MLE can manage imperfections when trained and evaluated with synthetic data, and they are consistent with the results obtained using ideal antennas of Chapter 5 and Chapter 6. In other words, when using synthetic data, our Deep-MLE can learn that the gain and phase error curves are angle-dependent and compensate accordingly.

7.5.2 Synthetic Data: Reduced-Size Neural Network

For the second set of experiments, we evaluated how a reduced-size model could handle gain and phase errors. For this, we have done a similar experiment to the first one. Here, we compared full-size models with reduced-size models.

The results of this experiment are shown in Figure 7.6, where the 1D ResNet represents the full-size ResNet-based DoA estimator with a post-processing algo-

rithm as previously discussed in [Chapter 5](#), the Deep-MLE represents the full-size Deep-MLE model A, the light 1D ResNet represents the reduced-size ResNet-based DoA estimator, and the light Deep-MLE represents the reduced-size Deep-MLE model C.

As shown in [Figure 7.6](#), although the reduced-size Deep-MLE model requires over 15 times less storage than the full-size Deep-MLE model A, it performs similarly to its full-size counterpart for one and two targets, and a bit worse for three and four targets.

7.5.3 Measured Data: Full-Size Neural Network

The third and last set of experiments involved the utilization of an FMCW automotive radar equipped with an MRA with 8 antennas. Two distinct datasets were created from the collected radar data - one for fine-tuning (2759 snapshots) and the other for evaluation purposes (29741 snapshots). Our so-called ground truth was obtained from a partner company that used its own techniques to estimate DoAs and the number of sources. This means that this ground truth may contain errors due to the lack of a universally perfect method for estimating DoAs and the number of sources.

As the size of our measured dataset (2759 snapshots) was insufficient to train a deep learning model fully, we trained it with synthetic data and fine-tuned it with measured data. As previously discussed in [Section 2.6](#), fine-tuning is a transfer learning technique often employed for small datasets. It involves reducing the learning rate to preserve parts of the previously learned features.

During training with synthetic data, we utilized 40 million generated points, containing 0 to 4 targets, an SNR varying from 0 to 40 dB, and the deviation errors from [Figure 7.4](#) as the angle-dependent imperfections. During the fine-tuning process, we utilized 2759 measured snapshots, 770 augmented snapshots, and 22070 synthetically generated snapshots. In addition, we have used k-fold cross-validation [[Ber18](#)] during the fine-tuning process.

In this experiment, both the DoAs and the number of targets are unknown and need to be estimated. As previously discussed in [Chapter 5](#), the Maximum Likelihood Estimation (MLE) algorithm can also be used to estimate the number of sources through an iterative process.

For the evaluation, we used a dataset that was collected within a parking lot close to a hill, as shown in [Figure 7.7](#) (a). During the recording, a cyclist passed by two corner reflectors positioned at the same distance from the radar but at different angles. This dataset consists of 470 frames with 29741 snapshots containing one target and 2978 snapshots containing two targets.

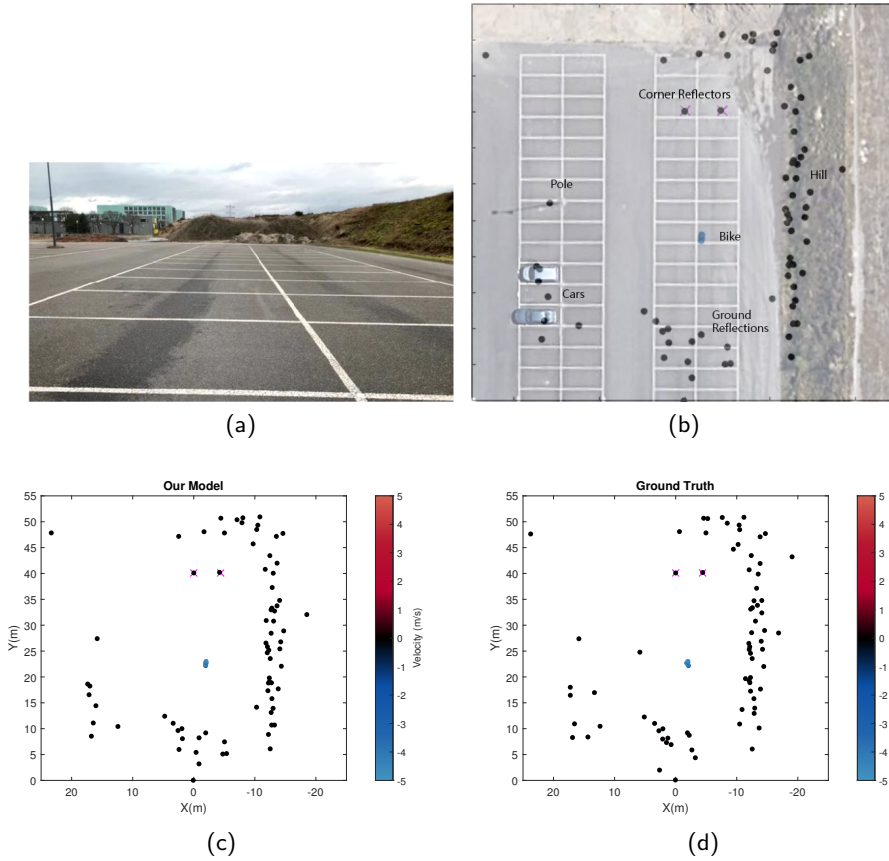


Figure 7.7: Visual inspection of the evaluation data: a parking lot close to a hill. Here, a moving bicycle passes through two stationary corner reflectors. (a) The location of where the evaluation data was recorded: a parking lot close to a hill, (b) an overlaying of a single frame of our results at estimating the DoAs and the number of sources with a Google Earth image of the data gathering location, (c) a single frame with its respective angles, distance, and speed using our Deep-MLE. (d) A single frame of our ground truth that uses different estimation methods.

Table 7.1: Results using measured data for 1 DoA.

Type	RMSE	Sensitivity within 1°	Sensitivity within 5°	Percentage of Sources Estimated Correctly
MLE	14.9°	0.703	0.871	40.1 %
Deep-MLE	15.2°	0.702	0.861	76.0 %

Table 7.2: Results using measured data for 2 DoAs.

Type	RMSE	Sensitivity within 1°	Sensitivity within 5°	Percentage of Sources Estimated Correctly
MLE	1.6°	0.665	0.987	74.1 %
Deep-MLE	2.0°	0.679	0.986	75.2 %

The qualitative results are shown in [Figure 7.7](#) (animation^{1,2}). They show that our Deep-MLE model B can correctly estimate the angles for various objects, including corner reflectors, cars, a light pole, and even the shape of the hill. Furthermore, when comparing the frames of our technique with the ground truth, we can see how similar they are.

The quantitative results are shown in [Table 7.1](#) and [Table 7.2](#). They show that our Deep-MLE model B performs comparably to the MLE algorithm when estimating DoAs, and it is significantly more accurate than the MLE algorithm when estimating the number of sources.

It is important to note that the high RMSE values of [Table 7.1](#) should not be seen as indicative of poor performance. Rather, these discrepancies arise from how our Deep-MLE model estimates nearby ground reflections differently than the provided ground truth.

A limitation of our method to mitigate antenna imperfections is having considered a simplified version of the error matrix. In reality, mutual coupling, time, and temperature are also part of the error matrix.

7.6 Conclusion

In this chapter, we improve our fusion model called Deep-MLE by incorporating antenna imperfections in the snapshot model when estimating DoAs and the number of sources.

¹ <https://mllimadeoliveira.github.io/Thesis/RFusion.html> An animation containing the full output from our Deep-MLE model using measured data with Google Earth location as the background.

² <https://mllimadeoliveira.github.io/Thesis/Comparison.html> An accelerated animation containing the comparison of our Deep-MLE's output and the used ground truth.

This updated version of Deep-MLE can learn antenna gain and phase errors based on the direction of arrival, rather than using a single compensation value for all angles.

Our experiments showed that our Deep-MLE models efficiently handle synthetic and measured snapshots. Unlike many data-driven DoA estimation techniques that lack experimental validation, our Deep-MLE performs closely to the MLE algorithm, at the same time that a visual inspection confirms the presence of the same crucial elements (bike, reflectors, hill, and so on) in our results and in the ground truth.

Conclusion

ABSTRACT

This thesis has introduced deep-learning techniques for improving object detection and DoA estimation using FMCW automotive radars. In this final chapter, we revisit the core concepts and contributions that were discussed in this thesis. We end this chapter by giving suggestions for possible future works.

In conclusion, impulsed by the new trends in the automotive industry, where vehicles are evolving into sophisticated electronic-mechanical hybrid machines equipped with hundreds of sensors, the needs within the radar signal processing field have changed to fit the recent demands for low-cost, small-size, accuracy, safety, latency, and so on. For that, the automotive radar signal processing field started to depart from handcrafted techniques designed to meet specific needs, shifting from purely analytical systems to hybrid ones by integrating deep learning models into its processing pipeline.

Despite the well-understood mathematics and physics of the traditional radar signal processing pipelines, non-linearities, non-Gaussian noise, and the lack of semantic information are some of the reasons to expect that there is room for improvement, and deep learning might bring the needed edge. However, reducing some of the gaps between both fields comes with its own challenges. As part of this initial movement, in this thesis, we have investigated some deep-learning applications for improving the automotive radar pipeline.

Throughout this thesis, our main goal was to design machine learning techniques that could be integrated into the classical analytical signal processing pipeline, offering alternative solutions to improve the performance of FMCW automotive radar systems. More precisely, we focused on improving object detec-

tion and DoA estimation in the traditional FMCW radar pipeline by incorporating data-driven elements.

In this thesis, we have presented some deep-learning applications possibilities for radar systems through the usage of Convolutional Autoencoders (CAEs), Generative Adversarial Networks (GANs), and Residual Neural Networks (ResNets) for various detection and estimation applications within the radar pipeline.

More precisely, we have started our investigation by discussing the usage of a computer-vision machine-learning technique to reduce noise and detect objects in range-Doppler maps. In the sequence, to deal with problems in annotated data, we created a machine-learning solution to generate labeled synthetic range-Doppler maps. At this point, we shifted our focus to MIMO radars, discussing the integration of data-driven techniques to estimate the direction of arrival (angles) and the number of targets in the scene. Furthermore, we discussed the usage of a fusion model that utilizes deep learning and an analytical algorithm to improve the DoA estimation for FMCW radars. Finally, we evaluated the performance of our DoA estimation models with imperfect antennas and measured data.

As a retrospective of the work we developed through this thesis, we can state that we successfully introduced models that filled some of the gaps between classic signal processing and deep learning by integrating data-driven methods into the traditional radar pipeline.

However, it is also important to address the shortcomings of data-driven techniques. While analytical signal processing techniques often introduce simplifications in their models that may not hold in practice, we noted through this thesis that data-driven methods encounter similar challenges. These techniques depend heavily on dataset quality. Without sufficient diversity and representation in the data, models developed via deep learning techniques might oversimplify complex real-world phenomena.

For the rest of this chapter, we provide a summary of this thesis and main findings in [Section 8.1](#), the delineation of the key contributions in [Section 8.2](#), and the prospects for future work in [Section 8.3](#).

8.1 Summary of the Thesis

In this section, we summarize the main ideas that were presented in each chapter of the thesis. In our opening chapter ([Chapter 1](#)), we introduced the topic to a broader audience and gradually narrowed it down from a wider perspective to our specific focus. The initial chapter serves as a gateway, providing context and scope for further discussions in the subsequent chapters.

In [Chapter 2](#), we first presented a detailed retrospective into the world of automotive FMCW MIMO radars, revealing the basic principles and laying out

the essential math behind the physical phenomenon of transmitting and receiving radar waves. After presenting the main concepts of automotive radar, we discussed the importance of detectors and estimators. Here we presented the main analytical detector and estimator algorithms that were later used as baseline. Subsequently, we presented the necessary background about deep learning and neural networks, including some of the later used architectures, activation functions, and loss functions. This is particularly important as our work uses deep learning as a starting point for developing data-driven detectors and estimators. At the end of our discussions about deep learning, we presented some advanced topics. Finally, we closed the chapter by briefly presenting the main metrics we used throughout this thesis to evaluate our proposed models.

Moreover, after presenting the fundamentals of automotive radar signal processing, analytical models, and deep learning, we discussed our first work in [Chapter 3](#). Here, we successfully demonstrated the efficacy of Convolutional AutoEncoder (CAE) models in mitigating noise and enhancing object detection in range-Doppler maps. For that, we have developed four distinct CAE systems. Two of them took advantage of the time-series nature of subsequential range-Doppler maps, and two of them took advantage of parallel neural networks, using a voting system to ensure that the correct objects were being reconstructed. The combination of parallel networks, time series, and a foreground detector appears to be a promising solution for detecting moving objects on RD maps.

One of the challenges we faced during the development of our CAE systems was the lack of available labeled data ready for use. While we had mathematically generated synthetic test data, it did not capture the essence of measured range-Doppler maps. To address this, in [Chapter 4](#), we presented a novel approach for generating synthetic RD maps that could mimic actual maps containing pedestrians and bicycles. This method showed the potential of mitigating issues related to small, incomplete, and unlabeled datasets. Our Generator-Predictor system not only produced new data but also predicted sequential future frames for better training of other NN-based approaches. For that, our proposed architecture included a Generative Adversarial Network (GAN) generator that was capable of producing new synthetic RD maps, and a deep convolutional autoencoder predictor to predict future frames containing the movement of such objects.

While range-Doppler maps and their applications are a key research point in the radar domain, determining the Direction of Arrival (DoA) using MIMO radars is equally important. After detecting an object in an RD map, the next step in the radar pipeline is to determine the number of targets and their angles (direction of arrival). In [Chapter 5](#), [Chapter 6](#), and [Chapter 7](#) we introduced effective approaches to estimate DoAs and the number of targets based on Residual Neural Networks (ResNet) architectures.

In our first work on estimating DoAs and the number of targets using a data-driven approach (in [Chapter 5](#)), our results showed that our ResNet-based estimator could sometimes outperform an analytical estimator based on the Maximum Likelihood Estimation (MLE) algorithm, especially in noisy scenarios.

While our ResNet-based estimator showed good results for estimating DoAs, we found out that we could improve our results by combining our data-driven model with an analytical algorithm and creating a fusion model. MLE is a well-understood algorithm with reliable performance, but it is computationally heavy for many applications. To mitigate this, in [Chapter 6](#), we developed a model for DoA estimation using a single snapshot, called Deep-MLE, that integrates a 1D ResNet-based neural network and the MLE algorithm in order to maximize their advantages and minimize their weaknesses. In other words, Deep-MLE (of [Chapter 6](#)) had better results than our previous ResNets-based NN (of [Chapter 5](#)), while maintaining a lower computational complexity than the traditional usage of the MLE algorithm.

However, although our Deep-MLE model showed a good performance estimating DoAs with synthetic data, outperforming (on average) other models and algorithms, it still required an evaluation with measured data to demonstrate its effectiveness in real-world scenarios, where realistic antennas have imperfections and disturbances beyond just Gaussian noise. To address this, in [Chapter 7](#), we utilized the knowledge obtained in our previous works and explored the usage of imperfect antennas, evaluating our updated version of Deep-MLE with synthetic and measured data. Our results showed that our model could learn the angle dependency of the imperfections, indicating that it could be a viable solution for real-world applications.

8.2 Contributions and Results

Our main contributions and results showcased in this thesis are listed as follows:

1. **Using CAE to denoise and detect objects in range-Doppler maps.**

The first contribution of this thesis involves enhancing object detection in range-Doppler (RD) maps using deep convolutional autoencoders. As a result, our CAE-based solution consistently outperformed the traditional Constant False Alarm Rate (CFAR) detection method. For example, when evaluating with data of a moving FMCW radar, our selected CAE-based system showed an improvement from 52.5% (CFAR) to 86.7% (3 CAEs - 3 frames) for mean sensitivity. For mean specificity, the improvement was from 54.1% (CFAR) to 79.3% (3 CAEs - 3 frames). In conclusion, for all the tested scenarios, our CAE-based detectors outperformed the CFAR-based detection method.

2. **Generating synthetic RD maps using GAN and CAE.** The second contribution of this thesis involves the construction of a data-driven model for generating synthetic range-Doppler maps. Our Generator-Predictor utilizes the GAN's generator model as a generator and a CAE as a predictor. Our results showed that training a NN-based detection model exclusively on the synthetic RD maps created by our Generator-Predictor and evaluating it with measured data resulted in a detection sensitivity of 95.2% while training a NN-based detection model exclusively on 1000 measured frames resulted in a detection sensitivity of 94.0%, for example. This is 1.2% better than when compared to a detector trained solely using measured data. Depending on the size of the measured dataset, the improvement could be up to 9.2%. In contrast, our synthetic data showed no further improvement when compared to a full FMCW dataset with augmented data. In other words, our results indicate that using our data-driven generated synthetic data can be better than using small or scarce datasets.
3. **Single-snapshot DoA estimation with ResNet.** The third contribution of this thesis involves using a data-driven model to estimate the Direction of Arrivals (DoAs) for a single snapshot using a deep learning model based on Residual Neural Networks (ResNets). Our results using algorithmically generated data indicate that our proposed ResNet-based DoA estimator outperforms MLE, particularly in scenarios with high quantities of noise (SNR below 15 dB), throughout the various configurations we have tested. For example, for two targets (with a known number of sources) our data-driven estimator achieved a minimum RMSE of 0.311° while the MLE algorithm achieved a minimum RMSE of 0.353° . On the other hand, for 3 targets our data-driven estimator achieved an RMSE of 1.930° when the SNR was 20 dB and 1.533° when the SNR was 30 dB while the MLE algorithm achieved an RMSE of 2.295° when the SNR was 20 dB and 1.504° when the SNR was 30 dB. In conclusion, we can state that for the tested scenario with synthetic data, the performance of our data-driven DoA estimator was at least as good as the performance of the MLE algorithm, surpassing it in many scenarios.
4. **A fusion model using 1D ResNet and MLE for DoA estimation.** The fourth contribution of this thesis involves introducing a fusion model, called Deep-MLE, that uses a data-driven approach (ResNet) and an analytical (MLE) algorithm for better estimating DoAs. Our results using synthetically generated data show that our model can, on average, outperform both MLE and 1D ResNet for both high and low-noise scenarios. For example, when comparing Deep-MLE and the MLE algorithm, the average estimation error in high-noise scenarios is lower for Deep-MLE by 3.4° , while in

low-noise scenarios, the difference is smaller at 0.06° , with Deep-MLE still performing better. At the same time, Deep-MLE can have a computational complexity of up to ten thousand times smaller than MLE when estimating four DoAs. For example, when estimating 4 DoAs, an MLE algorithm would have to process 2.2×10^{10} operations, while Deep-MLE would have to process 1.72×10^6 operations. In short, our model can outperform other models (for the tested scenarios), while maintaining a lower computational complexity than the MLE algorithm.

5. **Antenna imperfection mitigation for DoA estimation with real data.**

The fifth contribution of this thesis involves investigating the effects of antenna imperfections in our proposed fusion models. We tested our models with both synthetic and measured datasets. The results using synthetic data containing imperfections (gain and phase) aligned with our previous works that utilized ideal antennas as the RMSE vs. SNR curves were quite similar. Before testing our model with measured data, we first fine-tuned the neural network accordingly with some measured data. The results using measured data demonstrate that its performance was similar to MLE in estimating DoAs, showing that our model works in a real scenario. For example, when estimating the DoA of a single target, MLE had a sensitivity (within 1 degree) of 0.703 while our model had a sensitivity of 0.702. For two targets, MLE had a sensitivity (within 1 degree) of 0.665 while our model had a sensitivity of 0.679.

6. **Number of targets estimation using a single snapshot.** The final contribution of this thesis involves introducing NN-based methods for estimating the number of targets utilizing the main architecture of the deep learning model (mainly ResNet in our case). Estimating the number of targets present in a single snapshot can be computationally expensive for algorithms like MLE, as each iteration increases the computational load. Our results, using both synthetic and measured data, show that our data-driven models can outperform MLE when estimating the number of targets, while also requiring less computational power. When testing with synthetic data. For example, when evaluating our model with measured data and a single target, MLE correctly estimated the number of targets 40.1% of the time, while our model correctly estimated it 76.0%. For measured data and two targets, MLE correctly estimated the number of targets 74.1% of the time, while our model correctly estimated it 75.2%.

8.3 Directions for Future Works

The concepts and algorithms formulated in this work can potentially be further extended and improved through the following suggestions:

- **Detecting moving objects in range-Doppler maps.** The denoising and detection of range-Doppler maps using CAEs present many attractive research opportunities that could be further investigated. By integrating a tracking neural network into our existing architectures and utilizing the already extracted features of its encoder, one could combine detection and tracking seamlessly, for example, by adding an architecture, such as YOLO [Red+16b], into their model. Another possibility lies in the better utilization of CFAR. This includes the development of an NN that could actively fine-tune its hyperparameters to find the most effective cost-benefit configuration to detect objects using RD maps. Additionally, one could investigate other architectures that integrate Convolutional AutoEncoders (CAEs) and CFAR through a voting mechanism that might give better object detection.
- **Generating synthetic range-Doppler maps.** As one of the first on the topic, our study using NNs to generate synthetic range-Doppler maps opened several research opportunities. Exploring various Generative Adversarial Network (GAN) architectures, such as conditional GANs [Wan+18], could boost the quality and versatility of the generated synthetic data. In addition, integrating recurrent neural networks, long short-term memory neural networks [HS97], or 3D convolutional neural networks [HI22] into the predictor could offer an improvement for predictions of future frames. It is also important to research how a detection system trained with synthetic data would “react” to new and unfamiliar situations. Moreover, one could expand on our research by taking into account the real and imaginary parts of range-Doppler maps instead of their normalized power magnitudes.
- **Estimating DoAs with neural networks.** The DoA estimation field could potentially benefit from the exploration of more sophisticated neural network architectures. Future works could utilize complex-valued neural networks [LHG22], incorporating weights with complex numbers instead of real numbers, as the snapshots are complex, and our architecture had to convert them into real numbers. By directly processing complex-valued data, such a model could, perhaps, better preserve phase information and inherent signal structures that may lead to a more accurate DoAs estimation. Another possible direction would be to go into self-supervised or unsupervised learning techniques, allowing the model to gain insights from unlabeled or minimally labeled data.

- **Integrating NN and analytical models for DoA estimation.** Building upon the insights of our Deep-MLE fusion model, there are several possible ways to further integrate data-driven and analytical approaches. Future investigations could find alternative and computationally lighter algorithms for the last estimation step inside a fusion model. Another possible change in the fusion model lies in the neural network itself. Perhaps instead of using a ResNet model, a deep unfolding model [You+22] could be used, where each iteration of an analytical algorithm would be incorporated as a neural network layer. Furthermore, one could design a loss function that mimics the MLE optimization process, making the deep learning model work more similarly to the MLE algorithm.
- **Antenna imperfection mitigation with deep learning.** Regarding antenna imperfections, a possible future investigation lies in the development of a new separate deep learning model that pre-processes real data for other data-driven or analytical models by learning the effects of imperfect antennas. This new NN would serve as a previous step and could be applied to various existing techniques.
- **Data quality.** As data quality seems to be one of the most important factors (if not the most important factor) for a good neural network model [Li+19], we also suggest future investigations to look into how to generate or gather better quality and more diverse training and evaluating datasets.
- **Unified pipeline.** Our final suggestion is to create a unified library that incorporates all the models explored in this thesis into a single pipeline. This library could, for example, receive a radar data cube, use our proposed CAEs to denoise and detect moving objects in range-Doppler maps, and estimate the number of targets and their angles using our proposed Deep-MLE.

8.3.1 Final Considerations

The work presented in this thesis englobes new and traditional ideas for improving the performance of automotive FMCW radars, filling some gaps between data-driven and classical analytical approaches. Nevertheless, there are many stones left unturned in the search for novel solutions for automotive radars. As such, we hope that this work will inspire other researchers to continue this quest and stimulate further development of innovative tools that will eventually find their way into the industry, unleashing a new era of radar technology advancement.

Abbreviations

AI	Artificial Intelligence	1
ADAS	Advanced Driver Assistance Systems	28
FFT	Fast Fourier Transform	9
MIMO	Multiple-Input and Multiple-Output	9
Tx	Transmitter	28
Rx	Receiver	28
mmWave	Millimeter Wave	28
FMCW	Frequency Modulated Continuous Wave	29
ADC	Analog-to-Digital Converter	33
RD	Range-Doppler	33
DoA	Direction of Arrival	35
ULA	Uniform Linear Array	37
MRA	Minimum Redundancy Array	37
CFAR	Constant False-Alarm Rate	43
CUT	Cell Under Test	44
CA-CFAR	Cell-Averaging CFAR	44
OS-CFAR	Ordered-Statistic CFAR	44
MUSIC	MULTiple Signal Classification	47
ESPRIT	Estimation of Signal Parameters via Rotational Invariance Techniques	48
MLE	Maximum Likelihood Estimation	51
DML	Deterministic Maximum Likelihood	51
FOV	Field Of View	52
TP	True Positive	74
FN	False Negative	74
FP	False Positive	74
TN	True Negative	74
SNR	Signal-to-Noise Ratio	75

Abbreviations

PSNR	Peak Signal-to-Noise Ratio	75
RMSE	Root Mean Square Error	77
DL	Deep Learning	56
ML	Machine Learning	56
NN	Neural Network	56
MSE	Mean Square Error	60
BCE	Binary Cross-Entropy	60
SGD	Stochastic Gradient Descent	61
RMSProp	Root Mean Squared Propagation	61
Adam	Adaptive moment estimation	61
ReLU	Rectified Linear Unit	62
FCNN	Fully Connected Neural Network	64
MLP	Multi Layer Perceptron	64
FC	Fully Connected	64
FFNN	Feed Forward Neural Network	64
CNN	Convolutional Neural Network	65
CAE	Convolutional AutoEncoder	67
ResNet	Residual neural Network	68
GAN	Generative Adversarial Network	69

List of Symbols

τ	Time delay between a transmitted and received signal	30
t	Time	30
s_T	Transmitted radar signal	31
s_R	Received radar signal	31
f	Varying frequency for FMCW radars	31
ω	Varying angular frequency	31
Φ_τ	Phase shift due to the time delay τ	31
f_s	Carrier frequency or source frequency.	31
b	Beat signal	31
ω_b	Beat angular frequency	31
f_b	Beat frequency	31
T	Period of one chirp	31
B	Bandwidth of one chirp	31
r	Distance between the object and the radar	31
c	Speed of light	31
ω_d	Shift in angular frequency due to Doppler	32
f_d	Shift in frequency caused by the Doppler effect	32
v	Relative radial velocity between the moving object and the radar	32
z	Generic input for functions and algorithms	33
M	Number of antennas within a MIMO radar	36
$\mathbf{x}$	A snapshot for a single source	36
$\mathbf{a}$	Steering vector	36
θ	Direction of Arrival (DoA)	36
$\mathbf{s}$	Signal emitted by a source	36
$\mathbf{D}$	Geometry of the linear antenna array.	36
d	Minimum distance between two or more antennas.	36
λ	Wavelength of the transmitting signal	36

List of Symbols

N	Number of sources	36
$\mathbf{A}$	Steering Matrix	36
s	Vector of signal waveforms	36
η	Gaussian noise	37
x	A MIMO radar snapshot for one or multiple sources ..	37
p	A vector containing the element positions of a MRA.	39
P	Power	44
n	Generic Noise	44
ψ	Threshold compensation variable for CFAR	44
β	Threshold offset for CFAR	44
$\mathcal{T}$	Generic threshold	44
K	Number of samples	44
$\mathbf{R}$	Covariance matrix	46
E	Statistical Expectation	46
$\mathbf{P}$	Source's covariance matrix	46
$\mathbf{I}$	Identity matrix	46
σ_{η}^2	Noise's variance	46
$\mathbf{U}$	Unitary matrix	46
$\mathbf{\Lambda}$	Diagonal matrix of real eigenvalues	46
$\tilde{\mathbf{a}}$	Estimated steering vector.	47
p	Music spectrum	48
$\tilde{\theta}$	Estimated DoA	49
ε	Residual error	51
G	Search Grid	53
$\Delta\theta$	Angular resolution	53
Θ	Number of times a search is required for an algorithm	53
σ_{η}	Noise's standard deviation	54
T	Transformation	57
y	Generic vector or value containing true values	60
$\tilde{y}$	Generic vector or value containing predicted values ...	61
$\mathcal{L}$	Loss Function	61
w	Weight vector	61
lr	Learning rate	62
g	Activation Function	62
f	Generic NN function	65
l	A given layer within a NN model	65

W	Weight Matrix	65
b	Bias Vector	65
Z	Input matrix	66
K	CNN kernel matrix	66
k	1D CNN kernel vector	66
h	Latent Representation	68
F	Feature Map	69
$\mathcal{G}$	Generator data sample	71
$\mathcal{D}$	Discriminator probability	71
L	Number of Monte-Carlos trials	77
$\tilde{N}$	Estimated number of sources	111
X	Input of the 2D ResNet	112
c_0	Generic constant	112
v	The 1D ResNet's confidence distribution.	114
$\hat{\theta}$	Most probable angles of the distribution v	115
A	Amplitude of a signal	127
C	Number of filters (or channels) in a convolutional layer	128
Kf	Kernel filter size of a convolutional layer	128
S	Number os strides of a convolutional layer	128
I	Input or output size of a NN's layer	128
χ	Pre-processed snapshot x containing the real and imaginary parts as a single vector	130
α	Gain (or amplitude) error	139
φ	Phase error	139
E	Error matrix	139
$\mu_{\tilde{\alpha}}$	Mean gain error of a normal distribution of estimated gain errors.	143
$\mu_{\tilde{\varphi}}$	Mean phase error of a normal distribution of estimated phase errors.	143
$\sigma_{\tilde{\alpha}}$	Standard deviation of a normal distribution of estimated gain errors.	144
$\sigma_{\tilde{\varphi}}$	Standard deviation of a normal distribution of estimated phase errors.	144

Notation

Sets and spaces

$\mathbb{R}$	Real numbers.
$\mathbb{R}^n$	Space of n -dimensional vectors with real components ($n \times 1$ matrices).
$\mathbb{R}^{m \times n}$	Space of real matrices with m rows and n columns.
$\mathbb{Z}$	Integers.
$\mathbb{C}$	Complex numbers.
$\mathbb{C}^{m \times n}$	Space of complex matrices with m rows and n columns.

Vectors and matrices

v_i	The i -th element of a vector v .
$A_{i,j}$	The element that belongs to the i -th row and j -th column of the matrix A .
I	$n \times n$ identity matrix. Sometimes its size is implicitly defined and can be understood from the context.
x^*	Complex conjugate of vector x .
$A^\top$; $x^\top$	Transpose of matrix A ; transpose of vector x .
A^H ; x^H	Hermitian conjugate of matrix A ; hermitian conjugate of vector x . Hermitian conjugate is the complex conjugate transpose.
A^{-1}	Inverse operation of matrix A .
A^+	Pseudo-inverse (also called Moore–Penrose inverse) operation of matrix A .
$\text{diag}\{x\}$	Diagonal matrix with diagonal entries $x_1, \dots, x_n$.
$\text{tr}\{A\}$	Trace of matrix A . The sum of its diagonal entries.
$\det\{A\}$	Determinant of matrix A .
$\text{abs}(A)$; $\text{abs}(v)$	Matrix with entries $[\text{abs}(A)]_{ij} = [A]_{ij} $; vector with components $[\text{abs}(v)]_i = v_i $. In other words, entrywise absolute value function.

Notation

$\|A\| ; \|x\|$ 2-norm (ℓ_2 -norm) of matrix A ; 2-norm of vector x .

Complex numbers

j Imaginary unity.
 $a + jb$ Complex number with a real part equal to a and imaginary part equal to b .
 $\text{real}\{a\}$ Real value of the complex number a .
 $\text{imag}\{a\}$ Imaginary value of the complex number a .
 ae^{jb} Euler representation of a complex number, where a is its amplitude and b its phase. It is the same as $a(\cos b + j \sin b)$

Functions and relations

$Y = f(X)$ f is a function mapping from X to Y .
 $\max\{v\}$ Maximum value of the vector v .
 $\min\{v\}$ Minimum value of the vector v .
 $\text{argmin}\{v\}$ The minimum argument of the vector v .

Others

$\mathcal{N}(\mu, \sigma)$ Normal (Gaussian) distribution with mean μ and standard deviation σ .
 $E\{.\}$ Expected value.
 $O(.)$ Operational complexity

Bibliography

- [Abo+17] Elias Aboutanios et al. “Fast Iterative Interpolated Beamforming for Accurate Single-Snapshot DOA Estimation”. In: *IEEE Geoscience and Remote Sensing Letters* 14.4 (2017), pp. 574–578 (cit. on p. 126).
- [Agg18] Charu C. Aggarwal. *Neural Networks and Deep Learning*. Springer, 2018 (cit. on pp. 8, 60, 62, 63, 67, 68, 71, 72).
- [AH23] Michael Adam and Clotilde Hocquard. *Artificial intelligence, democracy and elections*. 2023 (cit. on p. 1).
- [All+19] Stephen Alland et al. “Interference in Automotive Radar Systems”. In: *IEEE Signal Processing Magazine* 36.5 (2019), pp. 45–59 (cit. on pp. 4, 5).
- [Alo+18] Zahangir Alom et al. *The History Began from AlexNet : A Comprehensive Survey on Deep Learning Approaches*. 2018 (cit. on p. 3).
- [AMB21] Alankrita Aggarwal, Mamta Mittal, and Gopi Battineni. “Generative adversarial network: An overview of theory and applications”. In: *International Journal of Information Management Data Insights* 1.1 (2021) (cit. on p. 99).
- [AMK19] Arian Azarang, Hafez E. Manoochehri, and Nasser Kehtarnavaz. “Convolutional Autoencoder-Based Multispectral Image Fusion”. In: *IEEE Access* 7 (2019), pp. 35673–35683 (cit. on p. 80).
- [AN14] G V Anand and P V Nagesha. “Source Number Estimation in Non-Gaussian Noise”. In: *European Signal Processing Conference (EU-SIPCO)* 0 (2014), pp. 1711–1715 (cit. on p. 14).
- [ATN00] Khalid Al Midfa, George V. Tsoulos, and Andy Nix. “Performance Evaluation of Direction-Of-Arrival (DOA) Estimation Algorithms for Mobile COMMunication Systems”. In: *Vehicular Technology Conference* (2000), pp. 1055–1059 (cit. on pp. 14, 15).

- [Bas19] Emre Baseski. "16-Bit to 8-Bit Conversion in Remote Sensing Images by Using Image Content". In: *International Conference on Recent Advances in Space Technologies (RAST)* (2019) (cit. on pp. 34, 82).
- [Ben+18] Franja Z Benčić et al. "Trends in AI based Automotive Industry using Patent Analysis". In: *26th Telecommunications forum TELFOR*. 2018, pp. 1–4 (cit. on pp. 1, 2).
- [Ber18] Daniel Berrar. "Cross-validation". In: *Encyclopedia of Bioinformatics and Computational Biology: ABC of Bioinformatics* 1-3 (2018), pp. 542–545. arXiv: 1703.03167 (cit. on p. 147).
- [BGT19] Oded Bialer, Noa Garnett, and Tom Tirer. "Performance Advantages of Deep Neural Networks for Angle of Arrival Estimation". In: *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*. Vol. 2019-May. 2019, pp. 3907–3911. arXiv: 1902.03569 (cit. on pp. 7, 16, 108, 113, 117, 122).
- [Bis06] Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006 (cit. on pp. 7, 56, 59).
- [Bla88] Stephen Blake. "OS-CFAR Theory for Multiple Targets and Nonuniform Clutter". In: *IEEE Trans. Aerosp. Electron. Syst* 24.6 (1988), pp. 785–790 (cit. on pp. 14, 44, 45).
- [Bro05] Graham M Brooker. "Understanding Millimetre Wave FMCW Radars". In: *1st International Conference on Sensing Technology* January 2005 (2005), pp. 152–157 (cit. on p. 3).
- [Bro+20] Tom B Brown et al. *Language Models are Few-Shot Learners*. 2020 (cit. on p. 1).
- [Bur19] Andriy Burkov. *The Hundred-Page Machine Learning Book*. Vol. 5. 2. 2019 (cit. on pp. 56, 58–63, 65, 67, 71–73).
- [Cae+20] Holger Caesar et al. "nuScenes : A multimodal dataset for autonomous driving". In: *arXiv* (2020). arXiv: arXiv:1903.11027v5 (cit. on p. 6).
- [Cao+20] Yashuai Cao et al. "Complex ResNet Aided DoA Estimation for Near-Field MIMO Systems". In: *IEEE Transactions on Vehicular Technology* 69.10 (2020), pp. 11139–11151. arXiv: 2007.10590 (cit. on p. 7).

- [CD14] T. Chai and R. R. Draxler. “Root mean square error (RMSE) or mean absolute error (MAE)? -Arguments against avoiding RMSE in the literature”. In: *Geoscientific Model Development* 7.3 (2014), pp. 1247–1250 (cit. on p. 77).
- [Cou08] Pierre Courrieu. “Fast Computation of Moore-Penrose Inverse Matrices”. In: *Neural Information Processing - Letters and Reviews* 8.2 (2008), pp. 25–29. arXiv: 0804.4809 (cit. on p. 128).
- [CPK22] Hyeonjin Chung, Hyunwoo Park, and Sunwoo Kim. “Leveraging Deep Learning for Practical DoA Estimation: Experiments with Real Data Collected via USRP”. In: *Sensors (MDPI)* (2022) (cit. on pp. 7, 20, 138, 142).
- [CS84] P. Cappello and K. Steiglitz. “Some Complexity Issues in Digital Signal Processing”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 32.5 (1984), pp. 1037–1041 (cit. on p. 10).
- [CS96] Boon Chong and Chong Meng Samson See. “Sensor-Array Calibration Using a Maximum-Likelihood Approach”. In: *IEEE Transactions on Antennas and Propagation* 44.6 (1996) (cit. on pp. 16, 139).
- [CSC16] Ushemadzoro Chipengo, Arien P Sligar, and Shawn Carpenter. “High Fidelity Physics Simulation of 128 Channel MIMO Sensor for 77GHz Automotive Radar”. In: *IEEE Access* 4 (2016), pp. 1–10 (cit. on p. 39).
- [Cui+22] Zhuo-Xu Cui et al. *Deep unfolding as iterative regularization for imaging inverse problems*. 2022. arXiv: arXiv:2211.13452v1 (cit. on p. 7).
- [Dav+21] Alessandro Davoli et al. “Machine Learning and Deep Learning Techniques for Colocated MIMO Radars: A Tutorial Overview”. In: *IEEE Access* 9 (2021) (cit. on pp. 2, 3, 6, 7, 9).
- [de 17] Fabian de Ponte Müller. “Survey on ranging sensors and cooperative techniques for relative positioning of vehicles”. In: *Sensors (Switzerland)* 17.2 (2017), pp. 1–27 (cit. on p. 4).
- [Dud+10] Manuel Dudek et al. “Millimeter wave FMCW radar system simulations including a 3D ray tracing channel simulator”. In: *Asia-Pacific Microwave Conference Proceedings, APMC*. IEEE, 2010, pp. 1665–1668 (cit. on p. 96).

BIBLIOGRAPHY

- [Ed+13] Khalid El-darymli et al. "Target detection in synthetic aperture radar imagery : a state-of-the-art survey". In: *JARS* 7 (2013), pp. 1–36 (cit. on p. 81).
- [EGA19] Baris Erol, Sevgi Z. Gurbuz, and Moeness G. Amin. "GAN-based synthetic radar micro-doppler augmentations for improved human activity recognition". In: *2019 IEEE Radar Conference* (2019), pp. 0–4 (cit. on p. 97).
- [Elb20] Ahmet M. Elbir. "DeepMUSIC: Multiple Signal Classification via Deep Learning". In: *IEEE Sensors Letters* 4.4 (2020). arXiv: 1912.04357 (cit. on pp. 7, 16, 109, 112, 117, 127).
- [Eng19] G Englebiene. *Machine Learning Pattern Recognition*. 2019 (cit. on pp. 60, 71, 74, 75, 88).
- [Enh+21] Ida Merete Enholm et al. "Artificial Intelligence and Business Value: a Literature Review". In: *Springer Information Systems Frontiers* (2021), pp. 1709–1734 (cit. on p. 1).
- [FB19] Konstantinos Fatseas and Marco J. G. Bekooij. "Neural Network Based Multiple Object Tracking for Automotive FMCW Radar". In: *2019 International Radar Conference (RADAR2019)* (2019), pp. 1–6 (cit. on pp. 4, 5, 9).
- [FB21] Konstantinos Fatseas and Marco J.G. Bekooij. "Weakly Supervised Semantic Segmentation for Range-Doppler Maps". In: *18th European Radar Conference, EuRAD 2021* April (2021), pp. 70–73 (cit. on pp. 2, 6, 9).
- [Fuc+21] Alexander Fuchs et al. "Complex-valued Convolutional Neural Networks for Enhanced Radar Signal Denoising and Interference Mitigation". In: *IEEE Radar Conference* (2021), pp. 12–17 (cit. on pp. 7, 80).
- [FWG19] Jonas Fuchs, Robert Weigel, and Markus Gardill. "Single-snapshot direction-of-arrival estimation of multiple targets using a multi-layer perceptron". In: *2019 IEEE MTT-S International Conference on Microwaves for Intelligent Mobility, ICMIM 2019* (2019), pp. 0–3 (cit. on pp. 108, 113, 117).
- [Gam20] Jonah Gamba. *Radar Signal Processing for Autonomous Driving*. Springer, 2020, pp. 1–142 (cit. on pp. 28, 30–32, 35, 37, 39, 40, 44, 45, 47–50, 54).
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016 (cit. on pp. 57, 60–63, 65–67, 70–72, 74, 96, 99).

- [Gla17] Tobias Glasmachers. “Limits of End-to-End Learning”. In: *Proceedings of Machine Learning Research* 77 (2017), pp. 17–32 (cit. on p. 7).
- [GLR21] Shengguo Ge, Kuo Li, and Siti Nurulain Binti Mohd Rum. “Deep Learning Approach in DOA Estimation: A Systematic Literature Review”. In: *Mobile Information Systems* 2021 (2021) (cit. on p. 7).
- [Goo+14] Ian J. Goodfellow et al. *Generative adversarial nets*. Tech. rep. 2014, pp. 1–9. arXiv: 1406.2661 (cit. on pp. 18, 69, 71, 96, 99).
- [Ham99] J. K. Hammond. “Fundamentals of Signal Processing”. In: *Modal Analysis and Testing*. 1999, pp. 35–52 (cit. on pp. 2, 10).
- [He+16] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (2016), pp. 770–778 (cit. on pp. 5, 68, 69, 112, 114, 129, 130).
- [HI22] Lukas Hedegaard and Alexandros Iosifidis. “Continual 3D Convolutional Neural Networks for Real-time Processing of Videos”. In: *Computer Vision (ECCV)* (2022), pp. 369–385. arXiv: arXiv:2106.00050v3 (cit. on p. 157).
- [Hin12] G Hinton. *Neural networks for machine learning*. 2012 (cit. on pp. 61, 62).
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780 (cit. on p. 157).
- [HVD15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: (2015), pp. 1–9. arXiv: 1503.02531 (cit. on p. 72).
- [HY10] P. Häcker and B. Yang. “Single snapshot DOA estimation”. In: *Advances in Radio Science* 8 (2010), pp. 251–256 (cit. on pp. 15, 19, 37, 47, 48, 50, 54, 108, 110, 126).
- [HY19] Gor Hakobyan and Bin Yang. “High-Performance Automotive Radar: A review of signal processing algorithms and modulation schemes”. In: *IEEE Signal Processing Magazine* 36. September (2019), pp. 32–44 (cit. on p. 9).
- [JHS22] Wouter Jansen, Nico Huebel, and Jan Steckel. *Physical LiDAR Simulation in Real-Time Engine*. 2022. arXiv: arXiv:2208.10295v3 (cit. on p. 17).

- [Ji+13] Shuiwang Ji et al. "3D convolutional neural networks for human action recognition". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.1 (2013), pp. 221–231 (cit. on p. 66).
- [Jor+22] James Jordon et al. *Synthetic Data - what, why and how?* 2022. arXiv: arXiv:2205.03257v1 (cit. on pp. 17, 96).
- [Jos+19] Sushil Kumar Joshi et al. "Range-Doppler Based CFAR Ship Detection with Automatic Training Data Selection". In: *Remote Sensing* 11.11 (2019), pp. 1–36 (cit. on p. 28).
- [JYB16] Ahsan Jalil, Hassan Yousaf, and Muhammad Iram Baig. "Analysis of CFAR techniques". In: *Proceedings of 2016 13th International Bhurban Conference on Applied Sciences and Technology, IBCAST 2016* (2016), pp. 654–659 (cit. on pp. 13, 44, 45).
- [KB15] Diederik P Kingma and Jimmy Lei Ba. "ADAM: A Method for Stochastic Optimization". In: *International Conference of Learning Representations* (2015), pp. 1–15. arXiv: arXiv:1412.6980v9 (cit. on pp. 61, 62).
- [KBN14] Eyal Katz and Yeheskel Bar-Ness. "Comparison of SNR and Peak-SNR (PSNR) as performance measures and signals for peak-limited two-dimensional (2D) pixelated optical wireless communication". In: *Asilomar Conference on Signals, Systems and Computers* (2014), pp. 1880–1884 (cit. on p. 75).
- [Kir+21] Serkan Kiranyaz et al. "1D convolutional neural networks and applications: A survey". In: *Mechanical Systems and Signal Processing* 151 (2021), pp. 1–20. arXiv: 1905.03554 (cit. on p. 66).
- [KLA21] Tero Karras, Samuli Laine, and Timo Aila. "A Style-Based Generator Architecture for Generative Adversarial Networks". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.12 (2021), pp. 4217–4228. arXiv: 1812.04948 (cit. on p. 100).
- [KN21] M. Kathiresh and R. Neelaveni. *Automotive Embedded Systems*. 2021, pp. 1–231 (cit. on pp. 1, 2).
- [Kok14] Don Koks. "How to Create and Manipulate Radar Range-Doppler Plots". In: *Defense Science and Technology Organization* (2014), p. 95 (cit. on p. 12).
- [KSH12] Alex Sutskever, Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems* 25 (2012) (cit. on p. 65).

- [KV96] Hamid Krim and Mats Viberg. "Two Decades of Array Signal Processing Research". In: *IEEE Signal Processing Magazine* (1996), pp. 67–94 (cit. on pp. 9, 15, 16, 35–37, 46–52, 139).
- [Leg+23] Ronan Legin et al. "Beyond Gaussian Noise : A Generalized Approach to Likelihood Analysis with Non- Gaussian Noise". In: *The Astrophysical Journal Letters* (2023), pp. 1–7 (cit. on p. 10).
- [LHG22] ChiYan Lee, Hideyuki Hasegawa, and Shangce Gao. "Complex-Valued Neural Networks : A Comprehensive Survey". In: *IEEE/CAA Journal of Automatica Sinica* 9.8 (2022), pp. 1406–1426 (cit. on p. 157).
- [Li+19] Chunmei Li et al. "The Impact of Data Quality on Neural Network Models". In: *Cyber Security Intelligence and Analytics (CSIA 2019)* (2019), pp. 657–665 (cit. on p. 158).
- [Li+21] Zewen Li et al. "A Survey of Convolutional Neural Networks : Analysis , Applications , and Prospects". In: *IEEE Trans. on Neural Networks and Learning Systems* (2021), pp. 1–21 (cit. on p. 5).
- [Liu+21] Chengliang Liu et al. "Single Snapshot DOA Estimation Based on Spatial Smoothing MUSIC and CNN". In: *IEEE International Conference on Signal Processing, Communications and Computing 3* (2021) (cit. on p. 127).
- [LL18] Aifei Liu and Guisheng Liao. "An eigenvector based method for estimating DOA and sensor gain-phase errors". In: *Digital Signal Processing: A Review Journal* 79.12 (2018), pp. 116–124 (cit. on p. 139).
- [LM04] Nadav Levanon and Eli Mozeson. *Radar Signals*. 2004, pp. 1–402 (cit. on pp. 3, 7).
- [LZY18] Zhang Meng Liu, Chenwei Zhang, and Philip S. Yu. "Direction-of-Arrival Estimation Based on Deep Neural Networks with Robustness to Array Imperfections". In: *IEEE Transactions on Antennas and Propagation* 66.12 (2018), pp. 7315–7327 (cit. on p. 138).
- [Ma+23] He Ma et al. "A new array extension configuration method based on nesting and minimum redundancy". In: *IET Radar, Sonar and Navigation* (2023) (cit. on pp. 37, 39).
- [Mac16] Bohdan Macukow. *Neural Networks – State of Art, Brief History, Basic Models and Architecture*. 2016 (cit. on p. 5).

- [Mar+89] K. A. Marko et al. "Automotive control system diagnostics using neural nets for rapid pattern classification of large data sets". In: *Proceedings of the International Joint Conference on Neural Networks (IJCNN)* 2 (1989), pp. 13–16 (cit. on p. 5).
- [Mof68] Alan T. Moffet. "Minimum-Redundancy Linear Arrays". In: *IEEE Transactions on Antennas and Propagation* 16.2 (1968), pp. 172–175 (cit. on pp. 14, 29, 37).
- [Mon+19] Federico Monti et al. *Fake News Detection on Social Media using Geometric Deep Learning*. 2019. arXiv: 1902.06673 (cit. on p. 1).
- [Mor+18] Niago Moreira et al. "Deep Convolutional Autoencoder for EEG Noise Filtering". In: *IEEE BIBM*. 2018, pp. 2605–2612 (cit. on p. 80).
- [Mus+22] Anthony M Musolf et al. "What makes a good prediction? Feature importance and beginning to open the black box of machine learning in genetics". In: *Springer Human Genetics* 141.9 (2022), pp. 1515–1528 (cit. on p. 7).
- [MZS22] Yugang Ma, Yonghong Zeng, and Sumei Sun. "A Deep Learning Based Super Resolution DoA Estimator With Single Snapshot MIMO Radar Data". In: *IEEE Transactions on Vehicular Technology* 71.4 (2022), pp. 4142–4155 (cit. on p. 138).
- [Nas+14] A. T. Nasrabadi et al. "Investigating the PSNR calculation methods for video sequences with source and channel distortions". In: *IEEE BMSB* 0 (2014), pp. 1–4 (cit. on p. 87).
- [Ols21] Karin Olson. "What Are Data?" In: *Qualitative Health Research* (2021), pp. 1–3 (cit. on p. 2).
- [Pan+19] Zhaoqing Pan et al. "Recent Progress on Generative Adversarial Networks (GANs): A Survey". In: *IEEE Access* 7 (2019), pp. 36322–36333 (cit. on pp. 96, 100).
- [Pat+17] Sujeet Patole et al. "Automotive Radars: A review of signal processing techniques". In: *IEEE Signal Processing Magazine* March (2017) (cit. on pp. 3, 4, 9, 30).
- [Pic04] Massimo Piccardi. "Background subtraction techniques: a review". In: *IEEE SMC* (2004), pp. 3099–3104 (cit. on p. 84).
- [PLK22] Seonguk Park, Seungeui Lee, and Nojun Kwak. "Range-Doppler Map Augmentation by Generative Adversarial Network for Deep UAV Classification". In: *IEEE Radar Conference* (2022) (cit. on pp. 7, 97).

- [PME11] Gaurav Pandey, James R McBride, and Ryan M Eustice. "Ford Campus Vision and LiDAR Data Set". In: *International Journal of Robotics Research* 30.13 (2011), pp. 1543–1552 (cit. on p. 7).
- [PML23] Ivan Paduano, Andrea Mileto, and Egidio Lofrano. "A Perspective on AI-Based Image Analysis and Utilization Technologies in Building Engineering: Recent Developments and New Directions". In: *Springer Buildings* 13.5 (2023) (cit. on p. 1).
- [Qi+16] Charles R. Qi et al. "Volumetric and multi-view CNNs for object classification on 3D data". In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (2016), pp. 5648–5656. arXiv: 1604.03265 (cit. on p. 6).
- [Qi+17] Charles R Qi et al. *PointNet ++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space*. 2017. arXiv: arXiv:1706.02413v1 (cit. on p. 2).
- [QY16] Weichao Qiu and Alan Yuille. "UnrealCV: Connecting Computer Vision to Unreal Engine". In: *European Conference on Computer Vision* 9915 (2016), pp. 909–916. arXiv: 1603.06937 (cit. on pp. 17, 96).
- [Ram+21] Aditya Ramesh et al. "Zero-Shot Text-to-Image Generation". In: *Proceedings of Machine Learning Research* 139 (2021), pp. 8821–8831 (cit. on p. 1).
- [Rao18] Sandeep Rao. *MIMO Radar*. Tech. rep. 2018, pp. 1–13 (cit. on pp. 9, 37, 39).
- [Ras14] Sebastian Raschka. *An Overview of General Performance Metrics of Binary Classifier Systems*. 2014. arXiv: 1410.5330 (cit. on pp. 74, 75).
- [RB23] Dani Raphaeli and Igal Bilik. "Challenges in Automotive MIMO Radar Calibration in Anechoic Chamber". In: *IEEE Transactions on Aerospace and Electronic Systems* 59.5 (2023), pp. 6205–6214 (cit. on p. 142).
- [RB97] Bill M. Radich and Kevin M. Buckley. "Single-snapshot DOA estimation and source number detection". In: *IEEE Signal Processing Letters* 4.4 (1997), pp. 109–111 (cit. on p. 127).
- [Red+16a] Joseph Redmon et al. "You Only Look Once : Unified, Real-Time Object Detection". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 1–10 (cit. on p. 2).

BIBLIOGRAPHY

- [Red+16b] Joseph Redmon et al. "You Only Look Once: Unified, Real-Time Object Detection". In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (2016), pp. 779–788. arXiv: 1506.02640 (cit. on p. 157).
- [Ric05] Mark A Richards. *Fundamentals of Radar Signal Processing*. 2005, pp. 344–363 (cit. on p. 80).
- [Ric14] Mark A. Richards. *Fundamentals of Radar Signal Processing*. Second. 2014, pp. 1–894 (cit. on pp. 28, 30, 33, 40, 43–45, 74, 85).
- [RM19] Anupama Govinda Raj and James H. McClellan. "Super-resolution DOA Estimation for Arbitrary Array Geometries Using a Single Noisy Snapshot". In: *IEEE International Conference on Acoustics, Speech and Signal Processing* 2019-May.1 (2019), pp. 4145–4149. arXiv: 1903.10876 (cit. on p. 126).
- [RSC66] J. Ruze, F. I. Sheftman, and D. A. Cahlander. "Radar Ground-Clutter Shields". In: *Proceedings of the IEEE* 54.9 (1966), pp. 1171–1183 (cit. on p. 13).
- [Sch14] Jürgen Schmidhuber. "Deep Learning in neural networks: An overview". In: *Technical Report* (2014) (cit. on pp. 60, 63–65, 67, 71).
- [Sch15] Jürgen Schmidhuber. "Deep learning in neural networks : An overview". In: *Elsevier Neural Networks* 61 (2015), pp. 85–117 (cit. on p. 5).
- [Sch91] Louis L. Scharf. *Statistical Signal Processing*. 1991, pp. 1–521 (cit. on pp. 10, 13, 43).
- [Sen+20] Linda Senigagliesi et al. "People Walking Classification using Automotive Radar". In: *Electronics (MDPI)* (2020), pp. 1–15 (cit. on p. 33).
- [SER87] Merrill Skolnik, Jerry L Eaves, and Edward K Reedy. *Principles of Modern Radar*. 1987, pp. 1–708 (cit. on pp. 2, 42–44).
- [SG71] Naresh K. Sinha and Michael P. Griscik. "A Stochastic Approximation Method". In: *IEEE Transactions on Systems, Man and Cybernetics* 1.4 (1971), pp. 338–344 (cit. on p. 61).
- [SG99] Chris Stauffer and W. E L Grimson. "Adaptive background mixture models for real-time tracking". In: *IEEE CVPR* 2 (1999), pp. 246–252 (cit. on p. 84).

- [She+19] Jiaquan Shen et al. "Vehicle Detection in Aerial Images Based on Lightweight Deep Convolutional Network and Generative Adversarial Network". In: *IEEE Access* 7 (2019), pp. 148119–148130 (cit. on p. 99).
- [Sij+96] J. Sijbers et al. "Quantification and improvement of the signal-to-noise ratio in a magnetic resonance image acquisition procedure". In: *Magnetic Resonance Imaging* 14.10 (1996), pp. 1157–1163 (cit. on p. 75).
- [Ska18] Sandro Skansi. *Introduction to Deep Learning*. Springer, 2018 (cit. on pp. 7, 60, 71).
- [SN13] Zoltán Szalay and Lajos Nagy. "ULA and MRA beamforming using conventional and particle filter based algorithms". In: *International Conference on Applied Electromagnetics and Communications (ICE-Com)* (2013), pp. 8–11 (cit. on pp. 14, 30, 37).
- [SOG18] Mehmet S Seyfioglu, Ahmet M Ozbayoglu, and Sevgi Z Gurbuz. "Deep convolutional autoencoder for radar-based classification of similar aided and unaided human activities". In: *IEEE Trans. Aerosp. Electron. Syst* 54.4 (2018), pp. 1709–1723 (cit. on p. 80).
- [Som+99] Gerd Sommerkom et al. "Reduction of DoA Estimation Errors Caused by Antenna Array Imperfections". In: *29th European Microwave Conference* (1999), pp. 287–290 (cit. on p. 142).
- [SR10] Christoph Schroeder and Hermann Rohling. "X-band FMCW radar system with variable chirp duration". In: *IEEE National Radar Conference* (2010), pp. 1255–1259 (cit. on p. 30).
- [SS90] Petre Stoica and Kenneth C Sharman. "Maximum Likelihood Methods for Direction-of-Arrival Estimation". In: *IEEE TRANSACTIONS ON ACOUSTICS, SPEECH, AND SIGNAL PROCESSING* 38.7 (1990), pp. 1132–1143 (cit. on pp. 108, 126).
- [Sto92] A G Stove. "Linear FMCW radar techniques". In: *IEE Proceedings F* 139.5 (1992), pp. 343–350 (cit. on p. 28).
- [SY07] Michael Schoor and Bin Yang. "High-resolution angle estimation for an automotive FMCW radar sensor". In: *Proceedings International Radar Symposium (IRS)* (2007) (cit. on p. 14).
- [TBL18] Michael Tschannen, Olivier Bachem, and Mario Lucic. "Recent Advances in Autoencoder-Based Representation Learning". In: *NeurIPS*. 2018, pp. 1–26 (cit. on p. 81).

- [Tex18] Texas Instrument. *Traffic Monitoring Object Detection and Tracking Reference Design Using Single-Chip mmWave Radar*. Tech. rep. April. 2018, pp. 1–51 (cit. on p. 81).
- [TF09] Engin Tuncer and Benjamin Friedlander. *Classical and Modern Direction-of-Arrival Estimation*. Elsevier, 2009, pp. 1–429 (cit. on pp. 14, 16, 35, 37, 39, 46–48, 50, 138, 139).
- [Tho+19] Hugues Thomas et al. “KPConv : Flexible and Deformable Convolution for Point Clouds”. In: *IEEE International Conference on Computer Vision* (2019), pp. 6411–6420 (cit. on p. 6).
- [TZQ22] Xiaolin Tang, Zhiqiang Zhang, and Yechen Qin. “On-Road Object Detection and Tracking Based on Radar and Vision Fusion : A Review”. In: *IEEE Intelligent Transportation Systems Magazine* 14.October (2022), pp. 103–128 (cit. on pp. 2, 4, 9).
- [Van01] Harry L. Van Trees. *Detection, Estimation, and Modulation Theory*. 2001, pp. 1 –686 (cit. on p. 12).
- [Van02] Harry L. Van Trees. *Optimum Array Processing*. Vol. 4. 2002, pp. 0–471 (cit. on pp. 14, 29, 108, 122).
- [VBC14] Francois Vincent, Olivier Besson, and Eric Chaumette. “Approximate maximum likelihood estimation of two closely spaced sources”. In: *Signal Processing* 97 (2014), pp. 83–90 (cit. on pp. 14, 15, 48, 108).
- [Wan+18] Ting Chun Wang et al. “High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs”. In: *IEEE Conference on Computer Vision and Pattern Recognition* (2018), pp. 8798–8807. arXiv: 1711.11585 (cit. on pp. 100, 157).
- [Wan+20] Xin Wang et al. “The evolution of LiDAR and its application in high precision measurement”. In: *IOP Conference Series: Earth and Environmental Science* 502.1 (2020) (cit. on p. 2).
- [Win07] Volker Winkler. “Range Doppler detection for automotive FMCW radars”. In: *2007 European Radar Conference, EURAD* October (2007), pp. 166–169 (cit. on p. 13).
- [Wol09] Christian Wolff. *Radartutorial*. 2009 (cit. on p. 2).
- [Wol18] Christian Wolff. *Frequency-Modulated Continuous-Wave Radar*. Tech. rep. 2018, pp. 1–8 (cit. on p. 30).

- [Won+16] Sebastien C. Wong et al. "Understanding Data Augmentation for Classification: When to Warp?" In: *2016 International Conference on Digital Image Computing: Techniques and Applications, DICTA 2016* (2016), pp. 1–6. arXiv: 1609.08764 (cit. on pp. 8, 97).
- [WSJ08] Qiang Wang, Yi Shen, and Jing Jin. *Performance evaluation of image fusion techniques*. Elsevier Ltd, 2008, pp. 469–492 (cit. on pp. 75, 76).
- [WZP20] Xizhao Wang, Yanxia Zhao, and Farhad Pourpanah. "Recent advances in deep learning". In: *International Journal of Machine Learning and Cybernetics* 11.4 (2020), pp. 747–750 (cit. on p. 5).
- [Yan+22] Zihan Yang et al. "DeepDOA: A Novel Deep Learning-Based Method for DOA Superresolution in a Single Snapshot". In: *2022 IEEE 5th International Conference on Electronics Technology, ICET 2022* (2022), pp. 703–706 (cit. on p. 138).
- [YHB03] Daren Yu, Qinghua Hu, and Wen Bao. "Combining Multiple Neural Networks for Classification Based on Rough Set Reduction". In: *IEEE International Conference on Neural Networks and Signal Processing* 1 (2003), pp. 543–548 (cit. on p. 83).
- [You+22] Jihwan Youn et al. "Circular Convolutional Learned ISTA for Automotive Radar DOA Estimation". In: *European Radar Conference (EuRAD)* (2022), pp. 273–276 (cit. on pp. 8, 109, 158).
- [Zha+17] Bendong Zhao et al. "Convolutional Neural Networks for Time Series Classification". In: *JSEE* 28.1 (2017), pp. 162–169 (cit. on p. 84).
- [Zha+19] Han Zhang et al. "StackGAN++: Realistic Image Synthesis with Stacked Generative Adversarial Networks". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41.8 (2019), pp. 1947–1962. arXiv: 1710.10916 (cit. on p. 99).
- [Zha+21a] Aston Zhang et al. "Dive Into Deep Learning". In: *arXiv preprint* (2021). arXiv: 2106.11342 (cit. on pp. 60, 68, 69, 71, 73, 99).
- [Zha+21b] Hongchi Zhang et al. "Performance evaluation and parameter optimization of sparse Fourier transform". In: *Signal Processing* 179 (2021) (cit. on p. 10).
- [ZHL07] Jiankui Zeng, Zishu He, and Benyong Liu. "Maximum likelihood DOA estimation using particle swarm optimization algorithm". In: *CIE International Conference of Radar Proceedings* 5 (2007), pp. 0–3 (cit. on p. 127).

Bibliography

- [Zho+22] Yong Zhou et al. “Review on Millimeter-Wave Radar and Camera Fusion Technology”. In: *Sustainability (MDPI)* 14.9 (2022), pp. 1–32 (cit. on p. 2).

Publications

- [LB20a] Marcio L. Lima de Oliveira and Marco J. G. Bekooij. “Deep convolutional autoencoder applied for noise reduction in range-Doppler maps of FMCW radars”. In: *2020 IEEE International Radar Conference (RADAR 2020)* (2020), pp. 630–635 (cit. on pp. 5, 6, 20, 25, 79, 97, 98, 101).
- [LB20b] Marcio L. Lima de Oliveira and Marco J. G. Bekooij. “Generating Synthetic Short-Range FMCW Range-Doppler Maps Using Generative Adversarial Networks and Deep Convolutional Autoencoders”. In: *IEEE Radar Conference 2020 (RadarConf20)* (2020) (cit. on pp. 20, 25, 95, 97).
- [LB22a] Marcio L. Lima de Oliveira and Marco J. G. Bekooij. “Deep-Mle: Fusion Between a Neural Network and Mle for a Single Snapshot Doa Estimation”. In: *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing* May (2022), pp. 3673–3677 (cit. on pp. 22, 25, 51, 53, 109, 125).
- [LB22b] Marcio L. Lima de Oliveira and Marco J. G. Bekooij. “ResNet Applied for a Single-Snapshot DOA Estimation”. In: *2022 IEEE Radar Conference (RadarConf22)* (2022), pp. 1–6 (cit. on pp. 5, 16, 22, 25, 107, 109).
- [LB23] Marcio L Lima de Oliveira and Marco J. G. Bekooij. “Fusion Model Using a Neural Network and MLE for a Single Snapshot DOA Estimation with Imperfection Mitigation”. In: *2023 IEEE International Radar Conference (RADAR 2023)* (2023) (cit. on pp. 22, 25, 137).

ISBN 978-90-365-6600-1



9 789036 566001